

INTRODUCTION TO MACHINE LEARNING

Why Machine Learning is Popular

1. **High Data Volume:** Large companies like Facebook, Twitter, and YouTube generate huge amounts of data, which doubles annually.
2. **Reduced Storage Costs:** Declining hardware and storage costs make it easier to capture, store, and process vast amounts of data.
3. **Advanced Algorithms:** The development of complex algorithms, especially in deep learning, enables more powerful machine learning applications.

The Knowledge Pyramid

1. **Data:** Basic facts and raw numbers. Organizations store vast amounts of data from sources like databases and warehouses.
2. **Information:** Processed data revealing patterns or associations. For instance, analyzing sales data to determine the best-selling product.
3. **Knowledge:** Condensed information, such as historical patterns and future trends. Extracting knowledge from data is crucial for decision-making.
4. **Intelligence:** Applied knowledge. It represents actionable insights, such as strategies derived from knowledge.
5. **Wisdom:** The highest level, where intelligence evolves into maturity and sound judgment, typically exhibited by humans.

Objective of Machine Learning

Machine learning processes archival data to:

- Make better decisions.
- Design new products.
- Improve business processes.
- Build decision support systems.

What is Machine Learning?

Machine learning is a sub-field of AI that allows computers to learn without explicit programming. As Arthur Samuel defined: **“Machine learning is the field of study that gives computers the ability to learn without being explicitly programmed.”**

In conventional programming, we teach computers how to perform tasks step-by-step. However, real-world problems like image recognition or complex games require systems that can learn from data directly.

Evolution of Machine Learning

- Early systems like **expert systems** (e.g., MYCIN for medical diagnosis) relied on human rules and logic, but they didn't exhibit true intelligence.
- Machine learning evolved with **data-driven systems**, focusing on learning from data to automatically predict unknown outcomes.

Learning System

- **Human Learning (Fig. 1.2a):** Humans make decisions based on experience.
- **Machine Learning (Fig. 1.2b):** Machines create models from data patterns and use these models for prediction, akin to human experience.

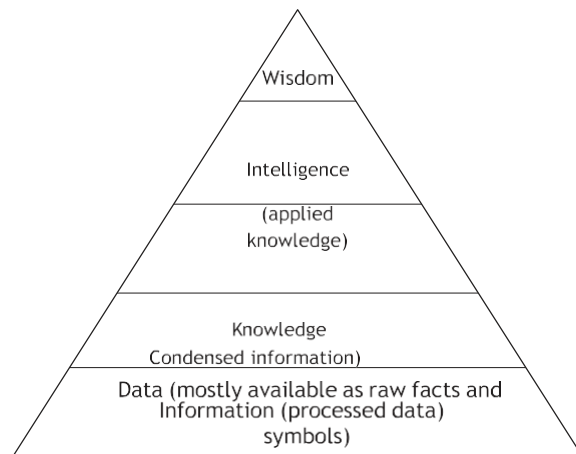


Figure 1.1: The Knowledge Pyramid

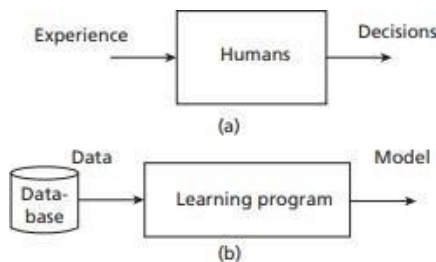


Figure 1.2: (a) A Learning System for Humans (b) A Learning System for Machine Learning

Data Quality and Learning

- The **quality of data** directly impacts the **quality of experience** and, ultimately, the **quality of learning systems**.

Statistical Learning

- In **statistical learning**, the relationship between input **x** and output **y** is modeled as: **$y = f(x)$**
 - **f** is the **learning function** mapping inputs to outputs.
- In **machine learning**, this is referred to as the **mapping of input to output**.

Model in Machine Learning

- A **model** is a summary of raw data, structured into a representation for decision-making.
- Models can be in different forms, such as:
 1. **Mathematical equations**: e.g., linear regression formula
 2. **Relational diagrams**: like **decision trees** or **graphs**
 3. **Logical rules**: like **if/else** rules (e.g., rule-based spam filters)
 4. **Clusters**: for grouping data (e.g., **k-means clustering**)

Patterns vs. Models

- **Pattern**: Local; applicable to specific parts of the data.
- **Model**: Global; fits the entire dataset.
 - **Example**: A model trained to detect spam can be used to predict whether an email is spam or not.

Tom Mitchell's Definition of Machine Learning

Tom Mitchell's famous definition:
"A computer program is said to learn from experience E, with respect to task T and some performance measure P, if its performance on T, measured by P, improves with experience E."

- **Experience (E)**: Data used to learn (e.g., thousands of images to train an object detection model).
- **Task (T)**: The job the machine does (e.g., detecting objects in images).
- **Performance measure (P)**: How well the machine performs the task (e.g., precision, recall).

Example:

- **Task (T)**: Detecting an object in images.
-

- **Experience (E):** Training data containing thousands of labeled images.
- **Performance measure (P):** The system is evaluated by how accurately it detects objects, using metrics like **precision** and **recall**. Improvements can be made if the system underperforms.

Human vs. Machine Experience

- **Human Experience:** Gained through learning, observing, imitation, and trial & error.
 - Example: Learning how to ride a bike through practice and observation.
- **Machine Experience:** Gained through **data processing** and model building:
 1. **Data Collection:** Gathering data from the environment (e.g., images, text, or numbers).
 2. **Abstraction:** Extracting key features from the data (e.g., identifying basic features of an elephant: trunk, ears).
 3. **Generalization:** Turning abstraction into an actionable form, like forming rules (heuristics) from past experiences.
 - Example: A self-driving car generalizes rules about stopping at red lights.
 4. **Heuristics:** Actionable “**rules of thumb**” that guide decisions based on prior experience.
 - **Example:** A heuristic rule: If you see a red light, stop.
 - Heuristics can sometimes fail but are typically effective.

Evaluation & Course Correction

- **Heuristics:** Often work, but sometimes fail due to limitations (it’s a general rule, not a certainty).
 - **Example:** If someone runs when sensing danger, it’s an automatic response based on past experience (heuristics).
- **Evaluation:** Assesses the effectiveness of the model or heuristic. If the model underperforms, we use **evaluation measures** to **adjust** and **improve** it.

1.1 MACHINE LEARNING IN RELATION TO OTHER FIELDS

Machine learning uses the concepts of Artificial Intelligence, Data Science, and Statistics primarily. It is the resultant of combined ideas of diverse fields.

1.3.1 Machine Learning and Artificial Intelligence

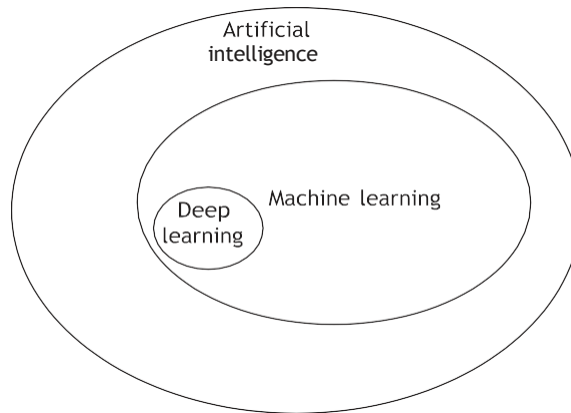


Figure 1.3: Relationship of AI with Machine Learning

Artificial Intelligence (AI) is a broad field focused on creating systems (called "intelligent agents") that can perform tasks autonomously, such as robots, humans, or other systems. The early goal of AI was ambitious: to create intelligent systems that could think and act like humans, focusing on logic and reasoning.

However, AI faced several challenges and periods of slow progress, called **AI winters**, where enthusiasm and funding declined. AI's resurgence came with the rise of **data-driven systems**—models that learn by finding patterns in data. This led to the development of **Machine Learning (ML)**, a key branch of AI.

Machine Learning aims to extract patterns from data to make predictions. Instead of explicitly programming systems for every possible scenario, ML algorithms "learn" from examples (training data) and can handle new, unseen situations. Machine learning includes various techniques like **reinforcement learning**, where agents learn by interacting with their environment.

Relationship Between AI and Machine Learning:

- AI is the broader field aiming to create intelligent agents.
 - ML is a subfield of AI that focuses on learning from data.
 - **Deep Learning**, a subset of ML, uses **neural networks** inspired by the human brain to build models. These networks consist of layers of interconnected units ("neurons") that process information in a way that mimics how the brain works, and they are especially useful for tasks like image and speech recognition.
-

1.3.2 Machine Learning, Data Science, Data Mining, and Data Analytics

Data Science is an umbrella term that covers various fields related to working with data. It involves gathering, processing, analyzing, and drawing insights from data. **Machine learning** starts with data, which makes it closely linked to data science. Here's how machine learning connects to related fields:

Big Data:

Big data is part of data science and refers to massive volumes of data generated by companies like Facebook, Twitter, and YouTube. It deals with three key characteristics:

1. **Volume:** The sheer amount of data being generated.
2. **Variety:** Data comes in many forms—text, images, videos, etc.
3. **Velocity:** The speed at which data is generated and processed.

Big data is essential for machine learning because many algorithms rely on large datasets for training. For example, deep learning (a subfield of ML) uses big data for tasks like image recognition and language translation.

Data Mining:

Data mining originally came from business applications. It's like "mining" for valuable information hidden in large datasets. While data mining and machine learning overlap significantly, the distinction is:

- **Data Mining:** Focuses on discovering hidden patterns in data.
- **Machine Learning:** Uses those patterns to make predictions.

Data Analytics:

Another branch of data science is **data analytics**, which aims to extract useful insights from raw data. There are different types of analytics, such as **predictive analytics**, which forecasts future events based on past data. Machine learning plays a major role in predictive analytics since many of its algorithms are used to make predictions.

Pattern Recognition:

Pattern recognition is an engineering field that uses machine learning algorithms to detect and classify patterns. While it's often considered a specific application of machine learning, it has its own identity as a field, dealing with tasks like facial recognition or speech analysis.

These relations are summarized in Figure 1.4.

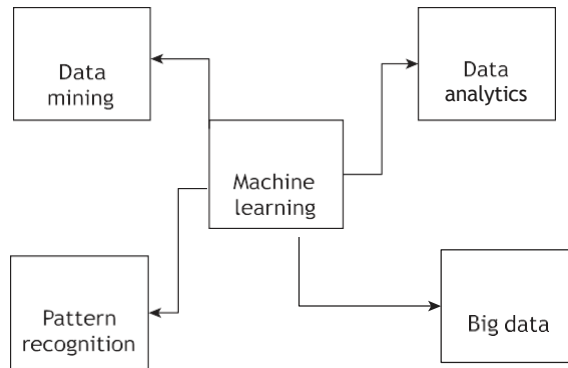


Figure 1.4: Relationship of Machine Learning with Other Major Fields

1.3.3 Machine Learning and Statistics

1. Statistics:

- **Definition:** A branch of mathematics focused on analyzing and interpreting data to uncover patterns and relationships.
- **Key Features:**
 - **Hypothesis-driven:** Starts with a hypothesis and tests it through experiments.
 - **Assumptions:** Requires strict assumptions (e.g., normal distribution, independence of variables).
 - **Mathematical Models:** Uses complex equations (e.g., regression, ANOVA) to explain data.
 - **Knowledge Required:** Strong statistical background needed for analysis and interpretation.
 - **Goal:** Primarily concerned with verifying relationships and patterns in data.

2. Machine Learning (ML):

- **Definition:** A branch of AI focused on building models that learn from data to make predictions or decisions without being explicitly programmed.
- **Key Features:**
 - **Data-driven:** Focuses on learning from data patterns for predictions.
 - **Less Assumptions:** Fewer restrictions on data (e.g., can handle non-normal data).
 - **Automation:** Emphasizes using tools and algorithms to automate the learning process.

- **Flexibility:** Works well with large, complex datasets; adaptable to different scenarios.
- **Goal:** Makes predictions based on learned patterns, often without needing detailed statistical knowledge.

1.2 TYPES OF MACHINE LEARNING

What does the word 'learn' mean? Learning, like adaptation, occurs as the result of interaction of the program with its environment. There are four types of machine learning as shown in Figure 1.5.

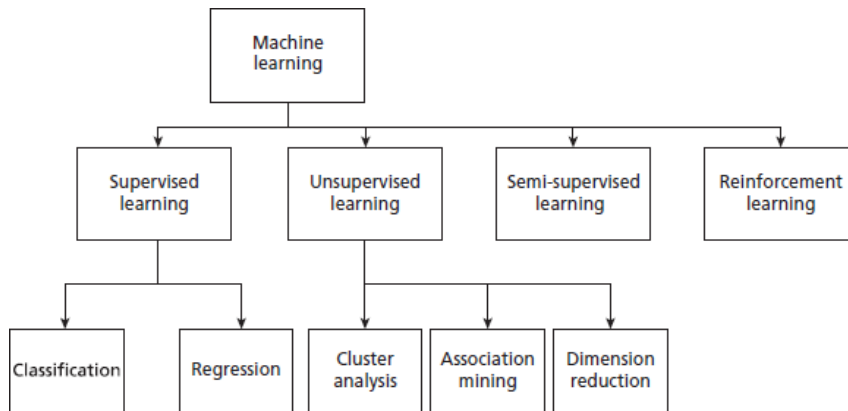


Figure 1.5: Types of Machine Learning

Before discussing the types of learning, it is necessary to discuss about data.

Labelled and Unlabelled Data: Data is a raw fact. Normally, data is represented in the form of a table. Data also can be referred to as a data point, sample, or an example. Each row of the table represents a data point. Features are attributes or characteristics of an object. Normally, the columns of the table are attributes. Out of all attributes, one attribute is important and is called a label. Label is the feature that we aim to predict. Thus, there are two types of data – labelled and unlabelled.

Labelled Data To illustrate labelled data, let us take one example dataset called Iris flower dataset or Fisher's Iris dataset. The dataset has 50 samples of Iris – with four attributes, length and width of sepals and petals. The target variable is called class. There are three classes – Iris setosa, Iris virginica, and Iris versicolor.

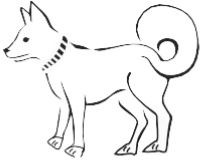
The partial data of Iris dataset is shown in Table 1.1.

Table 1.1: Iris Flower Dataset

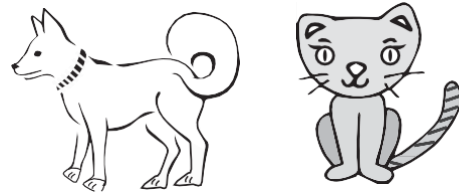
S.No.	Length ofPetal	Width ofPetal	Length ofSepal	Width ofSepal	Class
1.	5.5	4.2	1.4	0.2	Setosa
2.	7	3.2	4.7	1.4	Versicolor

3.	7.3	2.9	6.3	1.8	Virginica
----	-----	-----	-----	-----	-----------

A dataset need not be always numbers. It can be images or video frames. Deep neural networks can handle images with labels. In the following Figure 1.6, the deep neural network takes images of dogs and cats with labels for classification. In unlabelled data, there are no labels in the dataset.

Input	Label
	dog
	Cat

(a)



(b)

Figure 1.6: (a) Labelled Dataset (b) Unlabelled Dataset

1.4.1 Supervised Learning

Supervised algorithms use labelled dataset. As the name suggests, there is a supervisor or teacher component in supervised learning. A supervisor provides labelled data so that the model is constructed and generates test data.

In supervised learning algorithms, learning takes place in two stages. In layman terms, during the first stage, the teacher communicates the information to the student that the student is supposed to master. The student receives the information and understands it. During this stage, the teacher has no knowledge of whether the information is grasped by the student.

This leads to the second stage of learning. The teacher then asks the student a set of questions to find out how much information has been grasped by the student. Based on these questions, the student is tested, and the teacher informs the student about his assessment. This kind of learning is typically called supervised learning.

Supervised learning has two methods:

1. Classification
2. Regression

Classification

Classification is a type of supervised learning.

- **Supervised Learning:** The algorithm learns from labeled data, where we know the correct answers.
- **Independent Variables:** These are the input features, also called attributes.
- **Dependent Variable (Label):** This is the target we want to predict, and it's in the form of discrete categories or labels (e.g., dog or cat).

How Classification Works:

1. Training Stage:

- The algorithm is given a dataset that includes both the features (input) and their correct labels (output).
- The algorithm learns from this data and creates a model.

2. Testing Stage:

- The model is tested on new, unseen data (input), and it predicts the label (output).
- For example, if you input an image of a dog or cat that the model hasn't seen before, the model will assign the correct label based on what it has learned.

Example:

In the Iris dataset, if you input data like (6.3, 2.9, 5.6, 1.8, ?), the model will predict the missing label. This process of assigning a label to new data is called **classification**.

Applications of Classification:

Image Recognition: Classifying images of animals, plants, or even medical conditions like cancer.

Types of Classification Models:

Classification models can be grouped into two categories:

1. **Generative Models:** Focus on how the data is generated and its distribution (e.g., Naïve Bayes).
2. **Discriminative Models:** Focus only on distinguishing between different classes (e.g., Support Vector Machines).

Key Classification Algorithms:

- **Decision Tree**
 - **Random Forest**
 - **Support Vector Machines (SVM)**
 - **Naïve Bayes**
-

- **Artificial Neural Networks (ANN)** and **Deep Learning** (e.g., Convolutional Neural Networks - CNN)

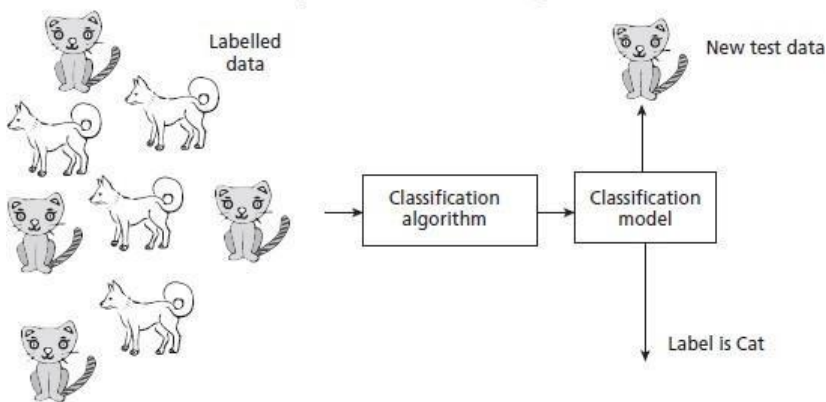


Figure 1.7: An Example Classification System

Regression Models

Regression is another type of **supervised learning**, similar to classification, but instead of predicting categories (labels), it predicts **continuous values**, like numbers.

Key Difference:

- **Regression:** Predicts continuous values (e.g., product sales, house prices).
- **Classification:** Predicts labels or categories (e.g., dog or cat).

How Regression Works:

In a regression model, we are trying to find a relationship between the **independent variable(s) (x)** and the **dependent variable (y)**.

For example, in Figure 1.8, the **independent variable (x)** is the number of weeks, and the **dependent variable (y)** is product sales. The regression model fits a line to the data, which can be used to predict future sales. This line is written as: shown in fig 1.8

$$\text{Sales (y)} = 0.66 \times \text{Week (x)} + 0.54$$

- Here, 0.66 and 0.54 are **regression coefficients** that the model learns from the data.
- If you want to predict the sales for the 8th week, you can substitute $x=8$ into the formula and calculate the predicted sales (y).

Example:

For the 8th week: $\text{Sales} = 0.66 \times 8 + 0.54$

$\text{Sales} = 0.66 \times 8 + 0.54$ This gives a predicted value of sales for week 8.

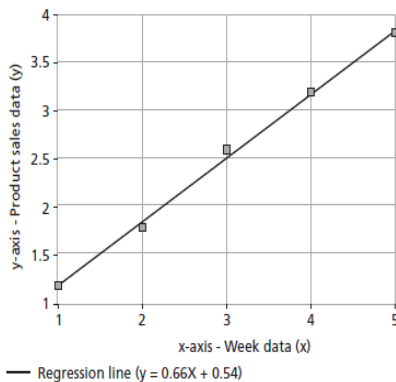


Figure 1.8: A Regression Model of the Form $y = ax + b$

Similarities Between Regression and Classification:

- Both are **supervised learning** methods, meaning they require a labeled training dataset.
- Both involve a **training stage** (where the model learns from data) and a **testing stage** (where the model is used to make predictions on new data).

Main Difference:

- **Regression:** Predicts **numbers** (continuous values).
- **Classification:** Predicts **categories** (discrete values, like class labels).

One of the most common regression algorithms is **linear regression**, which fits a straight line to the data.

Unsupervised learning

is a type of learning where there is **no supervisor or teacher** guiding the process. Instead, the algorithm **learns by itself** using trial and error.

How Unsupervised Learning Works:

- In this method, the algorithm is given data without any labels.
- The algorithm looks at the data and tries to find **patterns or groupings** on its own.
- The goal is to **group similar objects** together based on their characteristics.

Example of Unsupervised Learning:

Clustering

- **Clustering** is a common unsupervised learning technique.

- It groups objects into different **clusters**, where each cluster contains objects that are similar to each other.
- The objects in one cluster are different from those in other clusters.

For example, if you have a set of images of dogs and cats, a clustering algorithm will automatically group them into two clusters: one for dogs and one for cats, without needing any labels to tell it which is which.

Applications of Clustering:

- **Image Segmentation:** Grouping parts of an image, like separating a region of interest (e.g., identifying a tumor in a medical image).
- **Gene Analysis:** Finding groups of similar genes in a database.

In summary, unsupervised learning helps the algorithm **discover patterns** in data without any explicit instructions. **Cluster analysis** and **dimensional reduction** are key types of unsupervised learning.

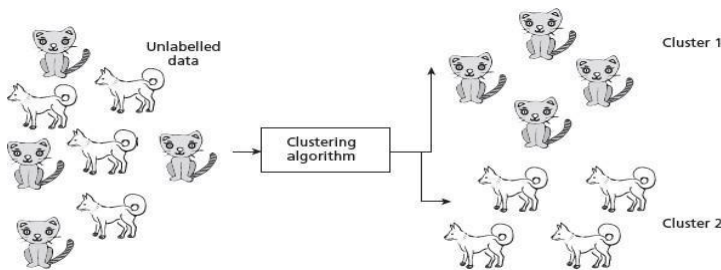


Figure 1.9: An Example Clustering Scheme

Some of the key clustering algorithms are:

- k-means algorithm
- Hierarchical algorithms

Dimensionality Reduction

Dimensionality reduction algorithms are examples of unsupervised algorithms. It takes a higher dimension data as input and outputs the data in lower dimension by taking advantage of the variance of the data. It is a task of reducing the dataset with few features without losing the generality. The differences between supervised and unsupervised learning are listed in the following Table 1.2.

Table 1.2: Differences between Supervised and Unsupervised Learning

S.No.	Supervised Learning	Unsupervised Learning
1.	There is a supervisor component	No supervisor component
2.	Uses Labelled data	Uses Unlabelled data
3.	Assigns categories or labels	Performs grouping process such that similar objects will be in one cluster

1.4.2 Semi-supervised Learning

There are circumstances where the dataset has a huge collection of unlabelled data and some labelled data. Labelling is a costly process and difficult to perform by the humans. Semi-supervised algorithms use unlabelled data by assigning a pseudo-label. Then, the labelled and pseudo-labelled dataset can be combined.

1.4.3 Reinforcement Learning

Reinforcement learning is a type of machine learning where an **agent** learns by interacting with its **environment**. The agent performs actions and receives feedback in the form of **rewards** or **penalties**, and its goal is to maximize the total reward over time.

Key Concepts:

- **Agent:** The learner or decision-maker.
- **Environment:** The world the agent interacts with.
- **Action:** What the agent can do.
- **Reward:** Feedback given to the agent based on its actions (positive for good actions, negative for bad actions).

The agent learns through **trial and error**, improving its strategy (called a **policy**) over time to make better decisions.

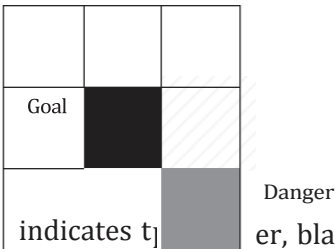
Example of Reinforcement Learning:

Consider a robot learning to walk:

- The robot (agent) takes steps (actions) in its environment.
- If the robot falls, it receives a **penalty** (negative reward). If it moves forward without falling, it receives a **reward** (positive reward).
- Over time, the robot adjusts its movements to maximize its forward motion and minimize falling, effectively learning how to walk.

In summary, reinforcement learning is about learning from **experience** to make better decisions in the future by maximizing rewards.

Consider the following example of a Grid game as shown in Figure 1.10.



In this grid game, the gray tile indicates a danger zone, black is a block, and the tile with diagonal lines is the goal. The agent starts at the bottom-left grid, using the actions left, right, top and bottom to reach the goal state.

To solve this sort of problem, there is no data. The agent interacts with the environment to get experience. In the above case, the agent tries to create a model by simulating many paths and finding rewarding paths. This experience helps in constructing a model.

1.5 CHALLENGES OF MACHINE LEARNING

Machine learning allows computers to solve certain types of problems much better than humans, especially tasks involving **computation**. For instance, computers can quickly calculate the square root of large numbers or win games like chess and Go against professional players.

However, humans are still better than machines at tasks like **recognition**, though modern machine learning systems, especially **deep learning**, are improving rapidly. For example, machines can recognize human faces instantly. But there are still challenges in machine learning, mainly due to the need for **high-quality data**.

Key Challenges in Machine Learning:

1. Well-Posed Vs Ill-Posed Problems:

- Machine learning works well with **well-posed problems**, where the problem is clearly defined and has enough information to find a solution.
- In **ill-posed problems**, there may be multiple possible answers, making it hard to find the correct one. For example, in a simple dataset (as shown in Table 1.3), several models could fit the data (e.g., multiplication or division). To solve such problems, more data is needed to narrow down the correct model.

Table 1.3: An Example

Input (x_1, x_2)	Output (y)
----------------------	----------------

1, 1	1
2, 1	2
3, 1	3
4, 1	4
5, 1	5

Can a model for this test data be multiplication? That is, $y = x1 * x2$. Well! It is true! But, this is equally true that y may be $y = x1 / x2$ or $y = x1 ^ x2$. So, there are three functions that fit the data.

This means that the problem is ill-posed. To solve this problem, one needs more example to check the model. Puzzles and games that do not have sufficient specification may become an ill-posed problem and scientific computation has many ill-posed problems.

2. Need for Huge, Quality Data:

- Machine learning requires **large amounts of high-quality data**. The data must be complete, without missing or incorrect values. Poor-quality data can lead to inaccurate models.

3. High Computational Power:

- With the growth of **Big Data**, machine learning tasks require powerful computers with specialized hardware like **GPUs** or **TPUs** to handle the high computational load. The increasing complexity of tasks has made high-performance computing essential.

4. Complexity of Algorithms:

- Choosing the right machine learning algorithm, explaining how it works, applying it correctly, and comparing different algorithms are now critical skills for data scientists. This makes the selection and evaluation of algorithms a significant challenge.

5. Bias-Variance Trade-off:

- Overfitting:** When a model performs well on training data but fails on test data, it's called overfitting. This means the model has learned the training data too well but lacks generalization to new data.
- Underfitting:** When a model fails to perform well on both training and test data, it's called underfitting. The model is too simple to capture the patterns in the data.
- Balancing between overfitting and underfitting is a major challenge for machine learning algorithms.

1.6 MACHINE LEARNING PROCESS

The emerging process model for the data mining solutions for business organizations is CRISP-DM. Since machine learning is like data mining, except for the aim, this process can

be used for machine learning. CRISP-DM stands for Cross Industry Standard Process – Data Mining. This process involves six steps. The steps are listed below in Figure 1.11.

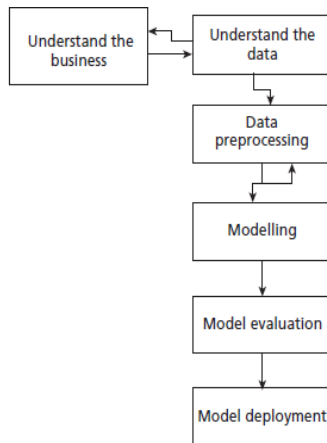


Figure 1.11: A Machine Learning/Data Mining Process

1. Understanding the business – This step involves understanding the objectives and requirements of the business organization. Generally, a single data mining algorithm is enough for giving the solution. This step also involves the formulation of the problem statement for the data mining process.
2. Understanding the data – It involves the steps like data collection, study of the characteristics of the data, formulation of hypothesis, and matching of patterns to the selected hypothesis.
3. Preparation of data – This step involves producing the final dataset by cleaning the raw data and preparation of data for the data mining process. The missing values may cause problems during both training and testing phases. Missing data forces classifiers to produce inaccurate results. This is a perennial problem for the classification models. Hence, suitable strategies should be adopted to handle the missing data.
4. Modelling – This step plays a role in the application of data mining algorithm for the data to obtain a model or pattern.
5. Evaluate – This step involves the evaluation of the data mining results using statistical analysis and visualization methods. The performance of the classifier is determined by evaluating the accuracy of the classifier. The process of classification is a fuzzy issue. For example, classification of emails requires extensive domain knowledge and requires domain experts. Hence, performance of the classifier is very crucial.
6. Deployment – This step involves the deployment of results of the data mining algorithm to improve the existing process or for a new situation.

1.7 MACHINE LEARNING APPLICATIONS

Machine Learning technologies are used widely now in different domains. Machine learning applications are everywhere! One encounters many machine learning applications in the day-to-day life. Some applications are listed below:

1. Sentiment analysis – This is an application of natural language processing (NLP) where the words of documents are converted to sentiments like happy, sad, and angry which are captured by emoticons effectively. For movie reviews or product reviews, five stars or onestars are automatically attached using sentiment analysis programs.
2. Recommendation systems – These are systems that make personalized purchases possible. For example, Amazon recommends users to find related books or books bought by people who have the same taste like you, and Netflix suggests shows or related movies of your taste. The recommendation systems are based on machine learning.
3. Voice assistants – Products like Amazon Alexa, Microsoft Cortana, Apple Siri, and Google Assistant are all examples of voice assistants. They take speech commands and perform tasks. These chatbots are the result of machine learning technologies.
4. Technologies like Google Maps and those used by Uber are all examples of machine learning which offer to locate and navigate shortest paths to reduce time.

The machine learning applications are enormous. The following Table 1.4 summarizes some of the machine learning applications.

Table 1.4: Applications' Survey Table

S.No.	Problem Domain	Applications
1.	Business	Predicting the bankruptcy of a business firm
2.	Banking	Prediction of bank loan defaulters and detecting credit card frauds
3.	Image Processing	Image search engines, object identification, image classification, and generating synthetic images
4.	Audio/Voice	Chatbots like Alexa, Microsoft Cortana. Developing chatbots for customer support, speech to text, and text to voice
5.	Telecommunication	Trend analysis and identification of bogus calls, fraudulent calls and its callers, churn analysis
6.	Marketing	Retail sales analysis, market basket analysis, product performance analysis, market segmentation analysis, and study of travel patterns of customers for marketing tours
7.	Games	Game programs for Chess, GO, and Atari video games
8.	Natural Language Translation	Google Translate, Text summarization, and sentiment analysis

9.	Web Analysis and Services	Identification of access patterns, detection of e-mail spams, viruses, personalized web services, search engines like Google, detection of promotion of user websites, and finding loyalty of users after web page layout modification
10.	Medicine	Prediction of diseases, given disease symptoms as cancer or diabetes. Prediction of effectiveness of the treatment using patient history and Chatbots to interact with patients like IBM Watson uses machine learning technologies.
11.	Multimedia and Security	Face recognition/identification, biometric projects like identification of a person from a large image or video database, and applications involving multimedia retrieval
12.	Scientific Domain	Discovery of new galaxies, identification of groups of houses based on house type/geographical location, identification of earthquake epicenters, and identification of similar land use

Key Terms:

- **Machine Learning** – A branch of AI that concerns about machines to learn automatically without being explicitly programmed.
- **Data** – A raw fact.
- **Model** – An explicit description of patterns in a data.
- **Experience** – A collection of knowledge and heuristics in humans and historical training data in case of machines.
- **Predictive Modelling** – A technique of developing models and making a prediction of unseen data.
- **Deep Learning** – A branch of machine learning that deals with constructing models using neural networks.
- **Data Science** – A field of study that encompasses capturing of data to its analysis covering all stages of data management.
- **Data Analytics** – A field of study that deals with analysis of data.
- **Big Data** – A study of data that has characteristics of volume, variety, and velocity.
- **Statistics** – A branch of mathematics that deals with learning from data using statistical methods.
- **Hypothesis** – An initial assumption of an experiment.
- **Learning** – Adapting to the environment that happens because of interaction of an agent with the environment.
- **Label** – A target attribute.
- **Labelled Data** – A data that is associated with a label.

- **Unlabelled Data** – A data without labels.
- **Supervised Learning** – A type of machine learning that uses labelled data and learns with the help of a supervisor or teacher component.
- **Classification Program** – A supervisory learning method that takes an unknown input and assigns a label for it. In simple words, finds the category of class of the input attributes.
- **Regression Analysis** – A supervisory method that predicts the continuous variables based on the input variables.
- **Unsupervised Learning** – A type of machine learning that uses unlabelled data and groups the attributes to clusters using a trial and error approach.
- **Cluster Analysis** – A type of unsupervised approach that groups the objects based on attributes so that similar objects or data points form a cluster.
- **Semi-supervised Learning** – A type of machine learning that uses limited labelled and large unlabelled data. It first labels unlabelled data using labelled data and combines it for learning purposes.
- **Reinforcement Learning** – A type of machine learning that uses agents and environment interaction for creating labelled data for learning.
- **Well-posed Problem** – A problem that has well-defined specifications. Otherwise, the problem is called ill-posed.
- **Bias/Variance** – The inability of the machine learning algorithm to predict correctly due to lack of generalization is called bias. Variance is the error of the model for training data. This leads to problems called overfitting and underfitting.
- **Model Deployment** – A method of deploying machine learning algorithms to improve the existing business processes for a new situation.

2.1 WHAT IS DATA?

- **Data** refers to raw facts that can be numbers, text, images, audio, or video.
- In computer systems, these facts are encoded in **bits**, allowing machines to process and store them.
- **Directly interpretable data**: Numbers or text, like "John is 25 years old."
- **Diffused data**: Data like images or videos that require computers to interpret, like identifying objects in a photo.

Types of Data Sources

1. **Flat files**: Simple files like CSV or text files.
2. **Databases**: Systems that store structured data.
3. **Data warehouses**: Centralized repositories for large volumes of data.

Operational vs. Non-operational Data

- **Operational data**: Data generated during regular business processes. Example: daily sales figures.
- **Non-operational data**: Data used for strategic decision-making, such as past sales data to predict future trends.

Data vs. Information

- **Data** alone is meaningless until it is processed to create **information**.
 - Example: A list of numbers is just data, but labeling it as "heights of students" gives it context, turning it into information.
- **Information** shows patterns, relationships, and associations.
 - Example: Analyzing sales data can reveal which products sell the most.

Big Data: Elements and Characteristics

What is Big Data?

- **Small data** can be processed using regular computers.
- **Big Data** is data that exceeds the capacity of standard computers and requires specialized tools.

The 6 Vs of Big Data

1. Volume:

- Refers to the **size** of data.
- Big Data is often measured in **petabytes (PB)** or **exabytes (EB)**, much larger than the gigabytes or terabytes of traditional data.

2. Velocity:

- The **speed** at which data is generated and processed.
- Thanks to IoT devices and the Internet, data arrives rapidly, often in real-time.

3. Variety:

- The **diversity** of data formats:
 - **Form:** Data comes as text, audio, video, graphs, etc.
 - **Function:** Data from sources like conversations, transactions, or archives.
 - **Source:** Data can come from public sources, social media, or multimodal sources (combining different types).

4. Veracity:

- Refers to the **accuracy** and **trustworthiness** of data.
- Errors like technical glitches or human mistakes can affect the reliability of data, making veracity crucial.

5. Validity:

- The **relevance** of data for a particular purpose, ensuring it is accurate and fit for decision-making.

6. Value:

- The **usefulness** of data based on the insights and information it provides, helping organizations make better decisions.

The data quality of the numeric attributes is determined by factors like precision, bias, and accuracy.

- Precision is defined as the closeness of repeated measurements. Often, standard deviation is used to measure the precision.
- Bias is a systematic result due to erroneous assumptions of the algorithms or procedures.
- Accuracy is the degree of measurement of errors that refers to the closeness of measurements to the true value of the quantity. Normally, the significant digits used to store and manipulate indicate the accuracy of the measurement.

2.1.1 Categories of Big Data

Big Data, data can be categorized into three types: **structured data**, **unstructured data**, and **semi-structured data**. Each type has its own characteristics, formats, and storage methods.

1. Structured Data

Definition: Structured data is organized and stored in a predefined format, such as a table in a database. This data is easy to search, retrieve, and analyze using tools like SQL.

Types of Structured Data:

- **Record Data:**
 - A dataset consists of a collection of measurements.
 - **Rows** (entities, cases, or records) represent objects.
 - **Columns** (attributes, features, or fields) represent measurements for each object.
 - A **label** refers to individual observations in the dataset.
- **Data Matrix:**

- A type of record data where all attributes are numeric.
- Data is represented as points in a multidimensional space where each attribute represents a dimension.
- Matrix operations can be applied to analyze this data.

- **Graph Data:**

- Represents relationships between objects.
- Example: In a web graph, **nodes** are web pages, and **edges** (hyperlinks) connect them.

- **Ordered Data:**

- Objects have attributes with an implicit order.
- Examples of ordered data:
 - **Temporal data:** Attributes associated with time, e.g., customer purchase patterns during festivals.
 - **Sequence data:** Sequence of elements without timestamps, e.g., DNA sequences (A, T, G, C).
 - **Spatial data:** Related to locations or positions, e.g., maps where points relate to geographical locations.

2. Unstructured Data

Unstructured data does not have a predefined organizational format. This type of data includes multimedia (video, image, audio) as well as text documents, blogs, and social media data.

- Examples of unstructured data include:
 - Videos on platforms like YouTube.
 - Images and photos.
 - Audio recordings, such as podcasts or songs.
 - Text documents, blogs, and posts from social media.

Key Point: It is estimated that around **80% of all data** is unstructured, making it a large and significant portion of Big Data.

3. Semi-Structured Data

Semi-structured data falls between structured and unstructured data. While it does not conform to a strict structure, it contains tags or markers that make it easier to organize.

- Examples of semi-structured data include:
 - **XML/JSON files:** Contain data with embedded tags or fields.
 - **RSS feeds:** Often follow a hierarchical structure, but not as rigid as a database.
 - **Hierarchical data:** Data that follows a parent-child relationship, like in a directory tree.

2.1.2 Data Storage and Representation

Once the dataset is assembled, it must be stored in a structure that is suitable for data analysis. The goal of data storage management is to make data available for analysis. There are different approaches to organize and manage data in storage files and systems from flat file to data warehouses. Some of them are listed below:

Flat Files These are the simplest and most commonly available data source. It is also the cheapest way of organizing the data. These flat files are the files where data is stored in plain ASCII or EBCDIC format. Minor changes of data in flat files affect the results of the data mining algorithms.

Hence, flat file is suitable only for storing small dataset and not desirable if the dataset becomes larger.

Some of the popular spreadsheet formats are listed below:

- **CSV files** – CSV stands for comma-separated value files where the values are separated by commas. These are used by spreadsheet and

database applications. The first row may have attributes and the rest of the rows represent the data.

- TSV files – TSV stands for Tab separated values files where values are separated by Tab. Both CSV and TSV files are generic in nature and can be shared. There are many tools like Google Sheets and Microsoft Excel to process these files.

Database System It normally consists of database files and a database management system (DBMS). Database files contain original data and metadata. DBMS aims to manage data and improve operator performance by including various tools like database administrator, query processing, and transaction manager. A relational database consists of sets of tables. The tables have rows and columns. The columns represent the attributes and rows represent tuples. A tuple corresponds to either an object or a relationship between objects. A user can access and manipulate the data in the database using SQL.

Different types of databases are listed below:

- 1 A transactional database is a collection of transactional records. Each record is a transaction. A transaction may have a time stamp, identifier and a set of items, which may have links to other tables. Normally, transaction databases are created for performing associational analysis that indicates the correlation among the items.
2. Time-series database stores time related information like log files where data is associated with a time stamp. This data represents the sequences of data, which represent values or events obtained over a period (for example, hourly, weekly or yearly) or repeated time span. Observing sales of product continuously may yield a time-series data.
3. Spatial databases contain spatial information in a raster or vector format. Raster formats are either bitmaps or pixel maps. For example, images can be stored as a raster data. On the other hand, the vector format can be used to store maps as maps use basic geometric primitives like points, lines, polygons and so forth.

World Wide Web (WWW) It provides a diverse, worldwide online information source.

The objective of data mining algorithms is to mine interesting patterns of information present in WWW.

XML (eXtensible Markup Language) It is both human and machine interpretable data format that can be used to represent data that needs to be shared across the platforms.

Data Stream It is dynamic data, which flows in and out of the observing environment. Typical characteristics of data stream are huge volume of data, dynamic, fixed order movement, and real-time constraints.

RSS (Really Simple Syndication) It is a format for sharing instant feeds across services.

JSON (JavaScript Object Notation) It is another useful data interchange format that is often used for many machine learning algorithms.

2.2 BIG DATA ANALYTICS AND TYPES OF ANALYTICS

The primary aim of data analysis is to assist business organizations to take decisions. For example, a business organization may want to know which is the fastest selling product, in order for them to market activities. Data analysis is an activity that takes the data and generates useful information and insights for assisting the organizations.

Data analysis and data analytics are terms that are used interchangeably to refer to the same concept. However, there is a subtle difference. Data analytics is a general term and data analysis is a part of it. Data analytics refers to the process of data collection, preprocessing and analysis. It deals with the complete cycle of data management. Data analysis is just analysis and is a part of data analytics. It takes historical data and does the analysis.

Data analytics, instead, concentrates more on future and helps in prediction.

There are four types of data analytics:

1. Descriptive analytics

2. Diagnostic analytics
3. Predictive analytics
4. Prescriptive analytics

Descriptive Analytics

Descriptive analytics is about summarizing the main features of the data you've collected. It tells you **what has happened** by using historical data and statistical techniques. The goal is to organize, describe, and present this data in an understandable way. Think of it as a report that explains "what is" without drawing any conclusions about why it happened.

Example: Imagine a store that collects data on monthly sales. Descriptive analytics would summarize this data by calculating the average sales, total revenue, or the most popular product in a given month.

Key Point: It doesn't explain why the sales were high or low—just tells you what the data shows.

Diagnostic Analytics

Diagnostic analytics answers the question **"Why did this happen?"** It's about understanding the root cause of an event. By examining the data closely, we look for patterns, trends, and relationships that explain the cause of an outcome.

Example: If the store's sales drop one month, diagnostic analytics would investigate why the drop happened. Maybe it's due to bad weather, a competitor's sale, or a new product that didn't perform well. The analysis focuses on finding and explaining the reasons behind the drop.

Key Point: It's all about cause and effect—identifying the reasons behind the data patterns.

Predictive Analytics

Predictive analytics looks into the future and answers the question "**What will happen?**" Using historical data and advanced algorithms (like machine learning), it predicts future trends and outcomes.

Example: The store uses data from previous years to predict what the sales will be in the upcoming holiday season. Algorithms analyze patterns like past holiday sales, customer behavior, and current market trends to make predictions.

Key Point: It focuses on forecasting future events based on current and past data.

Prescriptive Analytics

Prescriptive analytics goes a step further and asks "**What should we do?**" It not only predicts the future but also recommends actions to take. This type of analytics provides decision-making support by suggesting the best course of action to achieve desired outcomes.

Example: After predicting that sales will be low in the next quarter, prescriptive analytics suggests specific actions the store can take, such as launching a promotion, adjusting prices, or stocking more popular products. This helps businesses make better decisions and minimize risks.

Key Point: It's all about decision-making—helping businesses choose the best possible actions based on data.

2.3.1 Data Collection

The first task of gathering datasets are the collection of data. It is often estimated that most of the time is spent for collection of good quality data. A good quality data yields a better result. It is often difficult to characterize a 'Good data'. 'Good data' is one that has the following properties:

1. **Timeliness** – The data should be relevant and not stale or obsolete data.
2. **Relevancy** – The data should be relevant and ready for the machine learning or data mining algorithms. All the necessary

information should be available and there should be no bias in the data.

3. Knowledge about the data – The data should be understandable and interpretable, and should be self-sufficient for the required application as desired by the domain knowledge engineer.

Broadly, the data source can be classified as open/public data, social media data and multimodal data.

1. **Open or public data source** – It is a data source that does not have any stringent copyright

rules or restrictions. Its data can be primarily used for many purposes. Government census data are good examples of open data:

- Digital libraries that have huge amount of text data as well as document images
- Scientific domains with a huge collection of experimental data like genomic data and biological data
- Healthcare systems that use extensive databases like patient databases, health insurance data, doctors' information, and bioinformatics information

2. **Social media** – It is the data that is generated by various social media platforms like Twitter,

Facebook, YouTube, and Instagram. An enormous amount of data is generated by these platforms.

3. **Multimodal data** – It includes data that involves many modes such as text, video, audio and mixed types. Some of them are listed below:

- Image archives contain larger image databases along with numeric and text data
- The World Wide Web (WWW) has huge amount of data that is distributed on the Internet.

2.3.2 Data Pre-processing

Data Cleaning is the process of detecting and correcting (or removing) errors and inconsistencies in data to improve its quality before applying machine learning or data mining techniques. In the real world, raw data is often **'dirty'**, meaning it contains errors, missing information, or inconsistencies that can affect the results of the analysis.

Common Problems with Dirty Data:

1. **Incomplete Data:** When certain values are missing from the dataset.
2. **Inaccurate Data:** Data that has incorrect values or errors.
3. **Outliers:** Data points that are significantly different from the rest of the data, often due to errors or unusual circumstances.
4. **Missing Values:** Data entries where certain attributes are not provided.
5. **Inconsistent Values:** Mismatched or incorrectly formatted data values.
6. **Duplicate Data:** When the same data appears multiple times, unnecessarily.

Example of Dirty Data:

Let's refer to the table of patient data (from the image) to explain common data issues:

Table 2.1: Illustration of 'Bad' Data

Patient ID	Name	Age	Date of Birth (DoB)	Fever	Salary
1.	John	21		Low	-1500
2.	Andre	36		High	Yes
3.	David	5	10/10/1980	Low	" "
4.	Raju	136		High	Yes

Identifying the Problems:

1. **Incomplete Data:**

- For patients John, Andre, and Raju, the **Date of Birth (DoB)** is missing. This is an example of **missing values**.

2. Inaccurate Data:

- David's **age is recorded as 5**, but his **DoB is 10/10/1980**, which makes his real age much older than 5. This is **inconsistent data**.
- Raju's **age is recorded as 136**, which is not realistic. This might be a **typographical error** or an **outlier**.

3. Outliers:

- Raju's age of 136 is an **outlier**, as it is an unrealistic value when compared to normal human lifespans. Outliers are often caused by data entry errors.

4. Noisy Data:

- John's **salary** is recorded as **-1500**, which is not possible. Salary cannot be negative, making this an example of **noisy data**.
- The entry for David's **salary** is simply blank (" "), which is another instance of missing data.

5. Inconsistent Values:

- In the **salary** column, Andre and Raju both have **'Yes'** recorded, which doesn't make sense in the context of salary data. A salary should be a numeric value, not a text response.

How to Address These Issues?

1. Missing Data:

- **Ignore the Tuple:** If a lot of values are missing in a row, you may choose to ignore or remove that row from the dataset.
- **Fill Values Manually:** Domain experts can manually fill missing values, but this is time-consuming.
- **Use Global Constants:** Fill missing values with a placeholder like 'Unknown' or '0'.
- **Use Average/Mean Values:** Replace missing numeric values (like salary) with the average value of that column.

- **Prediction Techniques:** Machine learning algorithms can predict missing values based on patterns in other data (e.g., using decision trees).

2. Inaccurate Data:

- Correct entries by referring to other reliable data sources or consult domain experts. For example, David's age should be corrected based on his actual DoB.

3. Handling Outliers:

- Investigate outliers to determine if they are errors or legitimate data. Raju's age of 136 may be a typo and can be corrected if the correct age is known.

4. Noisy Data:

- Noisy data, like John's negative salary (-1500), can be corrected by setting a minimum limit (e.g., salary cannot be below 0). In this case, either correct or remove the invalid entry.

5. Inconsistent Values:

- Standardize the format for fields like salary. For Andre and Raju, change the text entries ('Yes') to numeric values or fill in missing data using estimation techniques.

Methods for Handling Missing Data:

1. **Ignoring the Tuple:** Discard rows with missing data (not ideal when a lot of data is missing).

2. **Filling Manually:** Domain experts analyze and fill the missing values.

3. **Global Constant:** Fill missing values with 'Unknown' or 'None'.

4. **Attribute Mean:** Replace missing numerical values with the average for that attribute.

5. **Class-based Mean:** Use the mean value of the same class or group to fill missing data.

6. **Most Probable Value:** Predict missing values using machine learning algorithms like decision trees.

Removal of Noisy or Outlier Data

In data analysis, **noise** refers to random errors or variations in the data that can distort the results of analysis. Noise can affect data accuracy and, if not removed, can lead to misleading conclusions. Therefore, it's important to clean noisy data before applying any analysis or machine learning algorithms.

What is Noise?

- Noise is random error or variance in measured values.
- It can appear as outliers, missing values, or inconsistent data.
- Noise reduction is an essential step in **data cleaning** to improve the quality of analysis.

Techniques for Removing Noise:

One common method to remove noisy data is **binning**, which organizes data into groups (bins) and then applies smoothing techniques to remove noise. Binning methods can also be used for **data discretization**, which reduces the number of values for easier analysis.

Binning Method:

- **Step 1:** Sort the data in increasing order.
- **Step 2:** Divide the sorted data into equal-frequency bins (also called **buckets**).
- **Step 3:** Apply smoothing techniques within each bin to reduce noise.

Smoothing Techniques for Binning:

1. Smoothing by Means:

- Replace all values in the bin with the mean (average) of the bin values.

Example:

- Given data: $S = \{12, 14, 19, 22, 24, 26, 28, 31, 34\}$
- First, divide into bins of size 3:
 - **Bin 1:** {12, 14, 19}
 - **Bin 2:** {22, 24, 26}
 - **Bin 3:** {28, 31, 34}
- Now apply smoothing by means (replace all values with the bin's mean):
 - **Bin 1** (mean = 15): {15, 15, 15}
 - **Bin 2** (mean = 24): {24, 24, 24}
 - **Bin 3** (mean $\approx$ 31): {31, 31, 31}
- **Explanation:** Each value in the bin is replaced by the mean of the bin to smooth the data.

2. Smoothing by Medians:

- Replace all values in the bin with the **median** of the bin values (the middle value when the data is sorted).

Example:

- Given the same data and bins:
 - **Bin 1** (median = 14): {14, 14, 14}
 - **Bin 2** (median = 24): {24, 24, 24}
 - **Bin 3** (median = 31): {31, 31, 31}
- **Explanation:** Each value in the bin is replaced by the median, which reduces the effect of outliers or extreme values.

3. Smoothing by Bin Boundaries:

- Replace each value in the bin with the closest **boundary value** (minimum or maximum value in the bin).

Example:

- Given the same data and bins:
 - **Bin 1** (boundary values: 12 and 19): {12, 12, 19}
 - **Bin 2** (boundary values: 22 and 26): {22, 22, 26}
 - **Bin 3** (boundary values: 28 and 34): {28, 34, 34}
- **Explanation:** For each bin, values are replaced by the closest boundary value (either the minimum or maximum of that bin).
- **Example:** In Bin 1, the original data was {12, 14, 19}. The boundaries are 12 and 19, so the value 14 is closer to 12, and it's replaced by 12.

Why Use Binning to Remove Noise?

- **Smoothing by Means:** Reduces random noise by averaging the values within each bin.
- **Smoothing by Medians:** More robust against outliers than using means since medians are less sensitive to extreme values.
- **Smoothing by Bin Boundaries:** Eliminates noise by forcing all values within the bin to adhere to the boundaries, creating a more consistent dataset.

Data Integration and Data Transformations

Data integration involves routines that merge data from multiple sources into a single data source.

So, this may lead to redundant data. The main goal of data integration is to detect and remove redundancies that arise from integration. Data transformation routines perform operations like normalization to improve the performance of the data mining algorithms. It is necessary to transform data so that it can be processed. This can be considered as a preliminary stage of data conditioning. Normalization is one such technique. In normalization, the attribute values are scaled

to fit in a range (say 0-1) to improve the performance of the data mining algorithm. Often, in neural networks, these techniques are used. Some of the normalization procedures used are:

1. Min-Max
2. z-Score

Min-Max Procedure It is a normalization technique where each variable V is normalized by its difference with the minimum value divided by the range to a new range, say 0–1. Often, neural networks require this kind of normalization. The formula to implement this normalization is given as:

$$\text{min-max} = \frac{V - \text{min}}{\text{max} - \text{min}} \times (\text{new max} - \text{new min}) + \text{new min} \quad (2.1)$$

Here max-min is the range. Min and max are the minimum and maximum of the given data, new max and new min are the minimum and maximum of the target range, say 0 and 1.

Example 2.2: Consider the set: $V = \{88, 90, 92, 94\}$. Apply Min-Max procedure and map the marks to a new range 0–1.

Solution: The minimum of the list V is 88 and maximum is 94. The new min and new max are 0 and 1, respectively. The mapping can be done using Eq. (2.1) as:

For marks 88,

$$\text{min-max} = \frac{88 - 88}{94 - 88} \times (1 - 0) + 0 = 0$$

Similarly, other marks can be computed as follows:

For marks 90,

$$\text{min-max} = \frac{90 - 88}{94 - 88} \times (1 - 0) + 0 = 0.33$$

For marks 92,

$$\text{min-max} = \frac{92 - 88}{94 - 88} \times (1 - 0) + 0 = \frac{4}{6} = 0.66$$

For marks 94,

$$\text{min-max} = \frac{94 - 88}{94 - 88} \times (1 - 0) + 0 = \frac{6}{6} = 1$$

So, it can be observed that the marks {88, 90, 92, 94} are mapped to the new range {0, 0.33, 0.66, 1}. Thus, the Min-Max normalization range is between 0 and 1.

z-Score Normalization This procedure works by taking the difference between the field value and mean value, and by scaling this difference by standard deviation of the attribute.

$$V^* = \frac{V - \mu}{\sigma} \quad (2.2)$$

Here, σ is the standard deviation of the list V and μ is the mean of the list V .

Example 2.3: Consider the mark list $V = \{10, 20, 30\}$, convert the marks to z-score.

Solution: The mean and Sample Standard deviation (σ) values of the list V are 20 and 10, respectively. So the z-scores of these marks are

calculated using Eq. (2.2) as:

$$\text{z-score of } 10 = \frac{10 - 20}{10} = -\frac{10}{10} = -1$$

$$\text{z-score of } 20 = \frac{20 - 20}{10} = \frac{0}{10} = 0$$

$$\text{z-score of } 30 = \frac{30 - 20}{10} = \frac{10}{10} = 1$$

Hence, the z-score of the marks 10, 20, 30 are -1, 0 and 1, respectively.

Data Reduction

Data reduction reduces data size but produces the same results. There are different ways in which data reduction can be carried out such as data aggregation, feature selection, and dimensionality reduction.

2.4 DESCRIPTIVE STATISTICS

Descriptive statistics is a branch of statistics that does dataset summarization. It is used to summarize and describe data. Descriptive statistics are just descriptive and do not go beyond that.

In other words, descriptive statistics do not bother too much about machine learning algorithms and its functioning.

Let us discuss descriptive statistics with the fundamental concepts of datatypes.

Dataset and Data Types

A dataset can be assumed to be a collection of data objects. The data objects may be records, points, vectors, patterns, events, cases, samples or observations. These records contain many attributes. An attribute can be defined as the property or characteristics of an object.

For example, consider the following database shown in sample Table 2.2.

Table 2.2: Sample Patient Table

Patient ID	Name	Age	Blood Test	Fever	Disease
1.	John	21	Negative	Low	No
2.	Andre	36	Positive	High	Yes

Every attribute should be associated with a value. This process is called measurement. The type of attribute determines the data types, often referred to as measurement scale types.

The data types are shown in Figure 2.1.

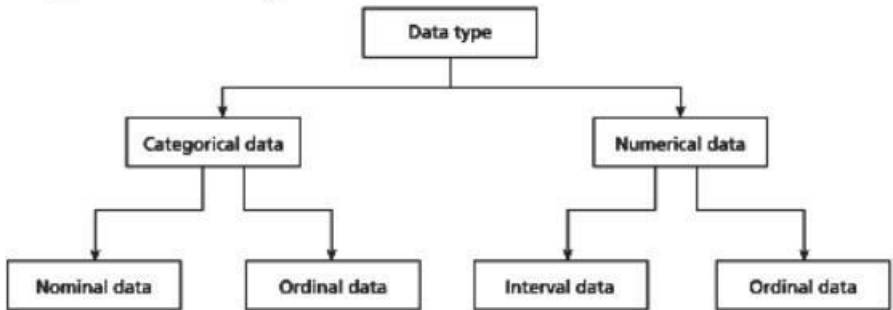


Figure 2.1: Types of Data

Broadly, data can be classified into two types:

1. Categorical or qualitative data
2. Numerical or quantitative data

Categorical or Qualitative Data The categorical data can be divided into two types. They are nominal type and ordinal type.

- **Nominal Data** – In Table 2.2, patient ID is nominal data. Nominal data are symbols and cannot be processed like a number. For example, the average of a patient ID does not make any statistical sense. Nominal data type provides only information but has no ordering among data. Only operations like $(=, \neq)$ are meaningful for these data. For example, the patient ID can be checked for equality and nothing else.

• **Ordinal Data** – It provides enough information and has natural order. For example, Fever = {Low, Medium, High} is an ordinal data. Certainly, low is less than medium and medium is less than high, irrespective of the value. Any transformation can be applied to these data to get a new value.

Numeric or Qualitative Data It can be divided into two categories. They are interval type and ratio type.

• **Interval Data** – Interval data is a numeric data for which the differences between values are meaningful. For example, there is a difference between 30 degrees and 40 degrees. Only the permissible operations are + and -.

• **Ratio Data** – For ratio data, both differences and ratio are meaningful. The difference between the ratio and interval data is the position of zero in the scale. For example, take the Centigrade-Fahrenheit conversion. The zeroes of both scales do not match.

Hence, these are interval data.

Another way of classifying the data is to classify it as:

1. Discrete value data

2. Continuous data

Discrete Data This kind of data is recorded as integers. For example, the responses of the survey can be discrete data. Employee identification number such as 10001 is discrete data.

Continuous Data It can be fitted into a range and includes decimal point. For example, age is a continuous data. Though age appears to be discrete data, one may be 12.5 years old and it makes sense. Patient height and weight are all continuous data.

Third way of classifying the data is based on the number of variables used in the dataset. Based on that, the data can be classified as univariate data, bivariate data, and multivariate data. This is shown in Figure 2.2.

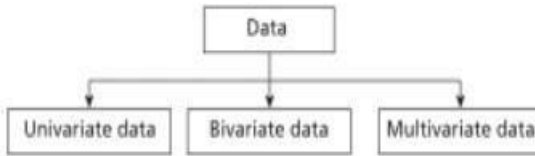


Figure 2.2: Types of Data Based on Variables

2.5 UNIVARIATE DATA ANALYSIS AND VISUALIZATION

Univariate analysis is the simplest form of statistical analysis. As the name indicates, the dataset has only one variable. A variable can be called as a category. Univariate does not deal with cause or relationships. The aim of univariate analysis is to describe data and find patterns. Univariate data description involves finding the frequency distributions, central tendency measures, dispersion or variation, and shape of the data.

2.5.1 Data Visualization

Let us consider some forms of graphs

Bar Chart A Bar chart (or Bar graph) is used to display the frequency distribution for variables.

Bar charts are used to illustrate discrete data. The charts can also help to explain the counts of nominal data. It also helps in comparing the frequency of different groups. The bar chart for students' marks {45, 60, 60, 80, 85} with Student ID = {1, 2, 3, 4, 5} is shown below in Figure 2.3.

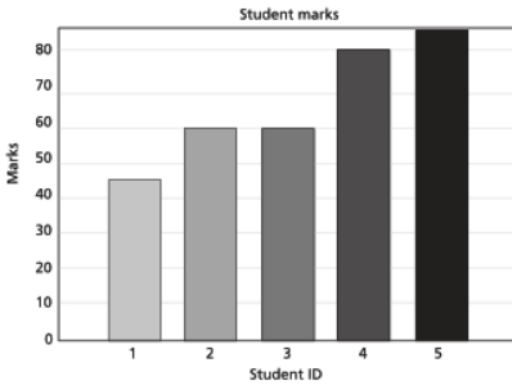


Figure 2.3: Bar Chart

Pie Chart These are equally helpful in illustrating the univariate data. The percentage frequency distribution of students' marks {22, 22, 40, 40, 70, 70, 70, 85, 90, 90} is below in Figure 2.4.

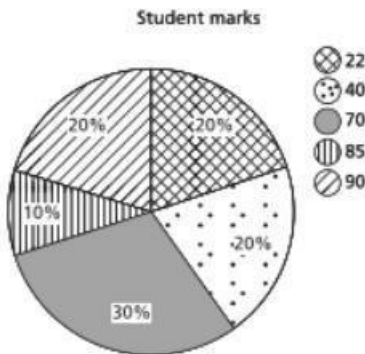


Figure 2.4: Pie Chart

It can be observed that the number of students with 22 marks are 2. The total number of students are 10. So, $2/10 \times 100 = 20\%$ space in a pie of 100% is allotted for marks 22 in Figure 2.4.

Histogram It plays an important role in data mining for showing frequency distributions.

The histogram for students' marks {45, 60, 60, 80, 85} in the group range of 0-25, 26-50, 51-75, 76-100 is given below in Figure 2.5. One can visually inspect from Figure 2.5 that the number of students in the

range 76-100 is 2.

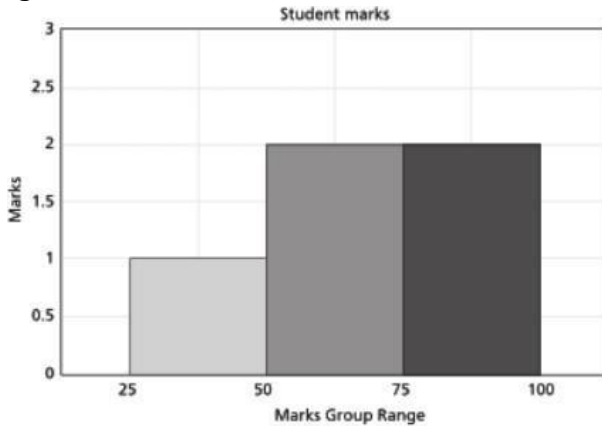


Figure 2.5: Sample Histogram of English Marks

Histogram conveys useful information like nature of data and its mode. Mode indicates the peak of dataset. In other words, histograms can be used as charts to show frequency, skewness present in the data, and shape.

Dot Plots These are similar to bar charts. They are less clustered as compared to bar charts, as they illustrate the bars only with single points. The dot plot of English marks for five students with ID as {1, 2, 3, 4, 5} and marks {45, 60, 60, 80, 85} is given in Figure 2.6. The advantage

is that by visual inspection one can find out who got more marks.

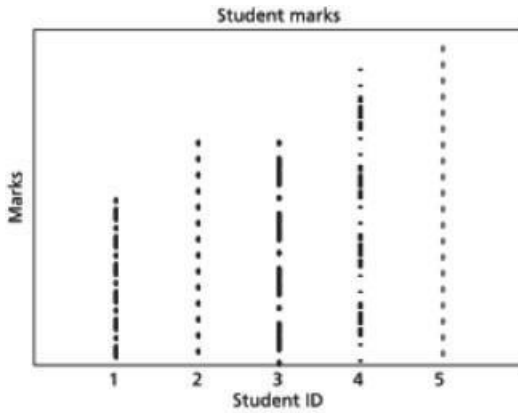


Figure 2.6: Dot Plots

2.5.2 Central Tendency

Therefore, a condensation or summary of the data is necessary. This makes the data analysis easy and simple. One such summary is called central tendency. Thus, central tendency can explain the characteristics of data and that further helps in comparison. Mass data have tendency to concentrate at certain values, normally in the central location. It is called measure of central tendency (or averages). Popular measures are mean, median and mode.

1. Mean – Arithmetic average (or mean) is a measure of central tendency that represents the ‘center’ of the dataset. Mathematically, the average of all the values in the sample (population) is denoted as $\bar{x}$. Let $x_1, x_2, \dots, x_N$ be a set of ‘N’ values or observations, then the arithmetic mean is given as:

$$\bar{x} = \frac{x_1 + x_2 + \dots + x_N}{N} = \frac{1}{N} \sum_{i=1}^N x_i \quad (2.3)$$

For example, the mean of the three numbers 10, 20, and 30 is 20

• **Weighted mean** – Unlike arithmetic mean that gives the weightage of all items equally, weighted mean gives different importance to all items as the item importance varies.

Hence, different weightage can be given to items. In case of frequency distribution, mid values of the range are taken for computation. This is illustrated in the following computation. In weighted mean, the mean is computed by adding the product of proportion and group mean. It is mostly used when the sample sizes are unequal.

• **Geometric mean** – Let $x_1, x_2, \dots, x_N$ be a set of 'N' values or observations. Geometric mean

is the Nth root of the product of N items. The formula for computing geometric mean is given as follows:

$$\text{Geometric mean} = \left(\prod_{i=1}^n x_i \right)^{\frac{1}{N}} = \sqrt[N]{x_1 \times x_2 \times \dots \times x_N} \quad (2.4)$$

Here, n is the number of items and x_i are values. For example, if the values are 6 and 8, the geometric mean is given as In larger cases, computing geometric mean is difficult. Hence, it is usually calculated as:

$$\text{Anti-log of } \frac{\log(x_1) + \log(x_2) + \dots + \log(x_N)}{N} \quad (2.5)$$

$$= \text{anti-log } \frac{\sum_{i=1}^n \log(x_i)}{N} \quad (2.6)$$

The problem of mean is its extreme sensitiveness to noise. Even small changes in the input affect the mean drastically. Hence, often the top 2% is chopped off and then the mean is calculated for a larger dataset.

2. Median – The middle value in the distribution is called median. If the total number of items in the distribution is odd, then the middle value is called median. A median class is that class where $(N/2)$ th item is present.

In the continuous case, the median is given by the formula:

$$\text{Median} = L_1 + \frac{\frac{N}{2} - cf}{f} \times i \quad (2.7)$$

Median class is that class where $N/2$ th item is present. Here, i is the class interval of the median class and L_1 is the lower limit of median class, f is the frequency of the median class, and cf is the cumulative frequency of all classes preceding median.

3. Mode – Mode is the value that occurs more frequently in the dataset. In other words, the value that has the highest frequency is called mode.

2.5.3 Dispersion

The spreadout of a set of data around the central tendency (mean, median or mode) is called dispersion. Dispersion is represented by various ways such as range, variance, standard deviation, and standard error. These are second order measures. The most common measures of the dispersion data are listed below:

Range Range is the difference between the maximum and minimum of values of the given list of data.

Standard Deviation The mean does not convey much more than a middle point. For example, the following datasets {10, 20, 30} and {10, 50, 0} both have a mean of 20. The difference between these two sets is the spread of data. Standard deviation is the average distance from the mean of the dataset to each point.

The formula for sample standard deviation is given by:

$$\sigma = \sqrt{\frac{\sum_{i=1}^N (x_i - \bar{x})^2}{N - 1}} \quad (2.8)$$

Here, N is the size of the population, x_i is observation or value from the population and $\bar{x}$ is the population mean. Often, $N - 1$ is used instead of N in the denominator of Eq. (2.8).

Quartiles and Inter Quartile Range It is sometimes convenient to subdivide the dataset using coordinates. Percentiles are about data that are less than the coordinates by some percentage of the total value. k th percentile is the property that the $k\%$ of the data lies at or below X_i . For example, median is 50th percentile and can be denoted as $Q_{0.50}$. The 25th percentile is called first quartile (Q_1) and the 75th percentile is called third quartile (Q_3). Another measure that is useful to measure dispersion is Inter Quartile Range (IQR). The IQR is the difference between Q_3 and Q_1 .

$$\text{Interquartile percentile} = Q_3 - Q_1 \quad (2.9)$$

Outliers are normally the values falling apart at least by the amount $1.5 \times \text{IQR}$ above the third quartile or below the first quartile.

$$\text{Interquartile is defined by } Q_{0.75} - Q_{0.25}. \quad (2.10)$$

Example 2.4: For patients' age list {12, 14, 19, 22, 24, 26, 28, 31, 34}, find the IQR.

Solution: The median is in the fifth position. In this case, 24 is the median. The first quartile is median of the scores below the mean i.e., {12, 14, 19, 22}. Hence, it's the median of the list below 24. In this case, the median is the average of the second and third values, that is, $Q_{0.25} = 16.5$. Similarly, the third quartile is the median of the values above the median, that is {26, 28, 31, 34}. So, $Q_{0.75}$ is the average of the seventh and eighth score. In this case, it is $28 + 31/2 = 59/2 = 29.5$.

Hence, the IQR using Eq. (2.10) is:

$$\begin{aligned} &= Q_{0.75} - Q_{0.25} \\ &= 29.5 - 16.5 = 13 \end{aligned}$$

Five-point Summary and Box Plots The median, quartiles Q_1 and Q_3 , and minimum and maximum written in the order < Minimum, Q_1 , Median, Q_3 , Maximum > is known as five-point summary. Example 2.5: Find the 5-point summary of the list {13, 11, 2, 3, 4, 8, 9}.

Solution: The minimum is 2 and the maximum is 13. The Q_1 , Q_2 and Q_3 are 3, 8 and 11, respectively. Hence, 5-point summary is {2, 3, 8, 11,

13}, that is, {minimum, Q1, median, Q3, maximum}. Box plots are useful for describing 5-point summary. The Box plot for the set is given in Figure 2.7.

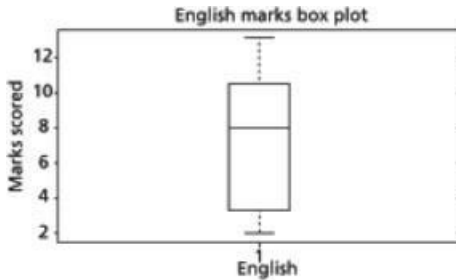


Figure 2.7: Box Plot for English Marks

2.5.4 Shape

Skewness and Kurtosis (called moments) indicate the symmetry/asymmetry and peak location of the dataset.

Skewness

The measures of direction and degree of symmetry are called measures of third order. Ideally, skewness should be zero as in ideal normal distribution. More often, the given dataset may not have perfect symmetry (consider the following Figure 2.8).

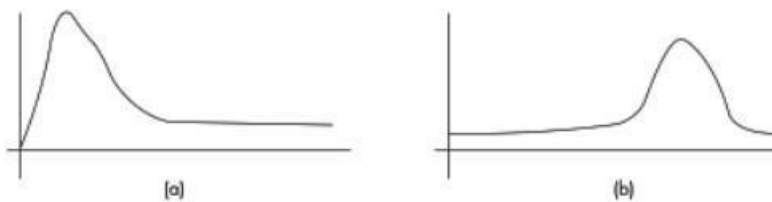


Figure 2.8: (a) Positive Skewed and (b) Negative Skewed Data

Generally, for negatively skewed distribution, the median is more than the mean. The relationship between skew and the relative size of the mean and median can be summarized by a convenient numerical skew

index known as Pearson 2 skewness coefficient.

$$\frac{3 \times (\mu - \text{median})}{\sigma} \quad (2.12)$$

Also, the following measure is more commonly used to measure skewness. Let $X_1, X_2, \dots, X_N$ be a set of 'N' values or observations then the skewness can be given as:

$$\frac{1}{N} \times \sum_{i=1}^N \frac{(x_i - \mu)^3}{\sigma^3} \quad (2.13)$$

Here, μ is the population mean and σ is the population standard deviation of the univariate data. Sometimes, for bias correction instead of N , $N - 1$ is used.

Kurtosis

Kurtosis also indicates the peaks of data. If the data is high peak, then it indicates higher kurtosis and vice versa. Kurtosis is measured using the formula given below:

$$\frac{\sum_{i=1}^N (x_i - \bar{x})^4 / N}{\sigma^4} \quad (2.14)$$

It can be observed that $N - 1$ is used instead of N in the numerator of Eq. (2.14) for bias correction. Here, $\bar{x}$ and σ are the mean and standard deviation of the univariate data, respectively.

Some of the other useful measures for finding the shape of the univariate dataset are mean absolute deviation (MAD) and coefficient of variation (CV).

Mean Absolute Deviation (MAD)

MAD is another dispersion measure and is robust to outliers. Normally, the outlier point is detected by computing the deviation from median and by dividing it by MAD. Here, the absolute deviation between the data and mean is taken. Thus, the absolute deviation is

given as:

$$|x - \mu| \tag{2.15}$$

The sum of the absolute deviations is given as $\sum |x - \mu|$

Therefore, the mean absolute deviation is given as: $\frac{\sum |x - \mu|}{N}$ (2.16)

Coefficient of Variation (CV)

Coefficient of variation is used to compare datasets with different units. CV is the ratio of standard deviation and mean, and %CV is the percentage of coefficient of variations.

2.5.5 Special Univariate Plots

The ideal way to check the shape of the dataset is a stem and leaf plot. A stem and leaf plot are a display that help us to know the shape and distribution of the data. In this method, each value is

split into a 'stem' and a 'leaf'. The last digit is usually the leaf and digits to the left of the leaf mostly form the stem. For example, marks 45 are divided into stem 4 and leaf 5 in Figure 2.9. The stem and leaf plot for the English subject marks, say, {45, 60, 60, 80, 85} is given in Figure 2.9.

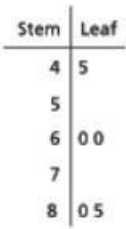


Figure 2.9: Stem and Leaf Plot for English Marks

It can be seen from Figure 2.9 that the first column is stem and the second column is leaf. For the given English marks, two students with 60 marks are shown in stem and leaf plot as stem-6 with 2 leaves with 0. The normal Q-Q plot for marks $x = [13 \ 11 \ 2 \ 3 \ 4 \ 8 \ 9]$ is given below in Figure 2.10.

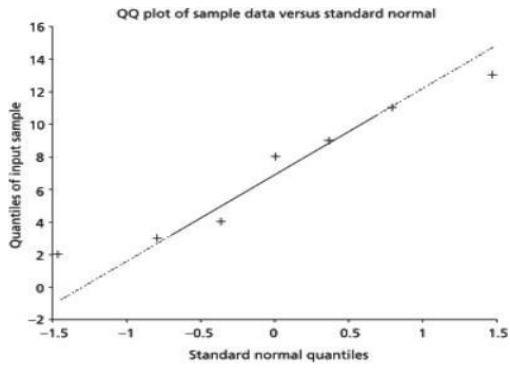


Figure 2.10: Normal Q-Q Plot

Module 2

Understanding Data

Bivariate and Multivariate data, Multivariate statistics, Essential mathematics for Multivariate data, Overview hypothesis, Feature engineering and dimensionality reduction techniques, **Basics of Learning Theory**: Introduction to learning and its types, Introduction computation learning theory, Design of learning system, Introduction concept learning. **Similarity-based learning**: Introduction to Similarity or instance based learning, Nearest-neighbour learning, weighted **k**- Nearest - Neighbour **algorithm**.

CHAPTER -2

2.6 BIVARIATE DATA AND MULTIVARIATE DATA

Bivariate Data involves two variables. Bivariate data deals with causes of relationships. The aim is to find relationships among data. Consider the following Table 2.3, with data of the temperature in a shop and sales of sweaters.

Table 2.3: Temperature in a Shop and Sales Data

Temperature (in centigrade)	Sales of Sweaters (in thousands)
5	200
10	150
15	140
20	75
22	60
23	55
25	20

Here, the aim of bivariate analysis is to find relationships among variables. The relationships can then be used in comparisons, finding causes, and in further explorations. To do that, graphical display of the data is necessary. One such graph method is called scatter plot.

Scatter plot is used to visualize bivariate data. It is useful to plot two variables with or without nominal variables, to illustrate the trends, and also to show differences. It is a plot between explanatory and response variables. It is a 2D graph showing the relationship between two variables. Line graphs are similar to scatter plots. The Line Chart for sales data is shown in Figure 2.12.

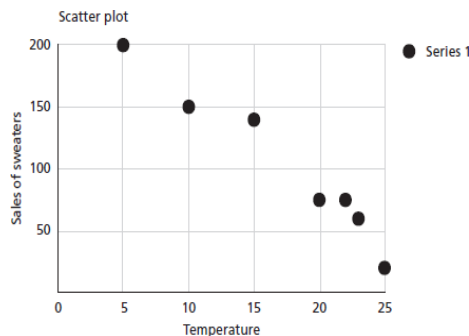


Figure 2.11: Scatter Plot

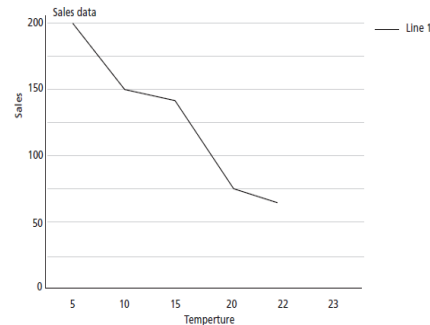


Figure 2.12: Line Chart

2.6.1 Bivariate Statistics

Covariance and Correlation are examples of bivariate statistics. Covariance is a measure of joint probability of random variables, say X and Y. Generally, random variables are represented in capital letters. It is defined

as covariance (X, Y) or $\text{COV}(X, Y)$ and is used to measure the variance between two dimensions. The formula for finding co-variance for specific x , and y are:

$$\text{cov}(X, Y) = \frac{1}{N} \sum_{i=1}^N (x_i - E(X))(y_i - E(Y))$$

Here, x_i and y_i are data values from X and Y . $E(X)$ and $E(Y)$ are the mean values of x_i and y_i . N is the number of given data. Also, the $\text{COV}(X, Y)$ is same as $\text{COV}(Y, X)$.

Example 2.6: Find the covariance of data $X = \{1, 2, 3, 4, 5\}$ and $Y = \{1, 4, 9, 16, 25\}$.

Solution: $\text{Mean}(X) = E(X) = \frac{15}{5} = 3$, $\text{Mean}(Y) = E(Y) = \frac{55}{5} = 11$. The covariance is computed using Eq. (2.17) as:

$$\frac{(1-3)(1-11) + (2-3)(4-11) + (3-3)(9-11) + (4-3)(16-11) + (5-3)(25-11)}{5} = 12$$

The covariance between X and Y is 12. It can be normalized to a value between -1 and $+1$. This is done by dividing it by the correlation of variables. This is called Pearson correlation coefficient. Sometimes, $N - 1$ is also can be used instead of N . In that case, the covariance is $60/4 = 15$.

If the given attributes are $X = (x_1, x_2, \dots, x_N)$ and $Y = (y_1, y_2, \dots, y_N)$, then the Pearson correlation coefficient, that is denoted as r , is given as: (σ_X, σ_Y are the standard deviations of X and Y .)

$$r = \frac{\text{COV}(X, Y)}{\sigma_X \sigma_Y} \quad (2.18)$$

2.7 MULTIVARIATE STATISTICS

In machine learning, almost all datasets are multivariable. Multivariate data is the analysis of more than two observable variables, and often, thousands of multiple measurements need to be conducted for one or more subjects. Multivariate data has three or more variables. The aim of the multivariate analysis is much more. They are regression analysis, factor analysis and multivariate analysis of variance.

Heatmap A heat map is a graphical representation of data where individual values are represented by colors. Heat maps are often used in data analysis and visualization to show patterns, density, or intensity of data points in a two-dimensional grid.

Example: Let's consider a heat map to display the average temperatures (in $^{\circ}\text{C}$) across different regions in a country over a week. Each cell in the heat map will represent a temperature for a specific region on a specific day. This is useful to quickly identify trends, such as higher temperatures in certain regions or specific days with unusual weather patterns. The **color gradient** (from blue to red) indicates the temperature range: cooler colors represent lower temperatures, while warmer colors represent higher temperatures.

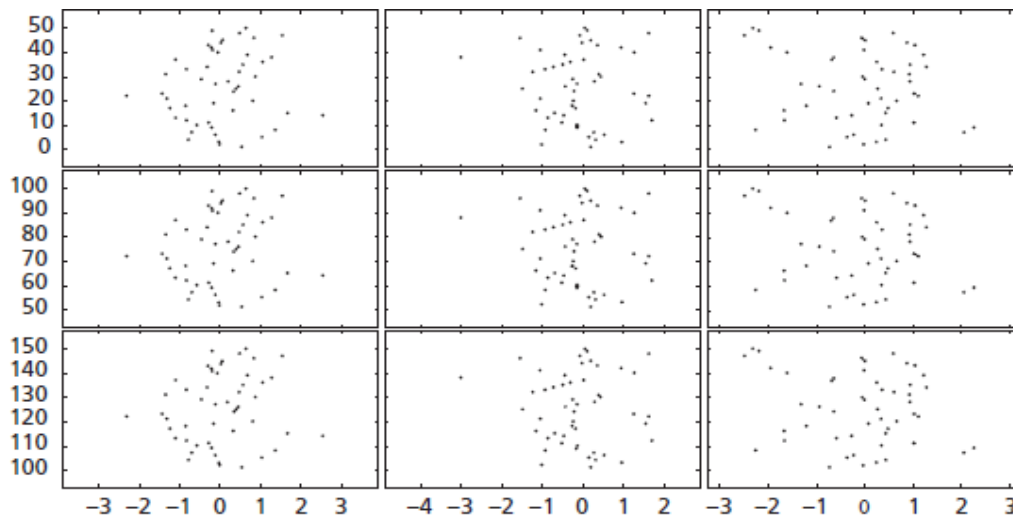
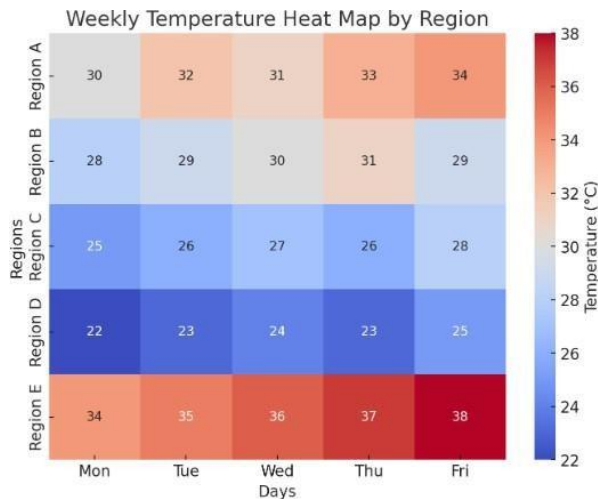


Figure 2.14: Pairplot for Random Data

Pairplot

Pairplot or scatter matrix is a data visual technique for multivariate data. A scatter matrix consists of several pair-wise scatter plots of variables of the multivariate data. A random matrix of three columns is chosen and the relationships of the columns is plotted as a pairplot (or scatter matrix) as shown in Figure 2.14.

2.8 ESSENTIAL MATHEMATICS FOR MULTIVARIATE DATA

Machine learning involves many mathematical concepts from the domain of Linear algebra, Statistics, Probability and Information theory. The subsequent sections discuss important aspects of linear algebra and probability.

2.8.1 Linear Systems and Gaussian Elimination for Multivariate Data

A linear system of equations is a group of equations with unknown variables. Let $Ax = y$, then the solution x is given as: $x = y/A = A^{-1}y$. This is true if y is not zero and A is not zero. The logic can be extended for N-

set of equations with 'n' unknown variables. It means if $A =$ and $y = (y_1 y_2 \dots y_n)$, then the unknown variable x can be computed as: $x = y/A = A^{-1}y$

If there is a unique solution, then the system is called **consistent independent**. If there are various solutions, then the system is called **consistent dependant**. If there are no solutions and if the equations are contradictory, then the system is called **inconsistent**.

For solving large number of system of equations, **Gaussian elimination** can be used. The procedure for applying Gaussian elimination is given as follows:

1. **Write the given matrix.**
2. **Append vector y to the matrix A . This matrix is called augmentation matrix.**
3. **Keep the element a_{11} as pivot and eliminate all a_{i1} in second row using the matrix operation,**

$R_2 - (a_{21}/a_{11})$, here R_2 is the 2nd row and (a_{21}/a_{11}) is called the multiplier.

The same logic can be used to remove a_{i1} in all other equations.

4. **Repeat the same logic and reduce it to reduced echelon form. Then, the unknown variable as:**

$$x_n = \frac{y_n}{a_{nn}}$$

5. **Then, the remaining unknown variables can be found by back-substitution as:**

$$x_{n-1} = \frac{y_{n-1} - a_{n-1,n} \cdot x_n}{a_{n-1,n-1}}$$

To facilitate the application of Gaussian elimination method, the following row operations are applied:

1. Swapping the rows
2. Multiplying or dividing a row by a constant
3. Replacing a row by adding or subtracting a multiple of another row to it

These concepts are illustrated in Example 2.8.

Example 2.8: Solve the following set of equations using Gaussian Elimination method.

$$2x_1 + 4x_2 = 6$$

$$4x_1 + 3x_2 = 7$$

Solution: Rewrite this in matrix form as follows:

$$\begin{pmatrix} 2 & 4 & | & 6 \\ 4 & 3 & | & 7 \end{pmatrix}$$

$$\sim \begin{pmatrix} 2 & 4 & | & 6 \\ 4 & 3 & | & 7 \end{pmatrix} R_1 = \frac{R_1}{2}$$

Apply the transformation by dividing the row 1 by 2. There are no general guidelines of row operations other than reducing the given matrix to row echelon form. The operator $\sim$ means reducing to. The above matrix can further be reduced as follows:

$$\sim \begin{pmatrix} 1 & 2 & | & 3 \\ 4 & 3 & | & 7 \end{pmatrix} R_2 = R_2 - 4R_1$$

$$\sim \begin{pmatrix} 1 & 2 & | & 3 \\ 0 & -5 & | & -5 \end{pmatrix} R_2 = R_2 / -5$$

$$\sim \begin{pmatrix} 1 & 2 & | & 3 \\ 0 & 1 & | & 1 \end{pmatrix} R_1 = R_1 - 2R_2$$

$$\sim \begin{pmatrix} 1 & 0 & | & 1 \\ 0 & 1 & | & 1 \end{pmatrix}$$

Therefore, in the reduced echelon form, it can be observed that:

$$x_2 = 1$$

$$x_1 = 1$$

2.8.2 Matrix Decomposition

It is often necessary to reduce a matrix to its constituent parts so that complex matrix operations can be performed.

Then, the matrix A can be decomposed as: $A = Q \Lambda Q^T$

where, Q is the matrix of eigen vectors, Λ is the diagonal matrix and Q^T is the transpose of matrix Q.

LU Decomposition

One of the simplest matrix decomposition is LU decomposition where the matrix A can be decomposed matrices: $A = LU$. Here, L is the lower triangular matrix and U is the upper triangular matrix. The decomposition can be done using Gaussian elimination method as discussed in the previous section. First, an identity matrix is augmented to the given matrix. Then, row operations and Gaussian elimination is applied to reduce the given matrix to get matrices L and U. Example 2.9 illustrates the application of Gaussian elimination to get LU.

Example 2.9: Find LU decomposition of the given matrix:

$$A = \begin{pmatrix} 1 & 2 & 4 \\ 3 & 3 & 2 \\ 3 & 4 & 2 \end{pmatrix}$$

Solution: First, augment an identity matrix and apply Gaussian elimination. The steps are as shown in:

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 4 \\ 3 & 3 & 2 \\ 3 & 4 & 2 \end{bmatrix} \quad \boxed{\text{Initial Matrix}}$$

$$\begin{bmatrix} 1 & 0 & 0 \\ 3 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 4 \\ 0 & -3 & -10 \\ 3 & 4 & 2 \end{bmatrix} \quad \boxed{R_2 = R_2 - 3R_1}$$

$$\begin{bmatrix} 1 & 0 & 0 \\ 3 & 1 & 0 \\ 3 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 4 \\ 0 & -3 & -10 \\ 0 & -2 & -10 \end{bmatrix} \quad \boxed{R_3 = R_3 - 3R_1}$$

$$\begin{bmatrix} 1 & 0 & 0 \\ 3 & 1 & 0 \\ 3 & \frac{2}{3} & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 4 \\ 0 & -3 & -10 \\ 0 & 0 & -\frac{10}{3} \end{bmatrix} \quad \boxed{R_3 = R_3 - \frac{2}{3}R_2}$$

Now, it can be observed that the first matrix is L as it is the lower triangular matrix whose values are the determiners used in the reduction of equations above such as 3, 3 and $2/3$.

The second matrix is U , the upper triangular matrix whose values are the values of the reduced matrix because of Gaussian elimination.

$$L = \begin{pmatrix} 1 & 0 & 0 \\ 3 & 1 & 0 \\ 3 & \frac{2}{3} & 1 \end{pmatrix} \text{ and } U = \begin{pmatrix} 1 & 2 & 4 \\ 0 & -3 & -10 \\ 0 & 0 & -\frac{10}{3} \end{pmatrix}.$$

Introduction to Machine Learning and Probability/Statistics

- **Importance:** Machine learning relies heavily on statistics and probability to make predictions and analyze data.
- **Statistics in ML:** Key for understanding data patterns, measuring relationships, and quantifying uncertainties.

Probability Distributions

- **Definition:** A probability distribution describes the likelihood of various outcomes for a variable XXX .
- **Types:**

- **Discrete Probability Distributions:** For countable events (e.g., binomial, Poisson).
- **Continuous Probability Distributions:** For measurable events on a continuum (e.g., normal, exponential).

Continuous Probability Distributions

1. Normal Distribution (Gaussian Distribution)

- **Shape:** Bell curve, symmetric around the mean.
- **Characteristics:** Defined by mean μ and standard deviation σ .
- Probability Density Function (PDF)

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

- **Applications:** Common in natural data (e.g., heights, exam scores).
- **Z-score:** Standardizes data points. $Z = (X - \mu) / \sigma$

2. Uniform Distribution (Rectangular Distribution)

- **Definition:** Equal probability for all outcomes within range $[a, b]$.
- **PDF :**

$$f(x) = \frac{1}{b - a} \quad \text{for } a \leq x \leq b$$

3. Exponential Distribution

Definition: Models time between events in a Poisson process

$$f(x; \lambda) = \lambda e^{-\lambda x} \quad \text{for } x \geq 0$$

Discrete Probability Distributions

1 Binomial Distribution

- **Definition:** For trials with two outcomes (success/failure).
- **Formula for Probability of k Successes in n Trials:**

$$P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$$

2 Poisson Distribution

- **Definition:** Models the number of events in a fixed interval of time.
- **PDF**

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}$$

3 Bernoulli Distribution

- **Definition:** Models a single trial with two outcomes (success/failure).
- **Probability Mass Function (PMF)**

$$P(X = x) = p^x (1 - p)^{1-x} \quad \text{for } x = 0 \text{ or } 1$$

Density Estimation

- **Goal:** Estimate the probability density function (PDF) of data.
- **Types:**
 - **Parametric Density Estimation:** Assumes a known distribution (e.g., Gaussian) and estimates parameters.
 - **Non-Parametric Density Estimation:** Does not assume a fixed distribution (e.g., Parzen window, k-Nearest Neighbors)

Parametric Density Estimation

1 Maximum Likelihood Estimation (MLE)

- **Definition:** A method for estimating the parameters of a distribution by maximizing the likelihood function.
- **Likelihood Function:** Maximize $L(\theta)$ for parameter θ

$$L(\theta) = P(X|\theta)$$

Gaussian Mixture Model (GMM) and Expectation-Maximization (EM) Algorithm

- **GMM:** A probabilistic model assuming data is generated from a mixture of Gaussian distributions.
- **EM Algorithm:**
 - **E-Step:** Estimate the distribution parameters for each latent variable.
 - **M-Step:** Optimize parameters using MLE.
- **Iteration:** Repeat until convergence.

Non-Parametric Density Estimation Methods

1 Parzen Window

- **Definition:** A non-parametric technique that estimates the PDF based on local samples.
- **Example:** Uses a kernel function like Gaussian around each data point.

2 k-Nearest Neighbors (KNN)

- **Definition:** Estimates density by considering the kkk closest neighbors.
- **Application:** Frequently used in classification tasks.

2.10 FEATURE ENGINEERING AND DIMENSIONALITY REDUCTION TECHNIQUES

Features are attributes. Feature engineering is about determining the subset of features that form an important part of the input that improves the performance of the model, be it classification or any other model in machine learning.

Feature engineering deals with two problems – Feature Transformation and Feature Selection. Feature transformation is extraction of features and creating new features that may be helpful in increasing performance. For example, the height and weight may give a new attribute called Body Mass Index (BMI).

Feature subset selection is another important aspect of feature engineering that focuses on selection of features to reduce the time but not at the cost of reliability.

The features can be removed based on two aspects:

1.Feature relevancy – Some features contribute more for classification than other features. For example, a mole on the face can help in face detection than common features like nose. In simple words, the features should be relevant.

Feature redundancy – Some features are redundant. For example, when a database table has a field called Date of birth, then age field is not relevant as age can be computed easily from date of birth.

So, the procedure is:

- 1.Generate all possible subsets**
- 2.Evaluate the subsets and model performance**
- 3.Evaluate the results for optimal feature selection**

Filter-based selection uses statistical measures for assessing features. In this approach, no learning algorithm is used. Correlation and information gain measures like mutual information and entropy are all examples of this approach.

Wrapper-based methods use classifiers to identify the best features. These are selected and evaluated by the learning algorithms. This procedure is computationally intensive but has superior performance.

2.10.1 Stepwise Forward Selection

This procedure starts with an empty set of attributes. Every time, an attribute is tested for statistical significance for best quality and is added to the reduced set. This process is continued till a good reduced set of attributes is obtained.

2.10.2 Stepwise Backward Elimination

This procedure starts with a complete set of attributes. At every stage, the procedure removes the worst attribute from the set, leading to the reduced set.

2.10.3 Principal Component Analysis

The idea of the principal component analysis (PCA) or KL transform is to transform a given set of measurements to a new set of features so that the features exhibit high information packing properties. This leads to a reduced and compact set of features. Consider a group of random vectors of the form:

The mean vector of the set of random vectors is defined as:

The operator E refers to the expected value of the population. This is calculated theoretically using the probability density functions (PDF) of the elements x_i and the joint probability density functions between the elements x_i and x_j . From this, the covariance matrix can be calculated as:

The mapping of the vectors x to y using the transformation can now be described as:

This transform is also called as Karhunen-Loeve or Hotelling transform. The original vector x can now be reconstructed as follows:

If K largest eigen values are used, the recovered information would be:

The PCA algorithm is as follows:

1. The target dataset x is obtained
2. The mean is subtracted from the dataset. Let the mean be m . Thus, the adjusted dataset is $X - m$. The objective of this process is to transform the dataset with zero mean.
3. The covariance of dataset x is obtained. Let it be C .
4. Eigen values and eigen vectors of the covariance matrix are calculated.
5. The eigen vector of the highest eigen value is the principal component of the dataset. The eigen values are arranged in a descending order. The feature vector is formed with these eigen vectors in its columns.
Feature vector = {eigen vector1, eigen vector2, ... , eigen vectorn}
6. Obtain the transpose of feature vector. Let it be A .
7. PCA transform is $y = A \times (x - m)$, where x is the input dataset, m is the mean, and A is the transpose of the feature vector.

The original data can be retrieved using the formula given below:

The new data is a dimensionally reduced matrix that represents the original data.

Figure 2.15. The scree plot indicates that only 6 out of 246 attributes are important.

From Figure 2.15, one can infer the relevance of the attributes. The scree plot indicates that the first attribute is more important than all other attributes.

2.10.4 Linear Discriminant Analysis

Linear Discriminant Analysis (LDA) is also a feature reduction technique like PCA. The focus of LDA is to project higher dimension data to a line (lower dimension data). LDA is also used to classify the data. Let there be two classes, c_1 and c_2 . Let m_1 and m_2 be the mean of the patterns of two classes. The mean of the class c_1 and c_2 can be computed as:

The aim of LDA is to optimize the function:

2.10.5 Singular Value Decomposition

Singular Value Decomposition (SVD) is another useful decomposition technique. Let A be the matrix, then the matrix A can be decomposed as:

Here, A is the given matrix of dimension $m \times n$, U is the orthogonal matrix whose dimension is $m \times m$, S is the diagonal matrix of dimension $n \times n$, and V is the orthogonal matrix. The procedure for finding decomposition matrix is given as follows:

1. For a given matrix, find AA^T

2. Find eigen values of AA^T

3. Sort the eigen values in a descending order. Pack the eigen vectors as a matrix U .

4. Arrange the square root of the eigen values in diagonal. This matrix is diagonal matrix, S .

5. Find eigen values and eigen vectors for $A^T A$. Find the eigen value and pack the eigen vector as a matrix called V .

Thus, $A = USV^T$. Here, U and V are orthogonal matrices. The columns of U and V are left and right singular values, respectively. SVD is useful in compression, as one can decide to retain only a certain component instead of the original matrix A as:

Based on the choice of retention, the compression can be controlled.

CHAPTER 3 - BASICS OF LEARNING THEORY

3.3 DESIGN OF A LEARNING SYSTEM

A system that is built around a learning algorithm is called a learning system. The design of systems focuses on these steps:

1. Choosing a training experience
2. Choosing a target function
3. Representation of a target function
4. Function approximation

3.4 INTRODUCTION TO CONCEPT LEARNING

Concept learning is a learning strategy that involves acquiring abstract knowledge or inferring a general concept based on the given training samples. It aims to derive a category or classification from the data, facilitating abstraction and generalization. In machine learning, concept learning is about finding a function that categorizes or labels instances correctly based on the observed features.

Concept learning requires three things:

1. Input – Training dataset which is a set of training instances, each labeled with the name of a concept or category to which it belongs. Use this past experience to train and build the model.
2. Output – Target concept or Target function f . It is a mapping function $f(x)$ from input x to output y . It is to determine the specific features or common features to identify an object. In other words, it is to find the hypothesis to determine the target concept. For e.g., the specific set of features to identify an elephant from all animals.
3. Test – New instances to test the learned model.

3.4.1 Representation of a Hypothesis

A hypothesis, denoted by h , is an approximation of the target function f . It represents the relationship between independent attributes (input features) and the dependent attribute (output or label) of the training instances. The hypothesis acts as the predicted model that maps inputs to outputs effectively. In concept learning, each hypothesis is represented as a conjunction (AND combination) of attribute conditions in the antecedent part, defining specific constraints on attributes to classify instances accurately.

3.4.2 Hypothesis Space

Hypothesis space is the set of all possible hypotheses that approximates the target function f .

The subset of hypothesis space that is consistent with all-observed training instances is called as **Version Space**.

3.4.3 Heuristic Space Search

Heuristic search is a search strategy that finds an optimized hypothesis/solution to a problem by iteratively improving the hypothesis/solution based on a given heuristic function or a cost measure.

3.4.4 Generalization and Specialization

Searching the Hypothesis Space

There are two ways of learning the hypothesis, consistent with all training instances from the large hypothesis space.

1. Specialization – General to Specific learning
2. Generalization – Specific to General learning

Generalization – Specific to General Learning This learning methodology will search through the hypothesis space for an approximate hypothesis by generalizing the most specific hypothesis.

Specialization – General to Specific Learning This learning methodology will search through the hypothesis space for an approximate hypothesis by specializing the most general hypothesis.

3.4.5 Hypothesis Space Search by Find-S Algorithm

Algorithm 3.1: Find-S

Input: Positive instances in the Training dataset

Output: Hypothesis ' h '

1. Initialize ' h ' to the most specific hypothesis.

$h = \langle \varphi \quad \varphi \quad \varphi \quad \varphi \quad \varphi \quad \dots \rangle$

2. Generalize the initial hypothesis for the first positive instance [Since ' h ' is more specific].
3. For each subsequent instances:

If it is a positive instance,

Check for each attribute value in the instance with the hypothesis ' h '.

If the attribute value is the same as the hypothesis value, then do nothing,

Else if the attribute value is different than the hypothesis value, change it to '?' in ' h '.

Else if it is a negative instance,

Ignore it.

Limitations of Find-S Algorithm

1. Find-S algorithm tries to find a hypothesis that is consistent with positive instances, ignoring all negative instances. As long as the training dataset is consistent, the hypothesis found by this algorithm may be consistent.
2. The algorithm finds only one unique hypothesis, wherein there may be many other hypotheses that are consistent with the training dataset.
3. Many times, the training dataset may contain some errors; hence such inconsistent data instances can mislead this algorithm in determining the consistent hypothesis since it ignores negative instances.

3.4.6 Version Spaces

The version space contains the subset of hypotheses from the hypothesis space that is consistent with all training instances in the training dataset.

List-Then-Eliminate Algorithm

Algorithm 3.2: List-Then-Eliminate

Input: Version Space – a list of all hypotheses

Output: Set of consistent hypotheses

1. Initialize the version space with a list of hypotheses.
2. For each training instance,
 - remove from version space any hypothesis that is inconsistent.

Candidate Elimination Algorithm

Algorithm 3.3: Candidate Elimination

Input: Set of instances in the Training dataset

Output: Hypothesis G and S

1. Initialize G , to the maximally general hypotheses.
2. Initialize S , to the maximally specific hypotheses.
 - Generalize the initial hypothesis for the first positive instance.
3. For each subsequent new training instance,
 - If the instance is **positive**,
 - o Generalize S to include the positive instance,
 - Check the attribute value of the positive instance and S ,
 - If the attribute value of positive instance and S are different, fill that field value with '?'.
 - If the attribute value of positive instance and S are same, then do no change.
 - o Prune G to exclude all inconsistent hypotheses in G with the positive instance.
 - If the instance is **negative**,
 - o Specialize G to exclude the negative instance,
 - Add to G all minimal specializations to exclude the negative example and be consistent with S .
 - If the attribute value of S and the negative instance are different, then fill that attribute value with S value.
 - If the attribute value of S and negative instance are same, no need to update ' G ' and fill that attribute value with '?'.
 - o Remove from S all inconsistent hypotheses with the negative instance.

The diagrammatic representation of deriving the version space is shown below:

Table 3.2: Training Dataset

CGPA	Interactiveness	Practical Knowledge	Communication Skills	Logical Thinking	Interest	Job Offer
≥9	Yes	Excellent	Good	Fast	Yes	Yes
≥9	Yes	Good	Good	Fast	Yes	Yes
≥8	No	Good	Good	Fast	No	No
≥9	Yes	Good	Good	Slow	No	Yes

Deriving the Version Space

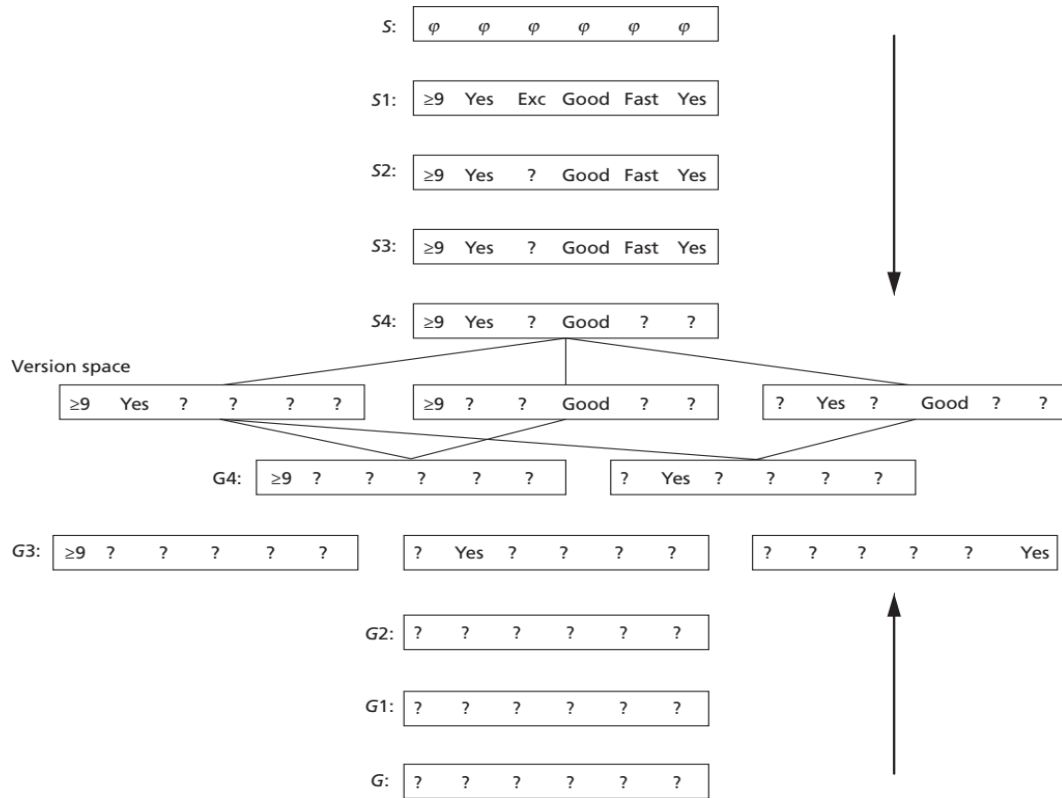


Figure 3.2: Deriving the Version Space

MODULE 3

CHAPTER 4

SIMILARITY-BASED LEARNING

Similarity or Instance-based Learning

Similarity-based classifiers use similarity measures to locate the nearest neighbors and classify a test instance which works in contrast with other learning mechanisms such as decision trees or neural networks. Similarity-based learning is also called as Instance-based learning/Just-in time learning since it does not build an abstract model of the training instances and performs lazy learning when classifying a new instance. This learning mechanism simply stores all data and uses it only when it needs to classify an unseen instance. The advantage of using this learning is that processing occurs only when a request to classify a new instance is given. This methodology is particularly useful when the whole dataset is not available in the beginning but collected in an incremental manner.

Difference between Instance-and Model-based Learning

Table 4.1: Differences between Instance-based Learning and Model-based Learning

Instance-based Learning	Model-based Learning
Lazy Learners	Eager Learners
Processing of training instances is done only during testing phase	Processing of training instances is done during training phase

Instance-based Learning	Model-based Learning
No model is built with the training instances before it receives a test instance	Generalizes a model with the training instances before it receives a test instance
Predicts the class of the test instance directly from the training data	Predicts the class of the test instance from the model built
Slow in testing phase	Fast in testing phase
Learns by making many local approximations	Learns by creating global approximation

Some examples of Instance-based Learning **algorithms** are:

KNN

Variants of KNN

Locally weighted regression

Learning vector quantization

Self-organizing maps

RBF networks

Nearest-Neighbor Learning

- A powerful classification algorithm used in pattern recognition.
- K nearest neighbors stores all available cases and classifies new cases based on a similarity measure (e.g distance function)
- One of the top data mining algorithms used today.
- A non-parametric lazy learning algorithm (An Instance based Learning method).
- Used for both classification and regression problems.
-

Algorithm 4.2: k-NN

Algorithm 4.1: k-NN

Inputs: Training dataset T , distance metric d , Test instance t , the number of nearest neighbors k

Output: Predicted class or category

Prediction: For test instance t ,

1. For each instance i in T , compute the distance between the test instance t and every other instance i in the training dataset using a distance metric (Euclidean distance).
[Continuous attributes - Euclidean distance between two points in the plane with coordinates (x_1, y_1) and (x_2, y_2) is given as $\text{dist}((x_1, y_1), (x_2, y_2)) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$]
[Categorical attributes (Binary) - Hamming Distance: If the value of the two instances is same, the distance d will be equal to 0 otherwise $d = 1$.]
2. Sort the distances in an ascending order and select the first k nearest training data instances to the test instance.
3. Predict the class of the test instance by majority voting (if target attribute is discrete valued) or mean (if target attribute is continuous valued) of the k selected nearest instances.

4.3 Weighted k-Nearest-Neighbor Algorithm

The weighted KNN is an extension of k-NN. It chooses the neighbors by using the weighted distance. In weighted kNN, the nearest k points are given a weight using a function called as the kernel function. The intuition behind weighted kNN, is to give more weight to the points which are nearby and less weight to the points which are farther away.

Algorithm 4.2: Weighted k-NN

Inputs: Training dataset ' T ', Distance metric ' d ', Weighting function $w(i)$, Test instance ' t ', the number of nearest neighbors ' k '

Output: Predicted class or category

Prediction: For test instance t ,

1. For each instance ' i ' in Training dataset T , compute the distance between the test instance t and every other instance ' i ' using a distance metric (Euclidean distance).
[Continuous attributes - Euclidean distance between two points in the plane with coordinates (x_1, y_1) and (x_2, y_2) is given as $\text{dist}((x_1, y_1), (x_2, y_2)) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$]
[Categorical attributes (Binary) - Hamming Distance: If the values of two instances are the same, the distance d will be equal to 0. Otherwise $d = 1$.]
2. Sort the distances in the ascending order and select the first ' k ' nearest training data instances to the test instance.
3. Predict the class of the test instance by weighted voting technique (Weighting function $w(i)$) for the k selected nearest instances:
 - Compute the inverse of each distance of the ' k ' selected nearest instances.
 - Find the sum of the inverses.
 - Compute the weight by dividing each inverse distance by the sum. (Each weight is a vote for its associated class).
 - Add the weights of the same class.
 - Predict the class by choosing the class with the maximum vote.

4.4 Nearest Centroid Classifier

The Nearest Centroids algorithm assumes that the centroids in the input feature space are different for each target label. The training data is split into groups by class label, then the centroid for each group of data is calculated. Each centroid is simply the mean value of each of the input variables, so it is also called as Mean Difference classifier. If there are two classes, then two centroids or points are calculated; three classes give three centroids, and so on.

Algorithm 4.3: Nearest Centroid Classifier

Inputs: Training dataset T , Distance metric d , Test instance t

Output: Predicted class or category

1. Compute the mean/centroid of each class.
2. Compute the distance between the test instance and mean/centroid of each class (Euclidean Distance).
3. Predict the class by choosing the class with the smaller distance.

4.5 Locally Weighted Regression (LWR)

Locally Weighted Regression (LWR) is a non-parametric supervised learning algorithm that performs local regression by combining regression model with nearest neighbor's model. LWR is also referred to as a memory-based method as it requires training data while prediction but uses only the training data instances locally around the point of interest. Using nearest neighbors algorithm, we find the instances that are closest to a test instance and fit linear function to each of those ' k ' nearest instances in the local regression model. The key idea is that we need to approximate the linear functions of all ' k ' neighbors that minimize the error such that the prediction line is no more linear but rather it is a curve.

Ordinary linear regression finds out a linear relationship between the input x and the output y .
 Given training dataset T ,
 Hypothesis function $h_\beta(x)$, the predicted target output is a linear function where β_0 is the intercept and β_1 is the coefficient of x .
 It is given in Eq. (4.1) as,

$$h_\beta(x) = \beta_0 + \beta_1 x \quad (4.1)$$

The cost function is such that it minimizes the error difference between the predicted value $h_\beta(x)$ and true value ' y ' and it is given as in Eq. (4.2).

$$J(\beta) = \frac{1}{2} \sum_{i=1}^m (h_\beta(x_i) - y_i)^2 \quad (4.2)$$

where ' m ' is the number of instances in the training dataset.

Now the cost function is modified for locally weighted linear regression including the weights only for the nearest neighbor points. Hence, the cost function is given as in Eq. (4.3).

$$J(\beta) = \frac{1}{2} \sum_{i=1}^m w_i (h_\beta(x_i) - y_i)^2 \quad (4.3)$$

where w_i is the weight associated with each x_i .

The weight function used is a Gaussian kernel that gives a higher value for instances that are close to the test instance, and for instances far away, it tends to zero but never equals to zero.

w_i is computed in Eq. (4.4) as,

$$w_i = e^{\frac{-(x_i - x)^2}{2r^2}} \quad (4.4)$$

Where, r is called the bandwidth parameter and controls the rate at which w_i reduces to zero with distance from x_i .

MODULE 3

CHAPTER 5

REGRESSION ANALYSIS

1.1 Introduction to Regression

Regression analysis is a fundamental concept that consists of a set of machine learning methods that predict a continuous outcome variable (y) based on the value of one or multiple predictor variables (x).

OR

Regression analysis is a statistical method to model the relationship between a dependent (target) and independent (predictor) variables with one or more independent variables.

Regression is a supervised learning technique which helps in finding the correlation between variables.

It is mainly used for prediction, forecasting, time series modelling, and determining the causal-effect relationship between variables.

Regression shows a line or curve that passes through all the data points on target-predictor graph in such a way that the vertical distance between the data points and the regression line is minimum." The distance between data points and line tells whether a model has captured a strong relationship or not.

- Function of regression analysis is given by:

$$Y=f(x)$$

Here, y is called dependent variable and x is called independent variable.

Applications of Regression Analysis

- Sales of a goods or services
- Value of bonds in portfolio management
- Premium on insurance companies
- Yield of crop in agriculture
- Prices of real estate

1.2 INTRODUCTION TO LINEARITY, CORRELATION AND CAUSATION

A correlation is the statistical summary of the relationship between two sets of variables. It is a core part of data exploratory analysis, and is a critical aspect of numerous advanced machine learning techniques.

Correlation between two variables can be found using a **scatter** plot

There are different types of correlation:

Positive Correlation: Two variables are said to be positively correlated when their values move in the same direction. For example, in the image below, as the value for X increases, so does the value for Y at a constant rate.

Negative Correlation: Finally, variables X and Y will be negatively correlated when their values change in opposite directions, so here as the value for X increases, the value for Y decreases at a constant rate.

Neutral Correlation: No relationship in the change of variables X and Y. In this case, the values are completely random and do not show any sign of correlation, as shown in the following image:

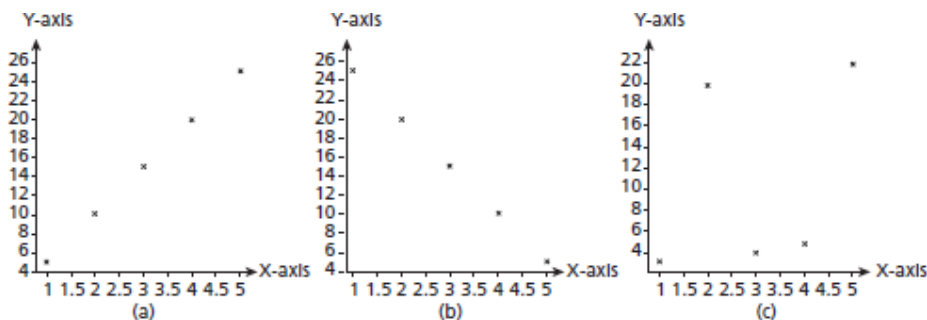


Figure 5.1: Examples of (a) Positive Correlation (b) Negative Correlation
(c) Random Points with No Correlation

Causation

Causation is about relationship between two variables as x causes y. This is called x implies y. Regression is different from causation. Causation indicates that one event is the result of the occurrence of the other event; i.e. there is a causal relationship between the two events.

Linear and Non-Linear Relationships

The relationship between input features (variables) and the output (target) variable is fundamental. These concepts have significant implications for the choice of algorithms, model complexity, and predictive performance.

Linear relationship creates a straight line when plotted on a graph, a Non-Linear relationship does not create a straight line but instead creates a curve.

Example:

Linear-the relationship between the hours spent studying and the grades obtained in a class.

Non-Linear-

Linearity:

Linear Relationship: A linear relationship between variables means that a change in one variable is associated with a proportional change in another variable. Mathematically, it can be represented as $y = a * x + b$, where y is the output, x is the input, and a and b are constants.

Linear Models: Goal is to find the best-fitting line (plane in higher dimensions) to the data points. Linear models are interpretable and work well when the relationship between variables is close to being linear.

Limitations: Linear models may perform poorly when the relationship between variables is non-linear. In such cases, they may underfit the data, meaning they are too simple to capture the underlying patterns.

Non-Linearity:

Non-Linear Relationship: A non-linear relationship implies that the change in one variable is not proportional to the change in another variable. Non-linear relationships can take various forms, such as quadratic, exponential, logarithmic, or arbitrary shapes.

Non-Linear Models: Machine learning models like decision trees, random forests, support vector machines with non-linear kernels, and neural networks can capture non-linear relationships. These models are more flexible and can fit complex data patterns.

Benefits: Non-linear models can perform well when the underlying relationships in the data are complex or when interactions between variables are non-linear. They have the capacity to capture intricate patterns.

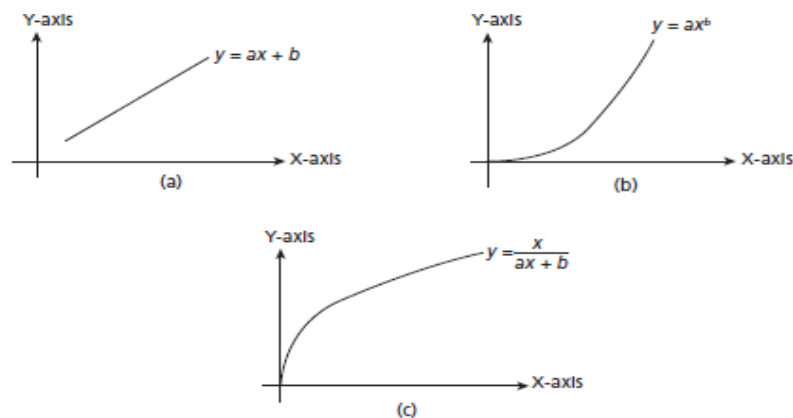


Figure 5.2: (a) Example of Linear Relationship of the Form $y = ax + b$ (b) Example of a Non-linear Relationship of the Form $y = ax^b$ (c) Examples of a Non-linear Relationship $y = \frac{x}{ax + b}$

Types of Regression

Types of Regression Methods

The classification of regression methods is shown in Figure 5.3.

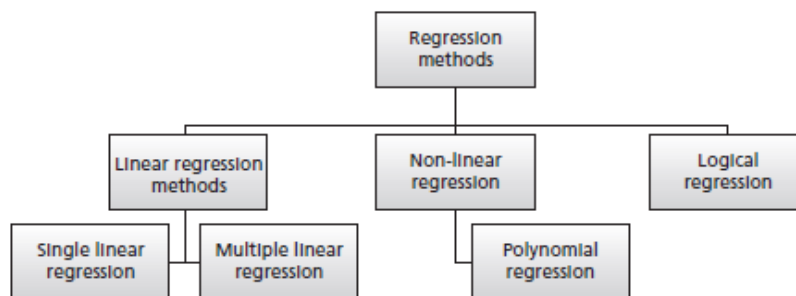


Figure 5.3: Types of Regression Methods

Linear Regression:

Single Independent Variable: Linear regression, also known as simple linear regression, is used when there is a single independent variable (predictor) and one dependent variable (target).

Equation: The linear regression equation takes the form: $Y = \beta_0 + \beta_1 X + \varepsilon$, where Y is the dependent variable, X is the independent variable, β_0 is the intercept, β_1 is the slope (coefficient), and ε is the error term.

Purpose: Linear regression is used to establish a linear relationship between two variables and make predictions based on this relationship. It's suitable for simple scenarios where there's only one predictor.

Multiple Regression:

Multiple Independent Variables: Multiple regression, as the name suggests, is used when there are two or more independent variables (predictors) and one dependent variable (target).

Equation: The multiple regression equation extends the concept to multiple predictors: $Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n + \varepsilon$, where Y is the dependent variable, $X_1, X_2, \dots, X_n$ are the independent variables, β_0 is the intercept, $\beta_1, \beta_2, \dots, \beta_n$ are the coefficients, and ε is the error term.

Purpose: Multiple regression allows you to model the relationship between the dependent variable and multiple predictors simultaneously. It's used when there are multiple factors that may influence the target variable, and you want to understand their combined effect and make predictions based on all these factors.

Polynomial Regression:

Use: Polynomial regression is an extension of multiple regression used when the relationship between the independent and dependent variables is non-linear.

Equation: The polynomial regression equation allows for higher-order terms, such as quadratic or cubic terms: $Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \dots + \beta_n X^n + \varepsilon$. This allows the model to fit a curve rather than a straight line.

Logistic Regression:

Use: Logistic regression is used when the dependent variable is binary (0 or 1). It models the probability of the dependent variable belonging to a particular class.

Equation: Logistic regression uses the logistic function (sigmoid function) to model probabilities: $P(Y=1) = 1 / (1 + e^{(-z)})$, where z is a linear combination of the independent variables: $z = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n$. It transforms this probability into a binary outcome.

Lasso Regression (L1 Regularization):

Use: Lasso regression is used for feature selection and regularization. It penalizes the absolute values of the coefficients, which encourages sparsity in the model.

Objective Function: Lasso regression adds an L1 penalty to the linear regression loss function: $\text{Lasso} = \text{RSS} + \lambda \sum |\beta_i|$, where RSS is the residual sum of squares, λ is the regularization strength, and $|\beta_i|$ represents the absolute values of the coefficients.

Ridge Regression (L2 Regularization):

Use: Ridge regression is used for regularization to prevent overfitting in multiple regression. It penalizes the square of the coefficients.

Objective Function: Ridge regression adds an L2 penalty to the linear regression loss function: $\text{Ridge} = \text{RSS} + \lambda \sum (\beta_i^2)$, where RSS is the residual sum of squares, λ is the regularization strength, and (β_i^2) represents the square of the coefficients.

Limitations of Regression

1. **Outliers** – Outliers are abnormal data. It can bias the outcome of the regression model, as outliers push the regression line towards it.
2. **Number of cases** – The ratio of independent and dependent variables should be at least 20 : 1. For every explanatory variable, there should be at least 20 samples. Atleast five samples are required in extreme cases.
3. **Missing data** – Missing data in training data can make the model unfit for the sampled data.
4. **Multicollinearity** – If exploratory variables are highly correlated (0.9 and above), the regression is vulnerable to bias. Singularity leads to perfect correlation of 1. The remedy is to remove exploratory variables that exhibit correlation more than 1. If there is a tie, then the tolerance ($1 - R^2$) is used to eliminate variables that have the greatest value.

1.3 INTRODUCTION TO LINEAR REGRESSION

Linear regression model can be created by fitting a line among the scattered data points. The line is of the form:

The diagram shows the linear regression equation $Y_i = \beta_0 + \beta_1 X_i + \epsilon_i$ with the following labels and annotations:

- Dependent Variable**: Points to Y_i .
- Population Y intercept**: Points to β_0 .
- Population Slope Coefficient**: Points to β_1 .
- Independent Variable**: Points to X_i .
- Random Error term**: Points to ϵ_i .
- Linear component**: A bracket under $\beta_0 + \beta_1 X_i$.
- Random Error component**: A bracket under ϵ_i .

The assumptions of linear regression are listed as follows:

1. The observations (y) are random and are mutually independent.
2. The difference between the predicted and true values is called an error. The error is also mutually independent with the same distributions such as normal distribution with zero mean and constant variables.
3. The distribution of the error term is independent of the joint distribution of explanatory variables.
4. The unknown parameters of the regression models are constants.

Ordinary Least Square Approach

The ordinary least squares (OLS) algorithm is a method for estimating the parameters of a linear regression model. **Aim:** To find the values of the linear regression model's parameters (i.e., the coefficients) that minimize the sum of the squared residuals.

In mathematical terms, this can be written as: **Minimize** $\sum (y_i - \hat{y}_i)^2$

where y_i is the actual value, $\hat{y}_i$ is the predicted value.

A linear regression model used for determining the value of the response variable, $\hat{y}$, can be represented as the following equation.

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n + e$$

- where: y - is the dependent variable, b_0 is the intercept, e is the error term
- $b_1, b_2, \dots, b_n$ are the coefficients of the independent variables $x_1, x_2, \dots, x_n$

The coefficients $b_1, b_2, \dots, b_n$ can also be called the **coefficients of determination**. The goal of the OLS method can be used to estimate the unknown parameters (**$b_1, b_2, \dots, b_n$**) by minimizing the sum of squared residuals (RSS). The sum of squared residuals is also termed the sum of squared error (SSE).

This method is also known as the **least-squares method** for regression or linear regression. Mathematically the line of equations for points are:

$$y_1 = (a_0 + a_1x_1) + e_1$$

$$y_2 = (a_0 + a_1x_2) + e_2 \quad \text{and so on}$$

$$\dots\dots y_n = (a_0 + a_1x_n) + e_n.$$

$$\text{In general } e_i = y_i - (a_0 + a_1x_i)$$

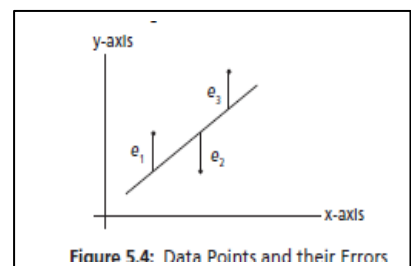


Figure 5.4: Data Points and their Errors

Here, the terms $(e_1, e_2, \dots, e_n)$ are error associated with the data points and denote the difference between the true value of the observation and the point on the line. This is also called as residuals. The residuals can be positive, negative or zero.

A regression line is the line of best fit for which the sum of the squares of residuals is minimum. The minimization can be done as minimization of individual errors by finding the parameters a_0 and a_1 such that:

$$E = \sum_{i=1}^n e_i = \sum_{i=1}^n (y_i - (a_0 + a_1 x_i)) \quad (5.5)$$

Or as the minimization of sum of absolute values of the individual errors:

$$E = \sum_{i=1}^n |e_i| = \sum_{i=1}^n |(y_i - (a_0 + a_1 x_i))| \quad (5.6)$$

Or as the minimization of the sum of the squares of the individual errors:

$$E = \sum_{i=1}^n (e_i)^2 = \sum_{i=1}^n (y_i - (a_0 + a_1 x_i))^2 \quad (5.7)$$

Sum of the squares of the individual errors, often preferred as individual errors (positive and negative errors), do not get cancelled out and are always positive, and sum of squares results in a large increase even for a small change in the error. Therefore, this is preferred for linear regression.

Therefore, linear regression is modelled as a minimization function as follows:

$$\begin{aligned} J(a_1, a_0) &= \sum_{i=1}^n [y_i - f(x_i)]^2 \\ &= \sum_{i=1}^n [y_i - (a_0 + a_1 x_i)]^2 \end{aligned} \quad (5.8)$$

Here, $J(a_0, a_1)$ is the criterion function of parameters a_0 and a_1 . This needs to be minimized. This is done by differentiating and substituting to zero. This yields the coefficient values of a_0 and a_1 . The values of estimates of a_0 and a_1 are given as follows:

$$a_1 = \frac{(\overline{xy}) - (\bar{x})(\bar{y})}{(\overline{x^2}) - (\bar{x})^2} \quad (5.9)$$

And the value of a_0 is given as follows:

$$a_0 = (\bar{y}) - a_1 \times \bar{x} \quad (5.10)$$

Let us consider a simple problem to illustrate the usage of the above concept.

Linear Regression Example

Example 5.1: Let us consider an example where the five weeks' sales data (in Thousands) is given as shown below in Table 5.1. Apply linear regression technique to predict the 7th and 9th month sales.

Table 5.1: Sample Data

x_i (Week)	y_i (Sales in Thousands)
1	1.2
2	1.8
3	2.6
4	3.2
5	3.8

Table 5.2: Computation Table

x_i	y_i	$(x_i)^2$	$x_i \times y_i$
1	1.2	1	1.2
2	1.8	4	3.6
3	2.6	9	7.8
4	3.2	16	12.8
5	3.8	25	19
Sum = 15	Sum = 12.6	Sum = 55	Sum = 44.4
Average of (x_i)	Average of (y_i)	Average of (x_i^2)	Average of $(x_i \times y_i)$
$= \bar{x} = \frac{15}{5}$	$= \bar{y} = \frac{12.6}{5}$	$= \overline{x_i^2} = \frac{55}{5}$	$= \overline{xy} = \frac{44.4}{5}$
$= 3$	$= 2.52$	$= 11$	$= 8.88$

Let us compute the slope and intercept now using Eq. (5.9) as:

$$a_1 = \frac{8.88 - 3(2.52)}{11 - 3^2} = 0.66$$

$$a_0 = 2.52 - 0.66 \times 3 = 0.54$$

The fitted line is shown in Figure 5.5.

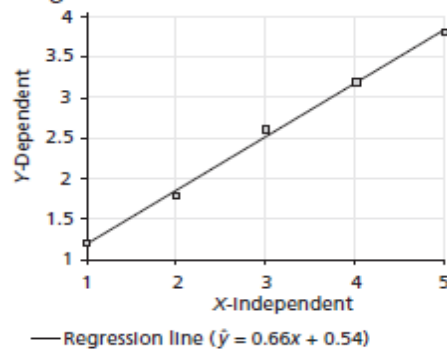


Figure 5.5: Linear Regression Model Constructed

Let us model the relationship as $y = a_0 + a_1 \times x$. Therefore, the fitted line for the above data is:
 $y = 0.54 + 0.66 \times x$.

The predicted 7th week sale would be (when $x = 7$), $y = 0.54 + 0.66 \times 7 = 5.16$ and the 12th month, $y = 0.54 + 0.66 \times 12 = 8.46$. All sales are in thousands.

Linear Regression in Matrix Form

Linear Regression in Matrix Form

Matrix notations can be used for representing the values of independent and dependent variables. This is illustrated through Example 5.2.

The Eq. (5.3) can be written in the form of matrix as follows:

$$\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} = \begin{pmatrix} 1 & x_1 \\ 1 & x_2 \\ 1 & \vdots \\ 1 & x_n \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \end{pmatrix} + \begin{pmatrix} e_1 \\ e_2 \\ \vdots \\ e_n \end{pmatrix} \quad (5.11)$$

This can be written as:

$Y = Xa + e$, where X is an $n \times 2$ matrix, Y is an $n \times 1$ vector, a is a 2×1 column vector and e is an $n \times 1$ column vector.

Example 5.2: Find linear regression of the data of week and product sales (in Thousands) given in Table 5.3. Use linear regression in matrix form.

Table 5.3: Sample Data for Regression

x_i (Week)	y_i (Product Sales in Thousands)
1	1
2	3
3	4
4	8

Solution: Here, the dependent variable X is be given as:

$$x^T = [1 \ 2 \ 3 \ 4]$$

And the independent variable is given as follows:

$$y^T = [1 \ 3 \ 4 \ 8]$$

The data can be given in matrix form as follows:

$$X = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{pmatrix}. \text{ The first column can be used for setting bias.}$$

$$\text{and } Y = \begin{pmatrix} 1 \\ 3 \\ 4 \\ 8 \end{pmatrix}$$

The regression is given as:

$$a = ((X^T X)^{-1} X^T) Y$$

The computation order of this equation is shown step by step as:

1. Computation of $(X^T X) = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \end{pmatrix} \times \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{pmatrix} = \begin{pmatrix} 4 & 10 \\ 10 & 30 \end{pmatrix}$
2. Computation of matrix inverse of $(X^T X)^{-1} = \begin{pmatrix} 4 & 10 \\ 10 & 30 \end{pmatrix}^{-1} = \begin{pmatrix} 1.5 & -0.5 \\ -0.5 & 0.2 \end{pmatrix}$
3. Computation of $((X^T X)^{-1} X^T) = \begin{pmatrix} 1.5 & -0.5 \\ -0.5 & 0.2 \end{pmatrix} \times \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \end{pmatrix} = \begin{pmatrix} 1 & 0.5 & 0 & -0.5 \\ -0.3 & -0.1 & 0.1 & 0.3 \end{pmatrix}$
4. Finally, $((X^T X)^{-1} X^T) Y = \begin{pmatrix} 1 & 0.5 & 0 & -0.5 \\ -0.3 & -0.1 & 0.1 & 0.3 \end{pmatrix} \times \begin{pmatrix} 1 \\ 3 \\ 4 \\ 8 \end{pmatrix} = \begin{pmatrix} -1.5 \\ 2.2 \end{pmatrix} = \begin{pmatrix} \text{Intercept} \\ \text{slope} \end{pmatrix}$

Thus, the substitution of values in Eq. (5.11) using the previous steps yields the fitted line as $2.2x - 1.5$.
of determination r^2 is the ratio of the explained and unexplained variations.

Standard Error

Residuals or error is the difference between the actual (y) and predicted value ($\hat{y}$).

If the residuals have normal distribution, then the mean is zero and hence it is desirable. This is a measure of variability in finding the coefficients. It is preferable that the error be less than the coefficient estimate. The standard deviation of residuals is called residual standard error. If it is zero, then it means that the model fits the data correctly.

Mean Absolute Error (MAE)

MAE is the mean of residuals. It is the difference between estimated or predicted target value and actual target incomes. It can be mathematically defined as follows:

$$MAE = \frac{1}{n} \sum_{i=0}^{n-1} |y_i - \hat{y}_i| \quad (5.12)$$

Here, $\hat{y}$ is the estimated or predicted target output and y is the actual target output, and n is the number of samples used for regression analysis.

Mean Squared Error (MSE)

It is the sum of square of residuals. This value is always positive and closer to 0. This is given mathematically as:

$$\frac{1}{n} \sum_{i=0}^{n-1} (y_i - \hat{y}_i)^2 \quad (5.13)$$

CHAPTER 5

REGRESSION ANALYSIS

2 Consider the following dataset in Table 5.11 where the week and number of working hours per week spent by a research scholar in a library are tabulated. Based on the dataset, predict the number of hours that will be spent by the research scholar in the 7th and 9th week. Apply Linear regression model.

Table 5.11

x_i (week)	1	2	3	4	5
y_i (Hours Spent)	12	18	22	28	35

Solution

The computation table is shown below:

x_i	y_i	$x_i \times x_i$	$x_i \times y_i$
1	12	1	12
2	18	4	36
3	22	9	66
4	28	16	112
5	35	25	175
Sum = 15 avg(x_i)=15/5=3	Sum = 115 avg(y_i)=115/5=23	Avg($x_i \times x_i$)=55/5=11	Avg($x_i \times y_i$)=401/5=80.2

The regression Equations are

$$a_1 = \frac{\left(\frac{\sum y_i}{n}\right) - \left(\frac{\sum x_i}{n}\right)\left(\frac{\sum y_i}{n}\right)}{\left(\frac{\sum x_i^2}{n}\right) - \left(\frac{\sum x_i}{n}\right)^2}$$

$$a_0 = \left(\frac{\sum y_i}{n}\right) - a_1 \times \bar{x}$$

$$a_1 = \frac{80.2 - 3(23)}{11 - 3^2} = \frac{80.2 - 69}{11 - 9} = \frac{11.2}{2} = 5.6$$

$$a_0 = 23 - 5.6 \times 3 = 23 - 16.8 = 6.2$$

Therefore, the regression equation is given as

$$y = 5.6 + 6.2 \times x$$

The prediction for the 7th week hours spent by the research scholar will be

$$y = 5.6 + 6.2 \times 7 = 49 \text{ hours}$$

The prediction for the 9th week hours spent by the research scholar will be

$$y = 5.6 + 6.2 \times 9 = 61.4 \approx 61 \text{ hours}$$

3 The height of boys and girls is given in the following Table 5.12.

Table 5.12: Sample Data

Height of Boys	65	70	75	78
Height of Girls	63	67	70	73

Fit a suitable line of best fit for the above data.

Solution

The computation table is shown below:

x_i	y_i	$x_i \times x_i$	$x_i \times y_i$
65	63	4225	4095
70	67	4900	4690
75	70	5625	5250
78	73	6084	5694
Sum = 288 Mean(x_i)=288/4=72	Sum = 273 Mean(y_i)=273/4=68.25	Avg ($x_i \times x_i$)=20834/4=5208.5	Avg($x_i \times y_i$)=19729/4=4932.25

The regression Equations are

$$a_1 = \frac{\left(\frac{\sum y}{n}\right) - \left(\frac{\sum x}{n}\right)\left(\frac{\sum y}{n}\right)}{\left(\frac{\sum x^2}{n}\right) - \left(\frac{\sum x}{n}\right)^2}$$

$$a_0 = \left(\frac{\sum y}{n}\right) - a_1 \times \bar{x}$$

$$a_1 = \frac{4932.25 - 72(68.25)}{5208.5 - 72^2} = \frac{18.25}{24.5} = 0.7449$$

$$a_0 = 68.25 - 0.7449 \times 72 = 68.25 - 53.6328 = 14.6172$$

Therefore, the regression line of best fit is given as

$$y = 0.7449 + 14.6172 \times x$$

4 Using multiple regression, fit a line for the following dataset shown in Table 5.13. Here, Z is the equity, X is the net sales and Y is the asset. Z is the dependent variable and X and Y are independent variables. All the data is in million dollars.

Table 5.13: Sample Data

Z	X	Y
4	12	8
6	18	12
7	22	16
8	28	36
11	35	42

Solution

The matrix X and Y is given as follows:

$$X = \begin{pmatrix} 1 & 12 & 8 \\ 1 & 18 & 12 \\ 1 & 22 & 16 \\ 1 & 28 & 36 \\ 1 & 35 & 42 \end{pmatrix}$$

$$Y = \begin{pmatrix} 4 \\ 6 \\ 7 \\ 8 \\ 11 \end{pmatrix}$$

The regression coefficients can be found as follows

$$\hat{a} = ((X^T X)^{-1} X^T) Y$$

Substituting the values one get,

$$\hat{a} = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 \\ 12 & 18 & 22 & 28 & 35 \\ 8 & 12 & 16 & 36 & 42 \end{pmatrix}^{-1} \begin{pmatrix} 1 & 12 & 8 \\ 1 & 18 & 12 \\ 1 & 22 & 16 \\ 1 & 28 & 36 \\ 1 & 35 & 42 \end{pmatrix}^T \begin{pmatrix} 4 \\ 6 \\ 7 \\ 8 \\ 11 \end{pmatrix}$$

$$= \begin{pmatrix} 5 & 115 & 114 \\ 115 & 2961 & 3142 \\ 114 & 3142 & 3524 \end{pmatrix}^{-1} \begin{pmatrix} 1 & 1 & 1 & 1 & 1 \\ 12 & 18 & 22 & 28 & 35 \\ 8 & 12 & 16 & 36 & 42 \end{pmatrix} \begin{pmatrix} 4 \\ 6 \\ 7 \\ 8 \\ 11 \end{pmatrix}$$

$$= \begin{pmatrix} -0.4135 \\ 0.39625 \\ -0.0658 \end{pmatrix}$$

Therefore, the regression line is given as

$$y = 0.39625x_1 - 0.0658x_2 - 0.4135$$

5.5 MULTIPLE LINEAR REGRESSION

Multiple regression model involves multiple predictors or independent variables and one dependent variable. This is an extension of the linear regression problem. The basic assumptions of multiple linear regression are that the independent variables are not highly correlated and hence multicollinearity problem does not exist. Also, it is assumed that the residuals are normally distributed.

For example, the multiple regression of two variables x_1 and x_2 is given as follows:

$$\begin{aligned} y &= f(x_1, x_2) \\ &= a_0 + a_1x_1 + a_2x_2 \end{aligned} \quad (5.21)$$

In general, this is given for ' n ' independent variables as:

$$\begin{aligned} y &= f(x_1, x_2, x_3, \dots, x_n) \\ &= a_0 + a_1x_1 + a_2x_2 + \dots + a_nx_n + \varepsilon \end{aligned} \quad (5.22)$$

Here, $(x_1, x_2, \dots, x_n)$ are predictor variables, y is the dependent variable, $(a_0, a_1, \dots, a_n)$ are the coefficients of the regression equation and ε is the error term. This is illustrated through Example 5.5.



Example 5.5: Apply multiple regression for the values given in Table 5.7 where weekly sales along with sales for products x_1 and x_2 are provided. Use matrix approach for finding multiple regression.

Table 5.7: Sample Data

x_1 (Product One Sales)	x_2 (Product Two Sales)	y Output Weekly Sales (in Thousands)
1	4	1
2	5	6
3	8	8
4	2	12

Solution: Here, the matrices for Y and X are given as follows:

$$X = \begin{pmatrix} 1 & 1 & 4 \\ 1 & 2 & 5 \\ 1 & 3 & 8 \\ 1 & 4 & 2 \end{pmatrix} \text{ and } Y = \begin{pmatrix} 1 \\ 6 \\ 8 \\ 12 \end{pmatrix}.$$

The coefficient of the multiple regression equation is given as $a = \begin{pmatrix} a_0 \\ a_1 \\ a_2 \end{pmatrix}$.

The regression coefficient for multiple regression is calculated the same way as linear regression:

$$\hat{a} = ((X^T X)^{-1} X^T) Y \quad (5.23)$$

Using Eq. (5.23), and substituting the values (Similar to Problem 5.2), one gets $\hat{a}$ as:

$$\begin{aligned} &= \left(\begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \\ 4 & 5 & 8 & 2 \end{pmatrix} \begin{pmatrix} 1 & 1 & 4 \\ 1 & 2 & 5 \\ 1 & 3 & 8 \\ 1 & 4 & 2 \end{pmatrix} \right)^{-1} \times \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \\ 4 & 5 & 8 & 2 \end{pmatrix} \begin{pmatrix} 1 \\ 6 \\ 8 \\ 12 \end{pmatrix} \\ &= \begin{pmatrix} -1.69 \\ 3.48 \\ -0.05 \end{pmatrix} \end{aligned}$$

Here, the coefficients are $a_0 = -1.69$, $a_1 = 3.48$ and $a_2 = -0.05$. Hence, the constructed model is:
 $y = -1.69 + 3.48x_1 - 0.05x_2$

Polynomial Regression

It can handle non-linear relationships among variables by using n^{th} degree of a polynomial. Instead of applying transforms, polynomial regression can be directly used to deal with different levels of curvilinearity.

Polynomial regression provides a non-linear curve such as quadratic and cubic. For example, the second-degree transformation (called quadratic transformation) is given as: $y = a_0 + a_1x + a_2x^2$ and the third-degree polynomial is called cubic transformation given as: $y = a_0 + a_1x + a_2x^2 + a_3x^3$. Generally, polynomials of maximum degree 4 are used, as higher order polynomials take some strange shapes and make the curve more flexible. It leads to a situation of overfitting and hence is avoided.

Let us consider a polynomial of 2nd degree. Given points $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$, the objective is to fit a polynomial of degree 2. The polynomial of degree 2 is given as:

$$y = a_0 + a_1x + a_2x^2 \quad (5.26)$$

Such that the error $E = \sum_{i=1}^n [y_i - (a_0 + a_1x_i + a_2x_i^2)]^2$ is minimized. The coefficients a_0, a_1, a_2 of Eq. (5.26) can be obtained by taking partial derivatives with respect to each of the coefficients as $\frac{\partial E}{\partial a_0}, \frac{\partial E}{\partial a_1}, \frac{\partial E}{\partial a_2}$ and substituting it with zero. This results in 2 + 1 equations given as follows:

$$\begin{aligned} na_0 + \left(\sum_{i=1}^n x_i \right) a_1 + \left(\sum_{i=1}^n x_i^2 \right) a_2 &= \sum_{i=1}^n y_i \\ \left(\sum_{i=1}^n x_i \right) a_0 + \left(\sum_{i=1}^n x_i^2 \right) a_1 + \left(\sum_{i=1}^n x_i^3 \right) a_2 &= \sum_{i=1}^n x_i y_i \\ \left(\sum_{i=1}^n x_i^2 \right) a_0 + \left(\sum_{i=1}^n x_i^3 \right) a_1 + \left(\sum_{i=1}^n x_i^4 \right) a_2 &= \sum_{i=1}^n x_i^2 y_i \end{aligned} \quad (5.27)$$

The best line is the line that minimizes the error between line and data points. Arranging the coefficients of the above equation in the matrix form results in:

$$\begin{bmatrix} n & \sum x_i & \sum x_i^2 \\ \sum x_i & \sum x_i^2 & \sum x_i^3 \\ \sum x_i^2 & \sum x_i^3 & \sum x_i^4 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} \sum y_i \\ \sum (x_i, y_i) \\ \sum (x_i^2, y_i) \end{bmatrix} \quad (5.28)$$

This is of the form $Xa = B$. One can solve this equation for a as:

$$a = X^{-1}B \quad (5.29)$$

Table 5.8: Sample Data

x	y
1	1
2	4
3	9
4	15

Solution: For applying polynomial regression, computation is done as shown in Table 5.9.

Here, the order is 2 and the sample i ranges from 1 to 4.

Table 5.9: Computation Table

x_i	y_i	$x_i y_i$	x_i^2	$x_i^2 y_i$	x_i^3	x_i^4
1	1	1	1	1	1	1
2	4	8	4	16	8	16
3	9	27	9	81	27	81
4	15	60	16	240	64	256
$\Sigma x_i = 10$	$\Sigma y_i = 29$	$\Sigma x_i y_i = 96$	$\Sigma x_i^2 = 30$	$\Sigma x_i^2 y_i = 338$	$\Sigma x_i^3 = 100$	$\Sigma x_i^4 = 354$

It can be noted that, $N = 4$, $\Sigma y_i = 29$, $\Sigma x_i y_i = 96$, $\Sigma x_i^2 y_i = 338$. When the order is 2, the matrix using Eq. (5.28) is given as follows:

$$\begin{bmatrix} 4 & 10 & 30 \\ 10 & 30 & 100 \\ 30 & 100 & 354 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} 29 \\ 96 \\ 338 \end{bmatrix}$$

Therefore, using Eq. (5.29), one can get coefficients as:

$$\begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} 4 & 10 & 30 \\ 10 & 30 & 100 \\ 30 & 100 & 354 \end{bmatrix}^{-1} \times \begin{bmatrix} 29 \\ 96 \\ 338 \end{bmatrix} = \begin{bmatrix} -0.75 \\ 0.95 \\ 0.75 \end{bmatrix}.$$

This leads to the regression equation using Eq. (5.26) as:

$$y = -0.75 + 0.95 x + 0.75 x^2$$

5.7 LOGISTIC REGRESSION

Linear regression predicts the numerical response but is not suitable for predicting the categorical variables. When categorical variables are involved, it is called classification problem. Logistic regression is suitable for binary classification problem. Here, the output is often a categorical variable. For example, the following scenarios are instances of predicting categorical variables.

1. Is the mail spam or not spam? The answer is yes or no. Thus, categorical dependant variable is a binary response of yes or no.
2. If the student should be admitted or not is based on entrance examination marks. Here, categorical variable response is admitted or not.
3. The student being pass or fail is based on marks secured.

Thus, logistic regression is used as a binary classifier and works by predicting the probability of the categorical variable. In general, it takes one or more features x and predicts the response y . If the probability is predicted via linear regression, it is given as:

$$p(x) = a_0 + a_1x$$

Hence, logistic regression tries to model the probability of the particular response variable. In email classification problem, say normal email or spam, if the probability of the response variable is 0.7, then there is a 70% possibility of a normal mail.

Linear regression generated value is in the range $-\infty$ to $+\infty$, whereas the probability of the response variable ranges between 0 and 1. Hence, there must be a mapping function to map the value $-\infty$ to $+\infty$ to 0–1. The core of the mapping function in logistic regression method is sigmoidal function. A sigmoidal function is a 'S' shaped function that yields values between 0 and 1. This is known as logit function. This is mathematically represented as:

$$\text{logit}(x) = \frac{1}{1 + e^{-x}} \quad (5.30)$$

Here, x is the independent variable and e is the Euler number. The purpose of the logit function is to map any real number to zero or 1.

Logistic regression can be viewed as an extension of linear regression, but the only difference is that the output of linear regression can be an extremely high number. This needs to be mapped into the range 0–1, as probability can have values only in the range 0–1. This problem is solved using log odd or logit functions. What is the difference between odds and probability? Odds and probability (or likelihood) are two sides of a coin and represent uncertainty. The odds are defined as the ratio of the probability of an event and probability of an event that is not happening. This is given as:

$$\text{odd} = \frac{\text{probability of an event}}{\text{probability of a non-event}} = \frac{p}{1-p} \quad (5.31)$$

Log-odds can be taken for the odds, resulting in:

$$\log\left(\frac{p(x)}{1-p(x)}\right) = a_0 + a_1x \quad (5.32)$$

Here, $\log(.)$ is a logit function or log odds function. One can solve for $p(x)$ by taking the inverse of the above function as:

$$p(x) = \frac{\exp(a_0 + a_1x)}{1 + \exp(a_0 + a_1x)} \quad (5.33)$$

This is the same sigmoidal function. It always gives the value in the range 0–1. Dividing the numerator and denominator by the numerator, one gets:

$$p(x) = \frac{1}{1 + \exp(-a_0 - a_1x)} \quad (5.34)$$

Example 5.7: Let us assume a binomial logistic regression problem where the classes are pass and fail. The student dataset has entrance mark based on the historic data of those who are selected or not selected. Based on the logistic regression, the values of the learnt parameters are $a_0 = 1$ and $a_1 = 8$. Assuming marks of $x = 60$, compute the resultant class.

Solution: The values of regression coefficients are $a_0 = 1$ and $a_1 = 8$, and given that $x = 60$.

Based on the regression coefficients, z can be computed as:

$$\begin{aligned} z &= a_0 + a_1 x \\ &= 1 + 8 \times 60 = 481 \end{aligned}$$

One can fit this in a sigmoidal function using Eq. (5.30) to get the probability as:

$$y = \frac{1}{1 + \exp(-481)} = \frac{1}{2.271} = 0.44$$

If we assume the threshold value as 0.5, then it is observed that $0.44 < 0.5$, therefore, the candidate with marks 60 is not selected.

To determine the relationship between dependant and independent variables, parameters need to be obtained. In logistic regression, the parameters are obtained through maximum likelihood function (MLE) using the training data. The aim is to learn the values of parameters of the logistic model (a 's) by minimizing the error in the probability predicted by the model.

There can be many different sets of coefficients available. The optimal value of the parameters is obtained by using the MLE function, which is a set of coefficients for which the probability of getting the observed data is maximum.

If π is the success of the outcome and $1 - \pi$ is the failure of the outcome, then the likelihood function is given as:

$$L(a : y) = \prod_{i=1}^n \left(\frac{\pi_i}{1 - \pi_i} \right)^{y_i} (1 - \pi_i) \quad (5.37)$$

To determine the value of the parameters, the log of the likelihood function is taken. Techniques like Newton method can be used to maximize the log-likelihood of the function.

Logistic regression is suitable for binary classification. The idea can be extended for multiple classes called multinomial logistic regression. Let us assume that there are three classes 1, 2 and 3. Then, the multinomial logistic regression creates three classification problems – class 1 and Not class 1, class 2 and Not class 2, and finally class 3 and Not class 3. Three problems are simultaneously used to find the maximum probability relative to others to get the appropriate class.

Logistic regression is a simple and efficient method for binary classification. The model can be easily interpreted too. The disadvantages of logistic regression are that multinomial logistic regression cannot handle many attributes and can handle only linear features. Also, if all the attributes have multicollinearity problem, the logistic method does not work effectively.

CHAPTER 6

DECISION TREE LEARNING

6.1 Introduction

Decision tree learning model, one of the most popular supervised predictive learning models, classifies data instances with high accuracy and consistency. The model performs an *inductive inference* that reaches a general conclusion from observed examples. This model is variably used for solving complex classification applications.

Decision tree is a concept tree which summarizes the information contained in the training dataset in the form of a tree structure. Once the concept model is built, test data can be easily classified.

- Why called as decision tree ?
 - As starts from root node and finds number of solutions .
- The benefits of having a decision tree are as follows :
 - It does not require any domain knowledge.
 - It is easy to comprehend.
 - The learning and classification steps of a decision tree are simple and fast.
- Example : Toll free number

6.1.1 Structure of a Decision Tree A decision tree is a structure that includes a root node, branches, and leaf nodes. Each internal node denotes a test on an attribute, each branch denotes the outcome of a test, and each leaf node holds a class label. The topmost node in the tree is the root node.

Applies to classification and regression model.

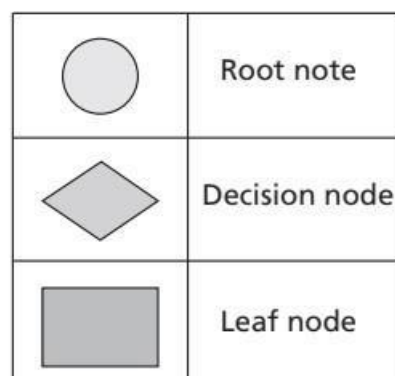


Figure 6.1: Nodes in a Decision Tree

Note: *A decision tree is not always a binary tree. It is a tree which can have more than two branches.*

The decision tree consists of 2 major procedures:

- 1) Building a tree and
- 2) Knowledge inference or classification.

Building the Tree

Goal Construct a decision tree with the given training dataset. The tree is constructed in a top-down fashion. It starts from the root node. At every level of tree construction, we need to find the best split attribute or best decision node among all attributes. This process is recursive and continued until we end up in the last level of the tree or finding a leaf node which cannot be split further. The tree construction is complete when all the test conditions lead to a leaf node. The leaf node contains the target class or output of classification.

Output Decision tree representing the complete hypothesis space.

Knowledge Inference or Classification

Goal Given a test instance, infer to the target class it belongs to.

Classification Inferring the target class for the test instance or object is based on inductive inference on the constructed decision tree. In order to classify an object, we need to start traversing the tree from the root. We traverse as we evaluate the test condition on every decision node with the test object attribute value and walk to the branch corresponding to the test's outcome. This process is repeated until we end up in a leaf node which contains the target class of the test object.

Output Target label of the test instance.

Advantages of Decision Trees

1. Easy to model and interpret
2. Simple to understand
3. The input and output attributes can be discrete or continuous predictor variables.
4. Can model a high degree of nonlinearity in the relationship between the target variables and the predictor variables
5. Quick to train

Disadvantages of Decision Trees

Some of the issues that generally arise with a decision tree learning are that:

1. It is difficult to determine how deeply a decision tree can be grown or when to stop growing it.
2. If training data has errors or missing attribute values, then the decision tree constructed may become unstable or biased.
3. If the training data has continuous valued attributes, handling it is computationally complex and has to be discretized.
4. A complex decision tree may also be over-fitting with the training data.
5. Decision tree learning is not well suited for classifying multiple output classes.
6. Learning an optimal decision tree is also known to be NP-complete.

6.1.2 Fundamentals of Entropy

- How to draw a decision tree ?

Entropy

Information gain

Entropy is the amount of uncertainty or randomness in the outcome of a random variable or an event. Moreover, entropy describes about the homogeneity of the data instances. The best feature is selected based on the entropy value. For example, when a coin is flipped, head or tail are the two outcomes, hence its entropy is lower when compared to rolling a dice which has got six outcomes.

Higher the entropy → Higher the uncertainty

Lower the entropy → Lower the uncertainty

Let P be the probability distribution of data instances from 1 to n as shown in Eq. (6.2).

$$\text{So, } P = P_1, \dots, P_n \quad (6.2)$$

Entropy of P is the information measure of this probability distribution given in Eq. (6.3),

$$\begin{aligned} \text{Entropy_Info}(P) &= \text{Entropy_Info}(P_1, \dots, P_n) \\ &= -(P_1 \log_2(P_1) + P_2 \log_2(P_2) + \dots + P_n \log_2(P_n)) \end{aligned} \quad (6.3)$$

where, P_1 is the probability of data instances classified as class 1 and P_2 is the probability of data instances classified as class 2 and so on.

$$P_1 = \frac{|\text{No of data instances belonging to class 1}|}{|\text{Total no of data instances in the training dataset}|}$$

Algorithm 6.1: General Algorithm for Decision Trees

Algorithm 6.1: General Algorithm for Decision Trees

1. Find the best attribute from the training dataset using an attribute selection measure and place it at the root of the tree.
2. Split the training dataset into subsets based on the outcomes of the test attribute and each subset in a branch contains the data instances or tuples with the same value for the selected test attribute.
3. Repeat step 1 and step 2 on each subset until we end up in leaf nodes in all the branches of the tree.
4. This splitting process is recursive until the stopping criterion is reached.

Stopping Criteria

The following are some of the common stopping conditions:

1. The data instances are homogenous which means all belong to the same class C_i and hence its entropy is 0.
2. A node with some defined minimum number of data instances becomes a leaf (Number of data instances in a node is between 0.25 and 1.00% of the full training dataset).
3. The maximum tree depth is reached, so further splitting is not done and the node becomes a leaf node.

6.2 DECISION TREE INDUCTION ALGORITHMS

There are many decision tree algorithms, such as ID3, C4.5, CART, CHAID, QUEST, GUIDE, CRUISE, and CTREE, that are used for classification in real-time environment. The most commonly used decision tree algorithms are ID3 (Iterative Dichotomizer 3), developed by J.R Quinlan in 1986, and C4.5 is an advancement of ID3 presented by the same author in 1993. CART, that stands for Classification and Regression Trees, is another algorithm which was developed by Breiman et al. in 1984.

The accuracy of the tree constructed depends upon the selection of the best split attribute. Different algorithms are used for building decision trees which use different measures to decide on the splitting criterion. Algorithms such as ID3, C4.5 and CART are popular algorithms used in the construction of decision trees. The algorithm ID3 uses 'Information Gain' as the splitting criterion whereas the algorithm C4.5 uses 'Gain Ratio' as the splitting criterion. The CART algorithm is popularly used for classifying both categorical and continuous-valued target variables. CART uses GINI Index to construct a decision tree.

6.2.1 ID3 Tree Construction(ID3 stands for Iterative Dichotomiser 3)

A decision tree is one of the most powerful tools of **supervised learning** algorithms used for both **classification** and **regression** tasks.

It builds a flowchart-like tree structure where each internal node denotes a test on an attribute, each branch represents an outcome of the test, and each leaf node (terminal

node) holds a class label. It is **constructed** by recursively splitting the training data into subsets based on the values of the attributes until a stopping criterion is met, such as the maximum depth of the tree or the minimum number of samples required to split a node.

Definitions

Let T be the training dataset.

Let A be the set of attributes $A = \{A_1, A_2, A_3, \dots, A_n\}$.

Let m be the number of classes in the training dataset.

Let P_i be the probability that a data instance or a tuple ' d ' belongs to class C_i .

It is calculated as,

$$P_i = \frac{\text{Total no of data instances that belongs to class } C_i \text{ in } T}{\text{Total no of tuples in the training set } T} \quad (6.6)$$

Mathematically, it is represented as shown in Eq. (6.7).

$$P_i = \frac{|d_{C_i}|}{|T|} \quad (6.7)$$

$$\text{Entropy_Info}(T) = - \sum_{i=1}^m P_i \log_2 P_i \quad (6.8)$$

$$\text{Entropy_Info}(T, A) = \sum_{i=1}^r \frac{|A_i|}{|T|} \times \text{Entropy_Info}(A_i) \quad (6.9)$$

$$\text{Information_Gain}(A) = \text{Entropy_Info}(T) - \text{Entropy_Info}(T, A) \quad (6.10)$$

Algorithm 6.2: Procedure to Construct a Decision Tree using ID3

1. Compute Entropy_Info Eq. (6.8) for the whole training dataset based on the target attribute.
2. Compute Entropy_Info Eq. (6.9) and Information_Gain Eq. (6.10) for each of the attribute in the training dataset.
3. Choose the attribute for which entropy is minimum and therefore the gain is maximum as the best split attribute.
4. The best split attribute is placed as the root node.
5. The root node is branched into subtrees with each subtree as an outcome of the test condition of the root node attribute. Accordingly, the training dataset is also split into subsets.
6. Recursively apply the same operation for the subset of the training set with the remaining attributes until a leaf node is derived or no more training instances are available in the subset.

6.2.2 C4.5 Construction

C4.5 is a widely used algorithm for constructing decision trees from a dataset.

Disadvantages of ID3 are: Attributes must be nominal values, dataset must not include missing data, and finally the algorithm tend to fall into overfitting.

To overcome this disadvantage Ross Quinlan, inventor of ID3, made some improvements for these bottlenecks and created a new algorithm named C4.5. Now, the algorithm can create a more generalized models including continuous data and could handle missing data. And also works with discrete data, supports post-pruning.

Given a Training dataset T ,

The Split_Info of an attribute A is computed as given in Eq. (6.11):

$$\text{Split_Info}(T, A) = - \sum_{i=1}^v \frac{|A_i|}{|T|} \times \log_2 \frac{|A_i|}{|T|}$$

where, the attribute A has got ' v ' distinct values $\{a_1, a_2, \dots, a_v\}$, and $|A_i|$ is the number of instances for distinct value ' i ' in attribute A .

The Gain_Ratio of an attribute A is computed as given in Eq. (6.12):

$$\text{Gain_Ratio}(A) = \frac{\text{Info_Gain}(A)}{\text{Split_Info}(T, A)}$$

Algorithm 6.3: Procedure to Construct a Decision Tree using C4.5

1. Compute Entropy_Info Eq. (6.8) for the whole training dataset based on the target attribute.
2. Compute Entropy_Info Eq. (6.9), Info_Gain Eq. (6.10), Split_Info Eq. (6.11) and Gain_Ratio Eq. (6.12) for each of the attribute in the training dataset.
3. Choose the attribute for which Gain_Ratio is maximum as the best split attribute.
4. The best split attribute is placed as the root node.
5. The root node is branched into subtrees with each subtree as an outcome of the test condition of the root node attribute. Accordingly, the training dataset is also split into subsets.
6. Recursively apply the same operation for the subset of the training set with the remaining attributes until a leaf node is derived or no more training instances are available in the subset.

Dealing with Continuous Attributes in C4.5

The C4.5 algorithm is further improved by considering attributes which are continuous, and a continuous attribute is discretized by finding a split point or threshold. When an attribute 'A' has numerical values which are continuous, a threshold or best split point 's' is found such that the set of values is categorized into two sets such as $A < s$ and $A \geq s$. The best split point is the attribute value which has maximum information gain for that attribute.

Now, let us consider the set of continuous values for the attribute CGPA in the sample dataset as shown in Table 6.12.

Table 6.12: Sample Dataset

S.No.	CGPA	Job Offer
1.	9.5	Yes
2.	8.2	Yes
3.	9.1	No
4.	6.8	No
5.	8.5	Yes
6.	9.5	Yes
7.	7.9	No
8.	9.1	Yes
9.	8.8	Yes
10.	8.8	Yes

First, sort the values in an ascending order.

6.8	7.9	8.2	8.5	8.8	8.8	9.1	9.1	9.5	9.5
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

Remove the duplicates and consider only the unique values of the attribute.

6.8	7.9	8.2	8.5	8.8	9.1	9.5
-----	-----	-----	-----	-----	-----	-----

Now, compute the Gain for the distinct values of this continuous attribute. Table 6.13 shows the computed values.

Table 6.13: Gain Values for CGPA

	6.8		7.9		8.2		8.5		8.8		9.1		9.5	
Range	≤	>	≤	>	≤	>	≤	>	≤	>	≤	>	≤	>
Yes	0	7	0	7	1	6	2	5	4	3	5	2	7	0
No	1	2	2	1	2	1	2	1	2	1	3	0	3	0
Entropy	0	0.7637	0	0.5433	0.9177	0.5913	1	0.6497	0.9177	0.8108	0.9538	0	0.8808	0
Entropy_Info (S, T)	0.6873		0.4346		0.6892		0.7898		0.8749		0.7630		0.8808	
Gain	0.1935		0.4462		0.1916		0.091		0.0059		0.1178		0	

Table 6.14: Discretized Instances

S.No.	CGPA Continuous	CGPA Discretized	Job Offer
1.	9.5	>7.9	Yes
2.	8.2	>7.9	Yes
3.	9.1	>7.9	No
4.	6.8	≤7.9	No
5.	8.5	>7.9	Yes
6.	9.5	>7.9	Yes
7.	7.9	≤7.9	No
8.	9.1	>7.9	Yes
9.	8.8	>7.9	Yes
10.	8.8	>7.9	Yes

6.2.3 Classification and Regression Trees Construction

Classification and Regression Trees (CART) is a widely used algorithm for constructing decision trees that can be applied to both classification and regression tasks. CART is similar to C4.5 but has some differences in its construction and splitting criteria.

The classification method CART is required to construct a decision tree based on Gini's impurity index. It serves as an example of how the values of other variables can be used to predict the values of a target variable. It functions as a fundamental machine-learning method and provides a wide range of use cases

subset. For example, if an attribute A has three distinct values say $\{a_1, a_2, a_3\}$, the possible subsets are $\{\}, \{a_1\}, \{a_2\}, \{a_3\}, \{a_1, a_2\}, \{a_1, a_3\}, \{a_2, a_3\}$, and $\{a_1, a_2, a_3\}$. So, if an attribute has 3 distinct values, the number of possible subsets is 2^3 , which means 8. Excluding the empty set $\{\}$ and the full set $\{a_1, a_2, a_3\}$, we have 6 subsets. With 6 subsets, we can form three possible combinations such as:

$\{a_1\}$ with $\{a_2, a_3\}$

$\{a_2\}$ with $\{a_1, a_3\}$

$\{a_3\}$ with $\{a_1, a_2\}$

Hence, in this CART algorithm, we need to compute the best splitting attribute and the best split subset i in the chosen attribute.

Higher the GINI value, higher is the homogeneity of the data instances.

Gini_Index(T) is computed as given in Eq. (6.13).

$$\text{Gini_Index}(T) = 1 - \sum_{i=1}^n P_i^2 \quad (6.13)$$

where,

P_i be the probability that a data instance or a tuple ' d ' belongs to class C_i . It is computed as:

$P_i = \text{No. of data instances belonging to class } i / \text{Total no of data instances in the training dataset } T$

GINI Index assumes a binary split on each attribute, therefore, every attribute is considered as a binary attribute which splits the data instances into two subsets S_1 and S_2 .

Gini_Index(T, A) is computed as given in Eq. (6.14).

$$\text{Gini_Index}(T, A) = \frac{|S_1|}{|T|} \text{Gini}(S_1) + \frac{|S_2|}{|T|} \text{Gini}(S_2) \quad (6.14)$$

The splitting subset with minimum Gini_Index is chosen as the best splitting subset for an attribute. The best splitting attribute is chosen by the minimum Gini_Index which is otherwise maximum ΔGini because it reduces the impurity.

ΔGini is computed as given in Eq. (6.15):

$$\Delta\text{Gini}(A) = \text{Gini}(T) - \text{Gini}(T, A) \quad (6.15)$$

Algorithm 6.4: Procedure to Construct a Decision Tree using CART

1. Compute Gini_Index Eq. (6.13) for the whole training dataset based on the target attribute.
2. Compute Gini_Index for each of the attribute Eq. (6.14) and for the subsets of each attribute in the training dataset.
3. Choose the best splitting subset which has minimum Gini_Index for an attribute.
4. Compute ΔGini Eq. (6.15) for the best splitting subset of that attribute.
5. Choose the best splitting attribute that has maximum ΔGini .
6. The best split attribute with the best split subset is placed as the root node.
7. The root node is branched into two subtrees with each subtree an outcome of the test condition of the root node attribute. Accordingly, the training dataset is also split into two subsets.
8. Recursively apply the same operation for the subset of the training set with the remaining attributes until a leaf node is derived or no more training instances are available in the subset.

6.2.4 Regression Trees

Regression trees are a variant of decision trees where the target feature is a continuous valued variable. These trees can be constructed using an algorithm called reduction in variance which uses standard deviation to choose the best splitting attribute.

Algorithm 6.5: Procedure for Constructing Regression Trees

1. Compute standard deviation for each attribute with respect to target attribute.
2. Compute standard deviation for the number of data instances of each distinct value of an attribute.
3. Compute weighted standard deviation for each attribute.

4. Compute standard deviation reduction by subtracting weighted standard deviation for each attribute from standard deviation of each attribute.
5. Choose the attribute with a higher standard deviation reduction as the best split attribute.
6. The best split attribute is placed as the root node.
7. The root node is branched into subtrees with each subtree as an outcome of the test condition of the root node attribute. Accordingly, the training dataset is also split into different subsets.
8. Recursively apply the same operation for the subset of the training set with the remaining attributes until a leaf node is derived or no more training instances are available in the subset.

MODULE-4

Bayes' Theorem is a fundamental concept in probability theory and forms the foundation of **Bayesian learning** in machine learning. It allows you to update the probability of a hypothesis (or event) based on new evidence.

Bayes' Theorem Explained

At its core, Bayes' Theorem relates current knowledge or belief about an event (the **prior** probability) to new data or evidence (the **likelihood**) to produce an updated belief (the **posterior** probability).

Mathematically, Bayes' Theorem is:

Mathematically, Bayes' Theorem is:

$$P(H|D) = \frac{P(D|H)P(H)}{P(D)}$$

Where:

- **$P(H | D)$** is the **posterior probability**: the probability of the hypothesis H being true given the data D .
- **$P(D | H)$** is the **likelihood**: the probability of observing the data D given that hypothesis H is true.
- **$P(H)$** is the **prior probability**: the initial belief about the hypothesis H before any data is observed.
- **$P(D)$** is the **marginal likelihood** or **evidence**: the total probability of the data under all possible hypotheses. This acts as a normalizing constant to ensure that the posterior is a valid probability distribution.

Breaking Down the Components of Bayes' Theorem

1. **Prior Probability ($P(H)$):**
 - This represents what we know or believe about a hypothesis before seeing any new data.
 - Example: In a medical test scenario, it could be the prior probability of a person having a disease before considering the test results (e.g., based on the general population statistics).
2. **Likelihood ($P(D | H)$):**
 - This is the probability of observing the data, assuming the hypothesis is true. It expresses how likely it is to see the given data under the assumption of the hypothesis.
 - Example: The likelihood would be the probability of getting a positive test result assuming the person has the disease.
3. **Evidence ($P(D)$):**
 - This is the total probability of the data across all hypotheses. It serves to normalize the posterior probability so that it sums to 1.
 - Example: The probability of getting a positive test result across all people, whether they have the disease or not.
4. **Posterior Probability ($P(H | D)$):**
 - This is the updated belief about the hypothesis after considering the new data (the evidence).
 - Example: The posterior would give the probability of a person having the disease after considering both the prior knowledge and the test results.

Intuition Behind Bayes' Theorem

Bayes' Theorem can be understood in terms of **updating beliefs**. When you receive new evidence, you modify your prior belief to form a new belief that incorporates both your prior knowledge and the new data.

- **Before** you collect any data, you have a prior belief about a hypothesis (e.g., the probability of a patient having a disease).
- **After** seeing new data (e.g., the result of a medical test), you update your belief about the hypothesis to reflect this new evidence.

Bayes' Theorem lets you do this systematically, ensuring that your updated belief (posterior) is proportional to the prior belief and the likelihood of observing the new data.

Example: Disease Diagnosis

Consider a simple example of diagnosing a disease using a medical test.

1. **Prior Probability ($P(H)$):**
 - The prior belief is the probability that a person has the disease. For example, in a population, 1% of people might have the disease, so $P(H)=0.01$ $P(H) = 0.01$.
2. **Likelihood ($P(D | H)$):**
 - This is the probability of getting a positive test result if the person has the disease. Suppose the test correctly identifies the disease 95% of the time, so $P(D|H)=0.95$ $P(D | H) = 0.95$.
3. **Evidence ($P(D)$):**
 - This is the total probability of a positive test result in the population. It includes both people who have the disease and those who do not.
4. **Posterior Probability ($P(H | D)$):**
 - After receiving a positive test result, we want to calculate the probability that the person actually has the disease.

Let's compute the **posterior probability** using Bayes' Theorem:

$$P(H|D) = \frac{P(D|H)P(H)}{P(D)}$$

Where:

- $P(D|H) = 0.95$ (the probability of a positive test result given the person has the disease)
- $P(H) = 0.01$ (the prior probability that the person has the disease)
- $P(D)$ is the total probability of getting a positive test result (both true positives and false positives).

To calculate $P(D)$, we use the law of total probability:

$$P(D) = P(D|H)P(H) + P(D|\neg H)P(\neg H)$$

To calculate $P(D)$, we use the law of total probability:

$$P(D) = P(D|H)P(H) + P(D|\neg H)P(\neg H)$$

Where:

- $P(D|\neg H)$ is the probability of getting a positive test result when the person does **not** have the disease (e.g., a false positive rate). Let's assume this is 5%, so $P(D|\neg H) = 0.05$.
- $P(\neg H) = 0.99$ is the prior probability that the person does not have the disease.

Now, we can calculate $P(D)$:

$$P(D) = (0.95)(0.01) + (0.05)(0.99) = 0.0095 + 0.0495 = 0.059$$

Finally, we can compute the **posterior probability**:

$$P(H|D) = \frac{(0.95)(0.01)}{0.059} = \frac{0.0095}{0.059} \approx 0.161$$

So, after a positive test result, the probability that the person actually has the disease is approximately **16.1%**, even though the test is 95% accurate. This is due to the low prior probability of the disease in the population.

Algorithm 8.1: Naïve Bayes

1. Compute the prior probability for the target class.
2. Compute Frequency matrix and likelihood Probability for each of the feature.
3. Use Bayes theorem Eq. (8.1) to calculate the probability of all hypotheses.
4. Use Maximum A Posteriori (MAP) Hypothesis, h_{MAP} Eq. (8.2) to classify the test object to the hypothesis with the highest probability.

Example 8.2: Assess a student's performance using Naïve Bayes algorithm with the dataset provided in Table 8.1. Predict whether a student gets a job offer or not in his final year of the course.

Table 8.1: Training Dataset

S.No.	CGPA	Interactiveness	Practical Knowledge	Communication Skills	Job Offer
1.	≥ 9	Yes	Very good	Good	Yes
2.	≥ 8	No	Good	Moderate	Yes
3.	≥ 9	No	Average	Poor	No
4.	< 8	No	Average	Good	No
5.	≥ 8	Yes	Good	Moderate	Yes
6.	≥ 9	Yes	Good	Moderate	Yes
7.	< 8	Yes	Good	Poor	No
8.	≥ 9	No	Very good	Good	Yes
9.	≥ 8	Yes	Good	Good	Yes
10.	≥ 8	Yes	Average	Good	Yes

Solution: The training dataset T consists of 10 data instances with attributes such as 'CGPA', 'Interactiveness', 'Practical Knowledge' and 'Communication Skills' as shown in Table 8.1. The target variable is Job Offer which is classified as Yes or No for a candidate student.

Step 1: Compute the prior probability for the target feature 'Job Offer'. The target feature 'Job Offer' has two classes, 'Yes' and 'No'. It is a binary classification problem. Given a student instance, we need to classify whether 'Job Offer = Yes' or 'Job Offer = No'.

From the training dataset, we observe that the frequency or the number of instances with 'Job Offer = Yes' is 7 and 'Job Offer = No' is 3.

The prior probability for the target feature is calculated by dividing the number of instances belonging to a particular target class by the total number of instances.

Hence, the prior probability for 'Job Offer = Yes' is $7/10$ and 'Job Offer = No' is $3/10$ as shown in Table 8.2.

Table 8.2: Frequency Matrix and Prior Probability of Job Offer

Job Offer Classes	No. of Instances	Probability Value
Yes	7	$P(\text{Job Offer} = \text{Yes}) = 7/10$
No	3	$P(\text{Job Offer} = \text{No}) = 3/10$

Step 2: Compute Frequency matrix and Likelihood Probability for each of the feature.

Step 2(a): Feature – CGPA

Table 8.3 shows the frequency matrix for the feature CGPA.

Table 8.3: Frequency Matrix of CGPA

CGPA	Job Offer = Yes	Job Offer = No
≥ 9	3	1
≥ 8	4	0
< 8	0	2
Total	7	3

Table 8.4 shows how the likelihood probability is calculated for CGPA using conditional probability.

Table 8.4: Likelihood Probability of CGPA

CGPA	$P(\text{Job Offer} = \text{Yes})$	$P(\text{Job Offer} = \text{No})$
≥ 9	$P(\text{CGPA} \geq 9 \mid \text{Job Offer} = \text{Yes}) = 3/7$	$P(\text{CGPA} \geq 9 \mid \text{Job Offer} = \text{No}) = 1/3$
≥ 8	$P(\text{CGPA} \geq 8 \mid \text{Job Offer} = \text{Yes}) = 4/7$	$P(\text{CGPA} \geq 8 \mid \text{Job Offer} = \text{No}) = 0/3$
< 8	$P(\text{CGPA} < 8 \mid \text{Job Offer} = \text{Yes}) = 0/7$	$P(\text{CGPA} < 8 \mid \text{Job Offer} = \text{No}) = 2/3$

As explained earlier the Likelihood probability is stated as the sampling density for the evidence given the hypothesis. It is denoted as $P(\text{Evidence} \mid \text{Hypothesis})$, which says how likely is the occurrence of the evidence given the parameters.

It is calculated as the number of instances of each attribute value and for a given class value divided by the number of instances with that class value.

For example $P(\text{CGPA} \geq 9 \mid \text{Job Offer} = \text{Yes})$ denotes the number of instances with 'CGPA ≥ 9 ' and 'Job Offer = Yes' divided by the total number of instances with 'Job Offer = Yes'.

From the Table 8.3 Frequency Matrix of CGPA, number of instances with 'CGPA ≥ 9 ' and 'Job Offer = Yes' is 3. The total number of instances with 'Job Offer = Yes' is 7. Hence, $P(\text{CGPA} \geq 9 \mid \text{Job Offer} = \text{Yes}) = 3/7$.

Similarly, the Likelihood probability is calculated for all attribute values of feature CGPA.

Step 2(b): Feature – Interactiveness

Table 8.5 shows the frequency matrix for the feature Interactiveness.

Table 8.5: Frequency Matrix of Interactiveness

Interactiveness	Job Offer = Yes	Job Offer = No
YES	5	1
NO	2	2
Total	7	3

Table 8.10: Likelihood Probability of Communication Skills

Communication Skills	$P(\text{Job Offer} = \text{Yes})$	$P(\text{Job Offer} = \text{No})$
Good	$P(\text{Communication Skills} = \text{Good} \mid \text{Job Offer} = \text{Yes}) = 4/7$	$P(\text{Communication Skills} = \text{Good} \mid \text{Job Offer} = \text{No}) = 1/3$
Moderate	$P(\text{Communication Skills} = \text{Moderate} \mid \text{Job Offer} = \text{Yes}) = 3/7$	$P(\text{Communication Skills} = \text{Moderate} \mid \text{Job Offer} = \text{No}) = 0/3$
Poor	$P(\text{Communication Skills} = \text{Poor} \mid \text{Job Offer} = \text{Yes}) = 0/7$	$P(\text{Communication Skills} = \text{Poor} \mid \text{Job Offer} = \text{No}) = 2/3$

Step 3: Use Bayes theorem Eq. (8.1) to calculate the probability of all hypotheses.

Given the test data = (CGPA ≥ 9 , Interactiveness = Yes, Practical knowledge = Average, Communication Skills = Good), apply the Bayes theorem to classify whether the given student gets a Job offer or not.

$P(\text{Job Offer} = \text{Yes} \mid \text{Test data}) = (P(\text{CGPA} \geq 9 \mid \text{Job Offer} = \text{Yes}) P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{Yes}) P(\text{Practical knowledge} = \text{Average} \mid \text{Job Offer} = \text{Yes}) P(\text{Communication Skills} = \text{Good} \mid \text{Job Offer} = \text{Yes}) P(\text{Job Offer} = \text{Yes})) / (P(\text{Test Data}))$

We can ignore $P(\text{Test Data})$ in the denominator since it is common for all cases to be considered.

Hence, $P(\text{Job Offer} = \text{Yes} \mid \text{Test data}) = (P(\text{CGPA} \geq 9 \mid \text{Job Offer} = \text{Yes}) P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{Yes}) P(\text{Practical knowledge} = \text{Average} \mid \text{Job Offer} = \text{Yes}) P(\text{Communication Skills} = \text{Good} \mid \text{Job Offer} = \text{Yes}) P(\text{Job Offer} = \text{Yes}))$

$$= 3/7 \times 5/7 \times 1/7 \times 4/7 \times 7/10$$

$$= 0.0175$$

Similarly, for the other case 'Job Offer = No',

We compute the probability,

$P(\text{Job Offer} = \text{No} \mid \text{Test data}) = (P(\text{CGPA} \geq 9 \mid \text{Job Offer} = \text{No}) P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{No}) P(\text{Practical knowledge} = \text{Average} \mid \text{Job Offer} = \text{No}) P(\text{Communication Skills} = \text{Good} \mid \text{Job Offer} = \text{No}) P(\text{Job Offer} = \text{No})) / (P(\text{Test Data}))$

$P(\text{CGPA} \geq 9 \mid \text{Job Offer} = \text{No}) P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{No}) P(\text{Practical knowledge} = \text{Average} \mid \text{Job Offer} = \text{No}) P(\text{Communication Skills} = \text{Good} \mid \text{Job Offer} = \text{No}) P(\text{Job Offer} = \text{No})$

$$= 1/3 \times 1/3 \times 2/3 \times 1/3 \times 3/10$$

$$= 0.0074$$

Step 4: Use Maximum A Posteriori (MAP) Hypothesis, h_{MAP} Eq. (8.2) to classify the test object to the hypothesis with the highest probability.

Since $P(\text{Job Offer} = \text{Yes} \mid \text{Test data})$ has the highest probability value, the test data is classified as 'Job Offer = Yes'.

Zero Probability Error

In Example 8.1, consider the test data to be (CGPA ≥ 8 , Interactiveness = Yes, Practical knowledge = Average, Communication Skills = Good)

When computing the posterior probability,

$P(\text{Job Offer} = \text{Yes} \mid \text{Test data}) = (P(\text{CGPA} \geq 8 \mid \text{Job Offer} = \text{Yes}) P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{Yes}) P(\text{Practical knowledge} = \text{Average} \mid \text{Job Offer} = \text{Yes}) P(\text{Communication Skills} = \text{Good} \mid \text{Job Offer} = \text{Yes}) P(\text{Job Offer} = \text{Yes})) / (P(\text{Test Data}))$

$P(\text{Job Offer} = \text{Yes} \mid \text{Test data}) = (P(\text{CGPA} \geq 8 \mid \text{Job Offer} = \text{Yes}) P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{Yes}) P(\text{Practical knowledge} = \text{Average} \mid \text{Job Offer} = \text{Yes}) P(\text{Communication Skills} = \text{Good} \mid \text{Job Offer} = \text{Yes}) P(\text{Job Offer} = \text{Yes}))$

$$= 4/7 \times 5/7 \times 1/7 \times 4/7 \times 7/10$$

$$= 0.0233$$

Similarly, for the other case 'Job Offer = No',

When we compute the probability:

$P(\text{Job Offer} = \text{No} \mid \text{Test data}) = (P(\text{CGPA} \geq 8 \mid \text{Job Offer} = \text{No}) P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{No}) P(\text{Practical knowledge} = \text{Average} \mid \text{Job Offer} = \text{No}) P(\text{Communication Skills} = \text{Good} \mid \text{Job Offer} = \text{No}) P(\text{Job Offer} = \text{No})) / (P(\text{Test Data}))$

$= P(\text{CGPA} \geq 8 \mid \text{Job Offer} = \text{No}) P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{No}) P(\text{Practical knowledge} = \text{Average} \mid \text{Job Offer} = \text{No}) P(\text{Communication Skills} = \text{Good} \mid \text{Job Offer} = \text{No}) P(\text{Job Offer} = \text{No})$

$$= 0/3 \times 1/3 \times 2/3 \times 1/3 \times 3/10$$

$$= 0$$

Since the probability value is zero, the model fails to predict, and this is called as Zero-Probability error. This problem arises because there are no instances in the given Table 8.1 for the attribute value $\text{CGPA} \geq 8$ and $\text{Job Offer} = \text{No}$ and hence the probability value of this case is zero. This zero-probability error can be solved by applying a smoothing technique called Laplace correction which means given 1000 data instances in the training dataset, if there are zero instances for a particular value of a feature we can add 1 instance for each attribute value pair of that feature which will not make much difference for 1000 data instances and the overall probability does not become zero.

Now, let us scale the values given in Table 8.1 for 1000 data instances. The scaled values without Laplace correction are shown in Table 8.11.

Table 8.11: Scaled Values to 1000 without Laplace Correction

CGPA	$P(\text{Job Offer} = \text{Yes})$	$P(\text{Job Offer} = \text{No})$
≥ 9	$P(\text{CGPA} \geq 9 \mid \text{Job Offer} = \text{Yes}) = 300/700$	$P(\text{CGPA} \geq 9 \mid \text{Job Offer} = \text{No}) = 100/300$
≥ 8	$P(\text{CGPA} \geq 8 \mid \text{Job Offer} = \text{Yes}) = 400/700$	$P(\text{CGPA} \geq 8 \mid \text{Job Offer} = \text{No}) = 0/300$
< 8	$P(\text{CGPA} < 8 \mid \text{Job Offer} = \text{Yes}) = 0/700$	$P(\text{CGPA} < 8 \mid \text{Job Offer} = \text{No}) = 200/300$

Now, add 1 instance for each CGPA-value pair for 'Job Offer = No'. Then,

$$P(\text{CGPA} \geq 9 \mid \text{Job Offer} = \text{No}) = 101/303 = 0.333$$

$$P(\text{CGPA} \geq 8 \mid \text{Job Offer} = \text{No}) = 1/303 = 0.0033$$

$$P(\text{CGPA} < 8 \mid \text{Job Offer} = \text{No}) = 201/303 = 0.6634$$

With scaled values to 1003 data instances, we get

$P(\text{Job Offer} = \text{Yes} \mid \text{Test data}) = (P(\text{CGPA} \geq 8 \mid \text{Job Offer} = \text{Yes}) P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{Yes}) P(\text{Practical knowledge} = \text{Average} \mid \text{Job Offer} = \text{Yes}) P(\text{Communication Skills} = \text{Good} \mid \text{Job Offer} = \text{Yes}) P(\text{Job Offer} = \text{Yes}))$

$$= 400/700 \times 500/700 \times 100/700 \times 400/700 \times 700/1003$$

$$= 0.02325$$

$P(\text{Job Offer} = \text{No} \mid \text{Test data}) = P(\text{CGPA} \geq 8 \mid \text{Job Offer} = \text{No}) P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{No}) P(\text{Practical knowledge} = \text{Average} \mid \text{Job Offer} = \text{No}) P(\text{Communication Skills} = \text{Good} \mid \text{Job Offer} = \text{No}) P(\text{Job Offer} = \text{No})$

$$= 1/303 \times 100/300 \times 200/300 \times 100/300 \times 303/1003$$

$$= 0.00007385$$

Thus, using Laplace Correction, Zero Probability error can be solved with Naïve Bayes classifier.

8.3.2 Brute Force Bayes Algorithm

Applying Bayes theorem, Brute Force Bayes algorithm relies on the idea of concept learning wherein given a hypothesis space H for the training dataset T , the algorithm computes the posterior probabilities for all the hypothesis $h_i \in H$. Then, Maximum A Posteriori (MAP) Hypothesis, h_{MAP} is used to output the hypothesis with maximum posterior probability. The algorithm is quite expensive since it requires computations for all the hypotheses. Although computing posterior probabilities is inefficient, this idea is applied in various other algorithms which is also quite interesting.

8.3.3 Bayes Optimal Classifier

Bayes optimal classifier is a probabilistic model, which in fact, uses the Bayes theorem to find the most probable classification for a new instance given the training data by combining the predictions of all posterior hypotheses. This is different from Maximum A Posteriori (MAP) Hypothesis, h_{MAP} which chooses the maximum probable hypothesis or the most probable hypothesis.

Here, a new instance can be classified to a possible classification value C_i by the following Eq. (8.4).

$$= \max_{C_i} \sum_{h_i \in H} P(C_i \mid h_i) P(h_i \mid T) \quad (8.4)$$

Example 8.3: Given the hypothesis space with 4 hypothesis h_1 , h_2 , h_3 and h_4 . Determine if the patient is diagnosed as COVID positive or COVID negative using Bayes Optimal classifier.

Solution: From the training dataset T , the posterior probabilities of the four different hypotheses for a new instance are given in Table 8.12.

Table 8.12: Posterior Probability Values

$P(h_i \mid T)$	$P(\text{COVID Positive} \mid h_i)$	$P(\text{COVID Negative} \mid h_i)$
0.3	0	1
0.1	1	0
0.2	1	0
0.1	1	0

h_{MAP} chooses h_1 which has the maximum probability value 0.3 as the solution and gives the result that the patient is COVID negative. But Bayes Optimal classifier combines the predictions of h_2 , h_3 and h_4 which is 0.4 and gives the result that the patient is COVID positive.

$$\sum_{k_i \in I} P(\text{COVID Negative} \mid h_i) P(h_i \mid T) = 0.3 \times 1 = 0.3$$

$$\sum_{k_i \in I} P(\text{COVID Positive} \mid h_i) P(h_i \mid T) = 0.1 \times 1 + 0.2 \times 1 + 0.1 \times 1 = 0.4$$

Therefore, $\max_{C_i \in \{\text{COVID Positive}, \text{COVID Negative}\}} \sum_{h_i \text{ all}} P(C_i | h_i) P(h_i | T) = \text{COVID Positive}$.

Thus, this algorithm, diagnoses the new instance to be COVID positive.

8.3.4 Gibbs Algorithm

The main drawback of Bayes optimal classifier is that it computes the posterior probability for all hypotheses in the hypothesis space and then combines the predictions to classify a new instance.

Gibbs algorithm is a sampling technique which randomly selects a hypothesis from the hypothesis space according to the posterior probability distribution and classifies a new instance. It is found that the prediction error occurs twice with the Gibbs algorithm when compared to Bayes Optimal classifier.

8.4 NAÏVE BAYES ALGORITHM FOR CONTINUOUS ATTRIBUTES

There are two ways to predict with Naive Bayes algorithm for continuous attributes:

1. Discretize continuous feature to discrete feature.
2. Apply Normal or Gaussian distribution for continuous feature.

Gaussian Naive Bayes Algorithm

In Gaussian Naive Bayes, the values of continuous features are assumed to be sampled from a Gaussian distribution.

Example 8.4: Assess a student's performance using Naïve Bayes algorithm for the continuous attribute. Predict whether a student gets a job offer or not in his final year of the course. The training dataset T consists of 10 data instances with attributes such as 'CGPA' and 'Interactiveness' as shown in Table 8.13. The target variable is Job Offer which is classified as Yes or No for a candidate student.

Table 8.13: Training Dataset with Continuous Attribute

S.No.	CGPA	Interactiveness	Job Offer
1.	9.5	Yes	Yes
2.	8.2	No	Yes
3.	9.3	No	No
4.	7.6	No	No
5.	8.4	Yes	Yes
6.	9.1	Yes	Yes
7.	7.5	Yes	No
8.	9.6	No	Yes
9.	8.6	Yes	Yes
10.	8.3	Yes	Yes

Solution:

Step 1: Compute the prior probability for the target feature 'Job Offer'.

Prior probabilities of both the classes are calculated using the same formula (refer to Table 8.14).

Table 8.14: Prior Probability of Target Class

Job Offer Classes	No. of Instances	Probability Value
Yes	7	$P(\text{Job Offer} = \text{Yes}) = 7/10$
No	3	$P(\text{Job Offer} = \text{No}) = 3/10$

Step 2: Compute Frequency matrix and Likelihood Probability for each of the feature.

Likelihood probabilities for a continuous attribute is obtained from Gaussian (Normal) Distribution. In the above data set, CGPA is a continuous attribute for which we need to apply Gaussian distribution to calculate the likelihood probability.

Gaussian distribution for continuous feature is calculated using the given formula,

$$P(X_i = x_i | C_j) = g(x_i, \mu_{ij}, \sigma_{ij}) \quad (8.5)$$

where,

X_i is the i^{th} continuous attribute in the given dataset and x_i is a value of the attribute.

C_j denotes the j^{th} class of the target feature.

μ_{ij} denotes the mean of the values of that continuous attribute X_i with respect to the class j of the target feature.

σ_{ij} denotes the standard deviation of the values of that continuous attribute X_i with respect to the class j of the target feature.

Hence, the normal distribution formula is given as:

$$P(X_i = x_i | C_j) = \frac{1}{\sigma_{ij} \sqrt{2\pi}} e^{-\frac{(x_i - \mu_{ij})^2}{2\sigma_{ij}^2}} \quad (8.6)$$

Step 2(a): Consider the feature CGPA

In this example CGPA is a continuous attribute,

To calculate the likelihood probability for this continuous attribute, first compute the mean and standard deviation for CGPA with respect to the target class 'Job Offer'.

Here, $X_i = \text{CGPA}$

$C_j = \text{'Job Offer' = Yes'}$

Mean and Standard Deviation for class 'Job Offer = Yes' are given as:

$$\mu_{ij} = \mu_{\text{CGPA} - \text{YES}} = 8.814286$$

$$\sigma_{ij} = \sigma_{\text{CGPA} - \text{YES}} = 0.58146$$

Mean and Standard Deviation for class 'Job Offer = No' are given as:

$C_j = \text{'Job Offer' = No'}$

$$\mu_{ij} = \mu_{\text{CGPA} - \text{NO}} = 8.133333$$

$$\sigma_{ij} = \sigma_{\text{CGPA} - \text{NO}} = 1.011599$$

Once Mean and Standard Deviation are computed, the likelihood probability for any test value using Gaussian distribution formula can be calculated.

Step 2(b): Consider the feature Interactiveness

Interactiveness is a discrete feature whose probability is calculated as earlier.

Table 8.15 shows the frequency matrix for the feature Interactiveness.

Table 8.15: Frequency Matrix of Interactiveness

Interactiveness	Job Offer = Yes	Job Offer = No
YES	5	1
NO	2	2
Total	7	3

Table 8.16 shows how the likelihood probability is calculated for Interactiveness using conditional probability.

Table 8.16: Likelihood Probability of Interactiveness

Interactiveness	$P(\text{Job Offer} = \text{Yes})$	$P(\text{Job Offer} = \text{No})$
YES	$P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{Yes}) = 5/7$	$P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{No}) = 1/3$
NO	$P(\text{Interactiveness} = \text{No} \mid \text{Job Offer} = \text{Yes}) = 2/7$	$P(\text{Interactiveness} = \text{No} \mid \text{Job Offer} = \text{No}) = 2/3$

Step 3: Use Bayes theorem to calculate the probability of all hypotheses.

Consider the test data to be (CGPA = 8.5, Interactiveness = Yes).

For the hypothesis 'Job Offer = Yes':

$P(\text{Job Offer} = \text{Yes} \mid \text{Test data}) = (P(\text{CGPA} = 8.5 \mid \text{Job Offer} = \text{Yes}) \times P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{Yes}) \times P(\text{Job Offer} = \text{Yes}))$

To compute $P(\text{CGPA} = 8.5 \mid \text{Job Offer} = \text{Yes})$ use Gaussian distribution formula:

$$P(X_i = x_k \mid C_j) = g(x_k \mid \mu_{ij}, \sigma_{ij})$$

$$P(X_{\text{CGPA}} = 8.5 \mid C_{\text{Job Offer} = \text{Yes}}) = \frac{1}{\sigma_y \sqrt{2\pi}} e^{-\frac{(x_k - \mu_y)^2}{2\sigma_y^2}}$$

$$P(X_{\text{CGPA}} = 8.5 \mid C_{\text{Job Offer} = \text{Yes}}) = \frac{1}{\sigma_{\text{CGPA-YES}} \sqrt{2\pi}} e^{-\frac{(8.5 - \mu_{\text{CGPA-YES}})^2}{2\sigma_{\text{CGPA-YES}}^2}}$$

$$P(\text{CGPA} = 8.5 \mid \text{Job Offer} = \text{Yes}) = g(x_k = 8.5, \mu_y = 8.814, \sigma_y = 0.581)$$

$$= \frac{1}{0.581 \sqrt{2\pi}} e^{-\frac{(8.5 - 8.814)^2}{2 \times 0.581^2}} = 0.594$$

$$P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{Yes}) = 5/7$$

$$P(\text{Job Offer} = \text{Yes}) = 7/10$$

Hence:

$P(\text{Job Offer} = \text{Yes} \mid \text{Test data}) = (P(\text{CGPA} = 8.5 \mid \text{Job Offer} = \text{Yes}) \times P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{Yes}) \times P(\text{Job Offer} = \text{Yes}))$

$$= 0.594 \times 5/7 \times 7/10$$

$$= 0.297$$

Similarly, for the hypothesis 'Job Offer = No':

$$P(\text{Job Offer} = \text{No} \mid \text{Test data}) = P(\text{CGPA} = 8.5 \mid \text{Job Offer} = \text{No}) \times P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{No}) \times P(\text{Job Offer} = \text{No})$$

$$P(\text{CGPA} = 8.5 \mid \text{Job Offer} = \text{No}) = g(x_i = 8.5, \mu_{ij} = 8.133, \sigma_{ij} = 1.0116)$$

$$P(X_{\text{CGPA}} = 8.5 \mid C_{\text{Job Offer} = \text{Yes}}) = \frac{1}{\sigma_{\text{CGPA-NO}} \sqrt{2\pi}} e^{-\frac{(8.5 - \mu_{\text{CGPA-NO}})^2}{2\sigma_{\text{CGPA-NO}}^2}}$$

$$= \frac{1}{1.0116 \sqrt{2\pi}} e^{-\frac{(8.5 - 8.133)^2}{2 \times 1.0116^2}} = 0.369$$

$$P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{No}) = 1/3$$

$$= P(\text{Job Offer} = \text{No}) = 0.369$$

Hence,

$$P(\text{Job Offer} = \text{No} \mid \text{Test data}) = P(\text{CGPA} = 8.5 \mid \text{Job Offer} = \text{No}) P(\text{Interactiveness} = \text{Yes} \mid \text{Job Offer} = \text{No}) \times P(\text{Job Offer} = \text{No})$$

$$= 0.369 \times 1/3 \times 3/10$$

$$= 0.0369$$

Step 4: Use Maximum A Posteriori (MAP) Hypothesis, h_{MAP} to classify the test object to the hypothesis with the highest probability.

Since $P(\text{Job Offer} = \text{Yes} \mid \text{Test data})$ has the highest probability value of 0.297, the test data is classified as 'Job Offer = Yes'.

Chapter 10

Artificial Neural Networks

The term "Artificial neural network" refers to a biologically inspired sub-field of artificial intelligence modelled after the brain.

An Artificial neural network is usually a computational network based on biological neural networks that construct the structure of the human brain.

Similar to a human brain has neurons interconnected to each other, artificial neural networks also have neurons that are linked to each other in various layers of the networks. These neurons are known as nodes.

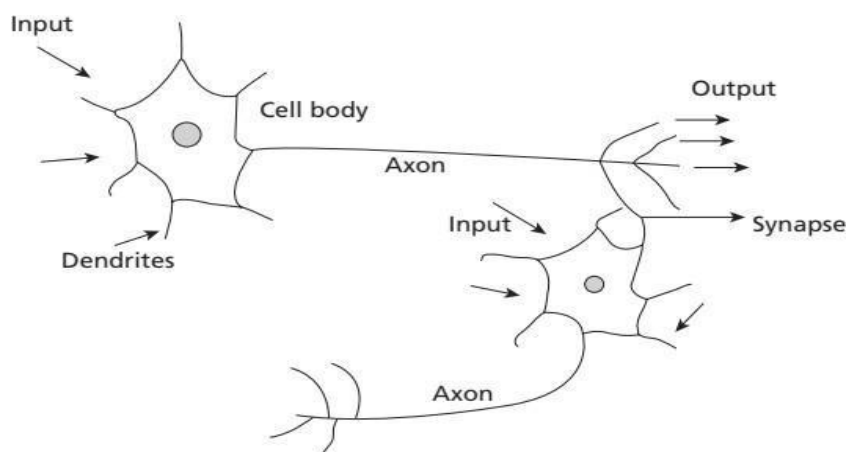
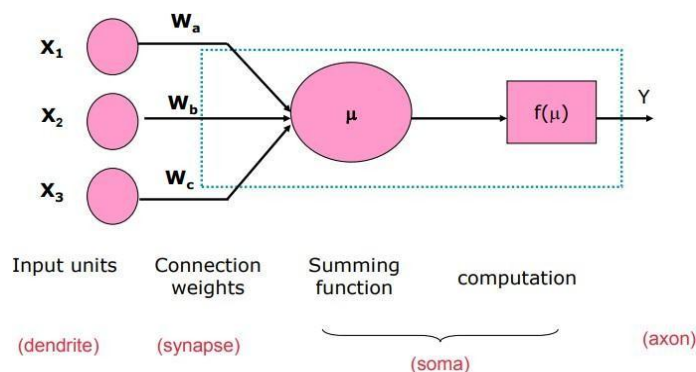


Figure 10.1: A Biological Neuron

The biological neuron consists of main **four** parts:

- **dendrites:** nerve fibres carrying electrical signals to the cell .
- **cell body:** computes a non-linear function of its inputs
- **axon:** single long fiber that carries the electrical signal from the cell body to other neurons
- **synapse:** the point of contact between the axon of one cell and the dendrite of another, regulating a chemical connection whose strength affects the input to the cell.

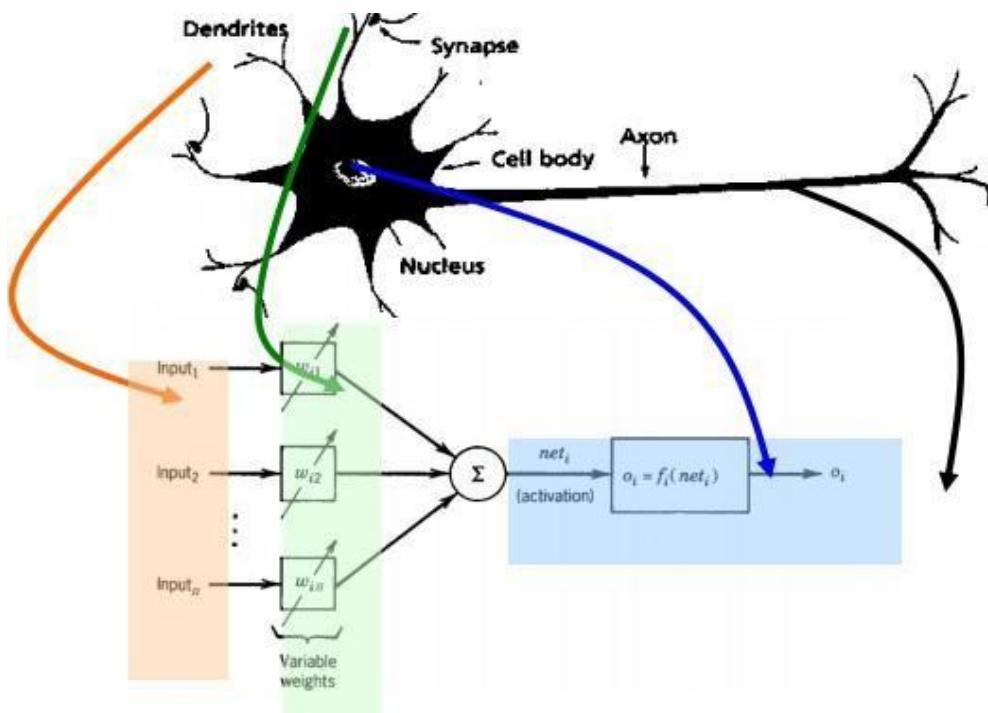


Dendrites are tree like networks made of nerve fiber connected to the cell body.

An **Axon** is a single, long connection extending from the cell body and carrying signals from the neuron. The end of axon splits into fine strands. It is found that each strand terminated into small bulb like organs called as synapse. It is through synapse that the neuron introduces its signals to other nearby neurons. The receiving ends of these synapses on the nearby neurons can be found both on the dendrites and on the cell body. There are approximately 104 synapses per neuron in the human body. Electric impulse is passed between synapse and dendrites. It is a chemical process which results in increase/decrease in the electric potential inside the body of the receiving cell. If the electric potential reaches a thresh hold value, receiving cell fires & pulse / action potential of fixed strength and duration is send through the axon to synaptic junction of the cell. After that, cell has to wait for a period called refractory period.

Difference between biological and Artificial Neuron

Characteristics	Artificial Neural Network	Biological(Real) Neural Network
Speed	Faster in processing information. Response time is in nanoseconds.	Slower in processing information. The response time is in milliseconds.
Processing	Serial processing.	Massively parallel processing.
Size & Complexity	Less size & complexity. It does not perform complex pattern recognition tasks.	Highly complex and dense network of interconnected neurons containing neurons of the order of 10^{11} with 10^{15} of interconnections.
Storage	Information storage is replaceable means new data can be added by deleting an old one.	Highly complex and dense network of interconnected neurons containing neurons of the order of 10^{11} with 10^{15} of interconnections.
Fault tolerance	Fault intolerant. Information once corrupted cannot be retrieved in case of failure of the system.	Information storage is adaptable means new information is added by adjusting the interconnection strengths without destroying old information
Control Mechanism	There is a control unit for controlling computing activities	No specific control mechanism external to the computing task.



ARTIFICIAL NEURONS:

Artificial neurons are like biological neurons that are linked to each other in various layers of the networks. These neurons are known as nodes.

A node or a neuron can receive one or more input information and process it. artificial neurons are connected by connection links to another neuron. Each connection link is associated with a synaptic weight. The structure of a single neuron is shown below:

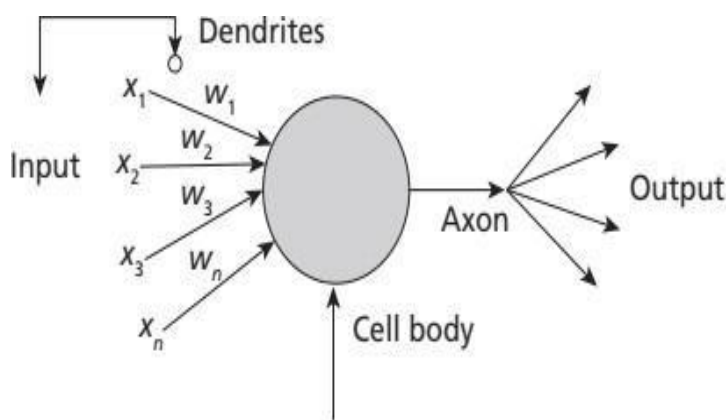


Figure 10.2: An Artificial Neuron

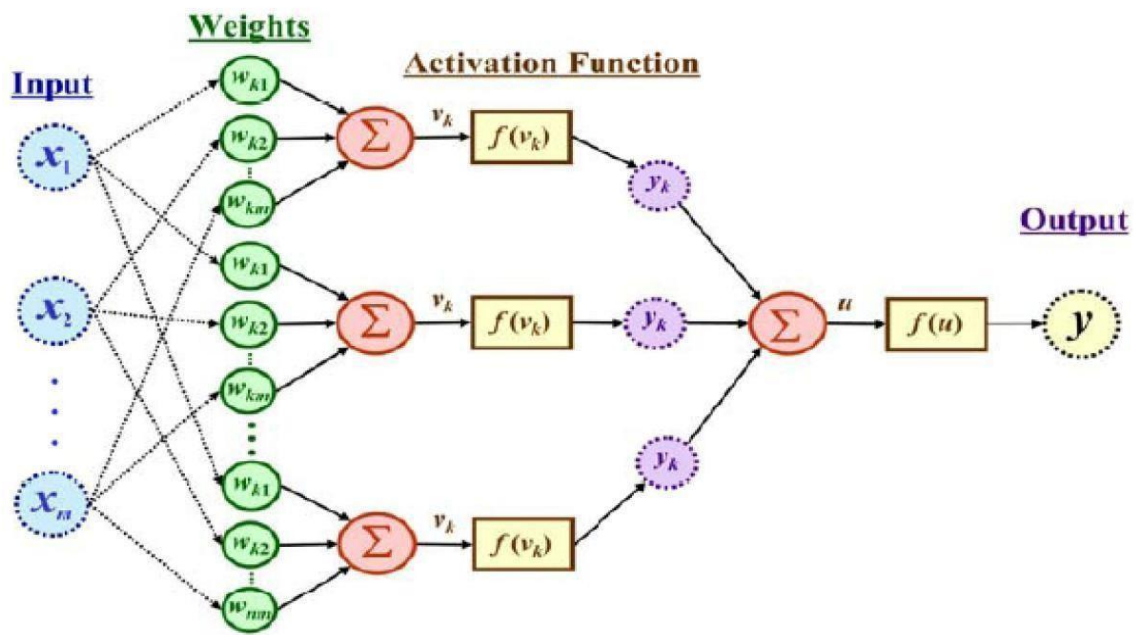


Fig: McCulloch-Pitts Neuron Mathematical model.

Simple Model of an ANN

The first mathematical model of a biological neuron was designed by McCulloch-Pitts in 1943.

It includes 2 steps:

1. It receives weighted inputs from other neurons.
2. It operates with a threshold function or activation function.

Basically, a neuron takes an input signal (dendrite), processes it like the CPU (soma), passes the output through a cable like structure to other connected neurons (axon to synapse to other neuron's dendrite).

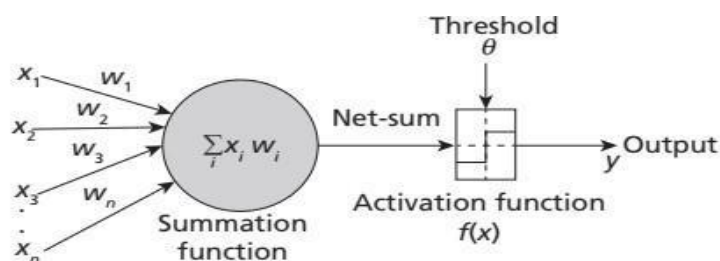
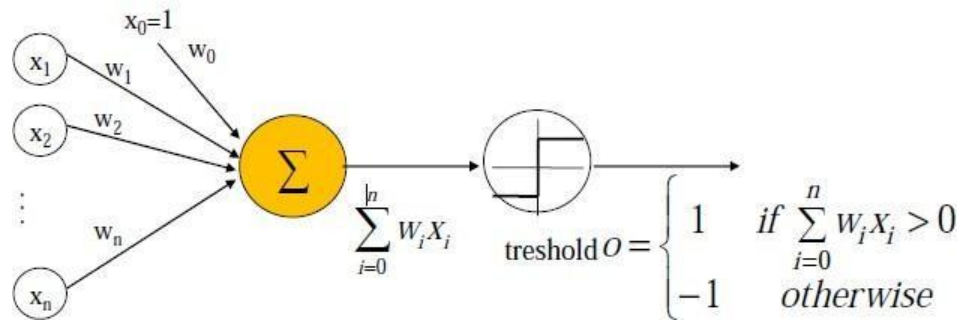


Figure 10.3: McCulloch & Pitts Neuron Mathematical Model

OR

$$W_0 + W_1 X_1 + \dots + W_n X_n = \sum_{i=0}^n W_i X_i = \vec{W} \cdot \vec{X}$$



Working:

The received input are computed as a weighted sum which is given to the activation function and if the sum exceeds the threshold value the neuron gets fired. The neuron is the basic processing unit that receives a set of inputs $x_1, x_2, x_3, \dots, x_n$ and their associated weights $w_1, w_2, w_3, \dots, w_n$. The summation function computes the weighted sum of the inputs received by the neuron.

$$\text{Sum} = \sum x_i w_i$$

Activation functions:

- To make work more efficient and for exact output, some force or activation is given. Like that, activation function is applied over the net input to calculate the output of an ANN. Information processing of processing element has two major parts: input and output. An integration function (f) is associated with input of processing element.
- Several **activation functions** are there.

- Identity function or Linear Function:** It is a linear function which is defined as $f(x) = x$ for all x

The output is same as the input ie the weighted sum. The function is useful when we do not apply any threshold. The output value ranged between $-\infty$ and $+\infty$

- Binary step function:** This function can be defined as

$$f(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases}$$

Where, θ represents threshold value. It is used in single layer nets to convert the net input to an output that is binary (0 or 1).

3. Bipolar step function: This function can be defined as

$$f(x) = \begin{cases} 1 & \text{if } x \geq \theta \\ -1 & \text{if } x < \theta \end{cases}$$

Where, θ represents threshold value. It is used in single layer nets to convert the net input to an output that is bipolar (+1 or -1).

4. Sigmoid function: It is used in Back propagation nets.

Two types:

a) **Binary sigmoid function:** It is also termed as logistic sigmoid function or unipolar sigmoid function. It is defined as

$$f(x) = \frac{1}{1 + e^{-\lambda x}}$$

where, λ represents steepness parameter. The range of sigmoid function is 0 to 1

b) **Bipolar sigmoid function:** This function is defined as

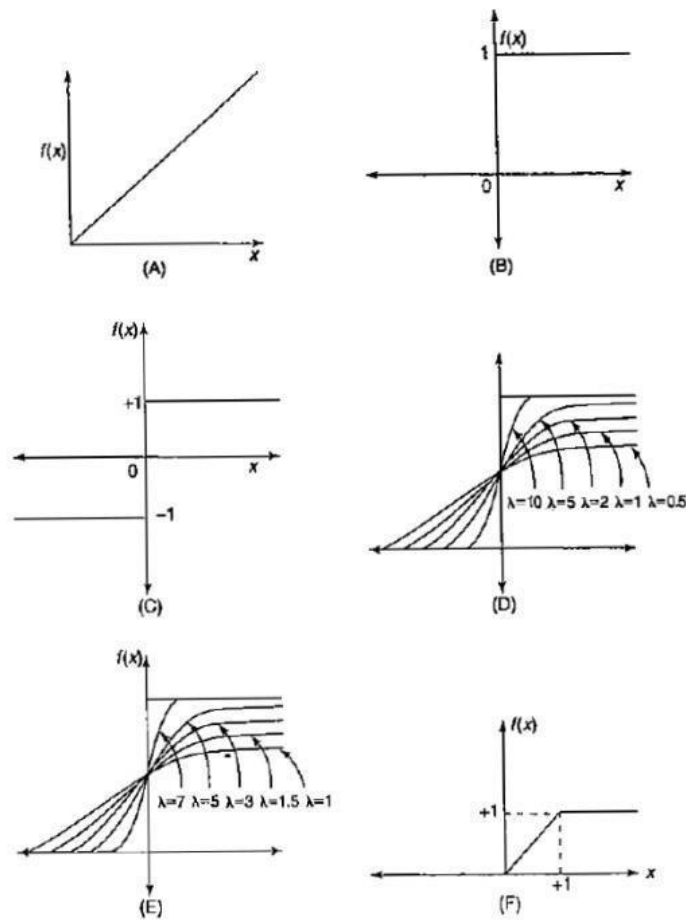
$$f(x) = \frac{2}{1 + e^{-\lambda x}} - 1 = \frac{1 - e^{-\lambda x}}{1 + e^{-\lambda x}}$$

Where λ represents steepness parameter and the sigmoid range is between -1 and +1.

5. Ramp function: The ramp function is defined as:

$$f(x) = \begin{cases} 1 & \text{if } x > 1 \\ x & \text{if } 0 \leq x \leq 1 \\ 0 & \text{if } x < 0 \end{cases}$$

It is a linear function whose upper and lower limits are fixed.



Depiction of activation functions: (A) identity function; (B) binary step function; (C) bipolar step function; (D) binary sigmoidal function; (E) bipolar sigmoidal function; (F) ramp function.

6. **Tanh-Hyperbolic tangent function** : Tanh function is very similar to the sigmoid/logistic activation function, and even has the same S-shape with the difference in output range of -1 to 1. In Tanh, the larger the input (more positive), the closer the output value will be to 1.0, whereas the smaller the input (more negative), the closer the output will be to -1.0.

$$f(x) = \frac{(e^x - e^{-x})}{(e^x + e^{-x})}$$

7. ReLU Function

ReLU stands for **Rectified Linear Unit**.

Although it gives an impression of a linear function, ReLU has a derivative function and allows for backpropagation while simultaneously making it computationally efficient.

The main catch here is that the ReLU function does not activate all the neurons at the same time.

The neurons will only be deactivated if the output of the linear transformation is less than 0

8. **Softmax function:** Softmax is an activation function that scales numbers/logits into probabilities. The output of a Softmax is a vector (say $\mathbf{v}$) with probabilities of each possible outcome. The probabilities in vector $\mathbf{v}$ sums to one for all possible outcomes or classes.

$$S(y)_i = \frac{\exp(y_i)}{\sum_{j=1}^n \exp(y_j)}$$

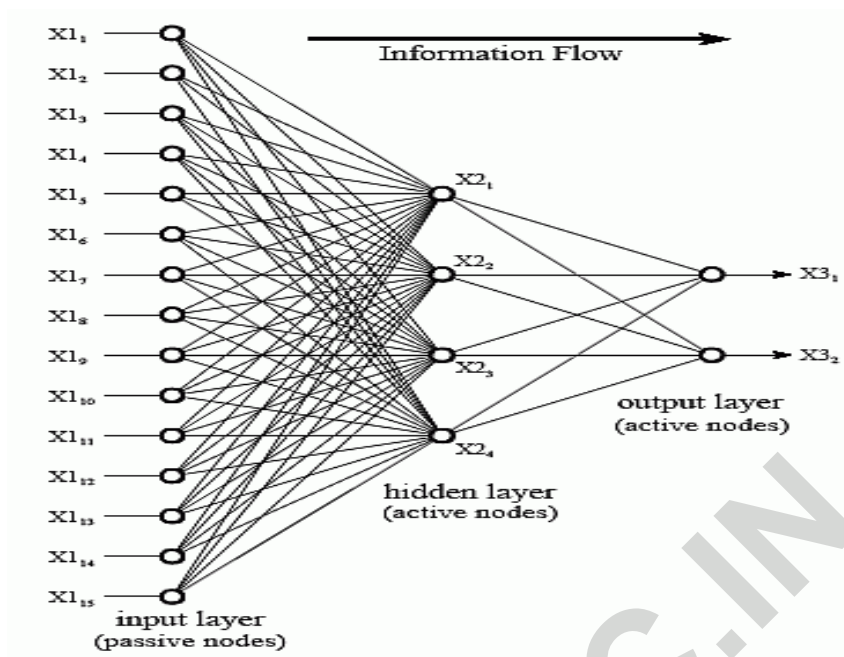
where,

y	is an input vector to a softmax function, S . It consist of n elements for n classes (possible outcomes)
y_i	the i -th element of the input vector. It can take any value between $-\infty$ and $+\infty$
$\exp(y_i)$	standard exponential function applied on y_i . The result is a small value (close to 0 but never 0) if $y_i < 0$ and a large value if y_i is large. eg $\exp(55) = 7.69e+23$ (A very large value) $\exp(-55) = 1.30e-24$ (A very small value close to 0) Note: $\exp(*)$ is just e^* where $e = 2.718$, the Euler's number.
$\sum_{j=1}^n \exp(y_j)$	A normalization term. It ensures that the values of output vector $S(y)_i$ sums to 1 for i -th class and each of them and each of them is in the range 0 and 1 which makes up a valid probability distribution.
n	Number of classes (possible outcomes)

Artificial Neural Network Structure

- Artificial Neural Networks Computational models inspired by the human brain: – Massively parallel, distributed system, made up of simple processing units (neurons) – Synaptic connection strengths among neurons are used to store the acquired knowledge.
- Knowledge is acquired by the network from its environment through a learning process.
- The Neural Network is constructed from 3 type of layers:**
- Input layer — initial data for the neural network.

- Hidden layers — intermediate layer between input and output layer and place where all the computation is done.
- Output layer — produce the result for given inputs.



PERCEPTRON AND LEARNING THEORY

- The perceptron is also a simplified model of a biological neuron.
- The perceptron is an algorithm for supervised learning of binary classifiers. It is a type of linear classifier, i.e. a classification algorithm that makes all of its predictions based on a linear predictor function combining a set of weights with the feature vector.
- One type of ANN system is based on a unit called a perceptron.

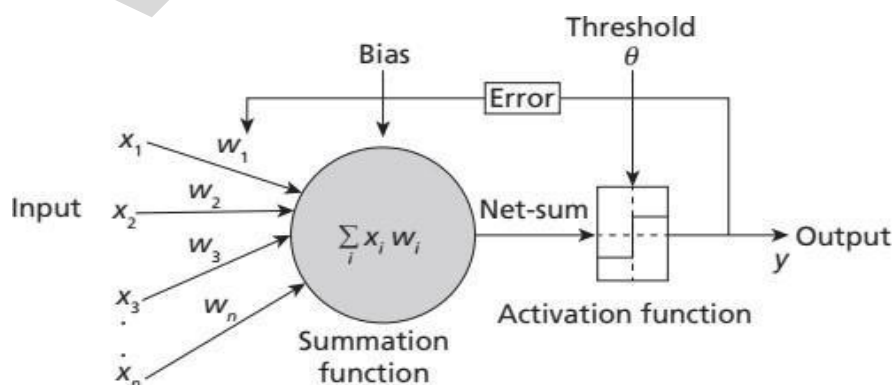
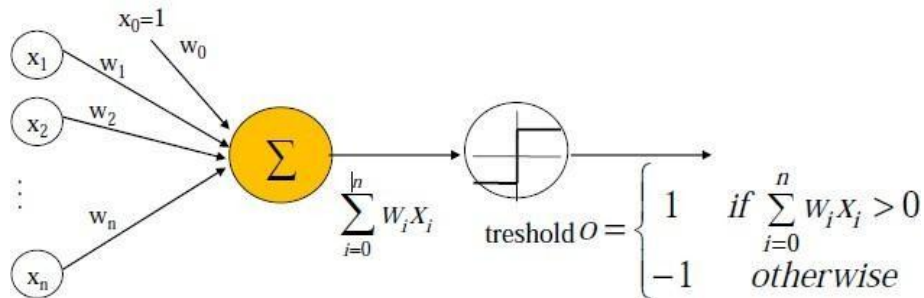


Figure 10.5: Perceptron Model

OR

$$W_0 + W_1 X_1 + \dots + W_n X_n = \sum_{i=0}^n W_i X_i = \vec{W} \cdot \vec{X}$$



- The perceptron can represent all boolean primitive functions AND, OR, NAND , NOR.
- Some boolean functions can not be represented .
 - E.g. the XOR function.

Major components of a perceptron

- Input
- Weight
- Bias
- Weighted summation
- Step/activation function
- output

WORKING:

- Feed the features of the model that is required to be trained as input in the first layer. All weights and inputs will be multiplied – the multiplied result of each weight and input will be added up. The Bias value will be added to shift the output function. This value will be presented to the activation function (the type of activation function will depend on the need). The value received after the last step is the output value.

The activation function is a binary step function which outputs a value 1, if $f(x)$ is above the threshold value Θ and a 0 if $f(x)$ is below the threshold value Θ . Then the output of a neuron is:

$$f(x) = \begin{cases} 1 & \text{if } f(x) \geq \theta \\ 0 & \text{if } f(x) < \theta \end{cases}$$

Algorithm 10.1: Perceptron Algorithm

Set initial weights $w_1, w_2, \dots, w_n$ and bias θ to a random value in the range $[-0.5, 0.5]$.

For each Epoch,

1. Compute the weighted sum by multiplying the inputs with the weights and add the products.

2. Apply the activation function on the weighted sum:

$$Y = \text{Step}((x_1 w_1 + x_2 w_2) - \theta)$$

3. If the sum is above the threshold value, output the value as positive else output the value as negative.

4. Calculate the error by subtracting the estimated output $Y_{\text{estimated}}$ from the desired output Y_{desired} :

$$\text{error } e(t) = Y_{\text{desired}} - Y_{\text{estimated}}$$

[If error $e(t)$ is positive, increase the perceptron output Y and if it is negative, decrease the perceptron output Y .]

5. Update the weights if there is an error:

$$\Delta w_i = \alpha \times e(t) \times x_i$$

$$w_i = w_i + \Delta w_i$$

where, x_i is the input value, $e(t)$ is the error at step t , α is the learning rate and Δw_i is the difference in weight that has to be added to w_i .

PROBLEM:

Design a 2 layer network of perceptron to implement NAND gate. Assume your own weights and biases in the range of $[-0.5, 0.5]$. Use learning rate as 0.4.

Solution:

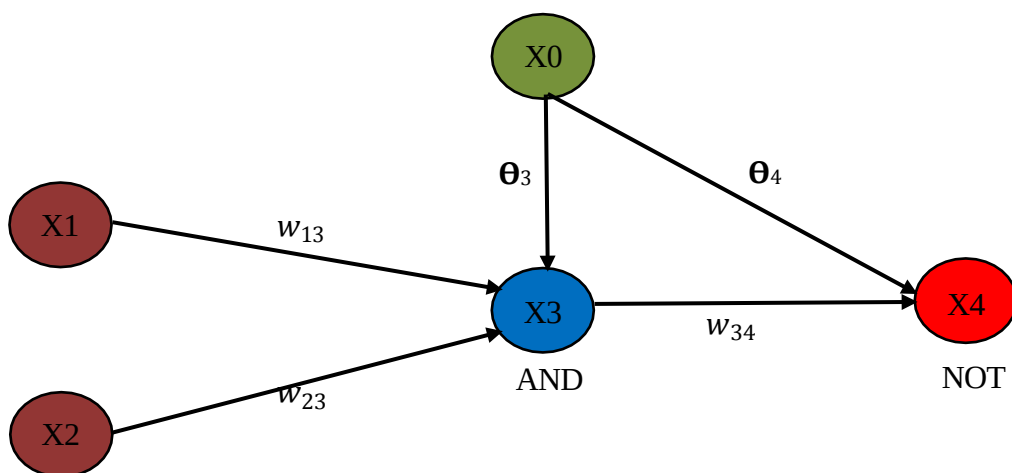


Figure 1 Two Layer Network for NAND gate

Table 1: Weights and Biases

X_1	X_2	O_{desired}	w_{13}	w_{23}	w_{34}	θ_3	θ_4	X_0
0	1	1	0.1	-0.4	0.3	0.2	-0.3	1

Table 2: Truth Table of NAND Gate

X_1	X_2	$X_1 \text{ AND } X_2$	$NAND = NOT(X_1 \text{ AND } X_2)$
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0

ITERATION 1:

Step 1: FORWARD PROPAGATION

1. Calculate net inputs and outputs in input layer as shown in Table 3.

Table 3: Net Input and Output Calculation

<i>Input Layer</i>	I_j	O_j
X_1	0	0
X_2	1	1

2. Calculate net inputs and outputs in hidden and output layer as shown in Table 4.

Table 4: Net Input and Output Calculation in Hidden and Output layer

<i>Unit_j</i>	<i>Net Input I_j</i>	<i>Net output O_j</i>
X_3	$I_3 = X_1W_{13} + X_2W_{23} + X_0\Theta_3$ $= 0(0.1) + 1(-0.4) + 1(0.2)$ $= -0.2$	$O_3 = \frac{1}{1 + e^{-I_3}}$ $= \frac{1}{1 + e^{-(-0.2)}}$ $= 0.450$
<i>Unit_k</i>	<i>Net Input I_k</i>	<i>Net output O_k</i>
X_4	$I_4 = O_3W_{34} + X_0\Theta_4$ $= (0.450 * 0.3) + 1(-0.3)$ $= -0.165$	$O_4 = \frac{1}{1 + e^{-I_4}}$ $= \frac{1}{1 + e^{-(-0.165)}}$ $= 0.458$

3. Calculate Error

$$\begin{aligned}
 \text{Error} &= O_{\text{desired}} - O_{\text{estimated}} \\
 &= 1 - 0.458
 \end{aligned}$$

$$Error = 0.542$$

Step 2: BACKWARD PROPAGATION

1. For each $unit_k$ in the output layer

$$Error_k = O_k * (1 - O_k) * (O_{desired} - O_k)$$

For each $unit_j$ in the hidden layer

$$Error_j = O_j * (1 - O_j) * (\sum_k Error_k * W_{jk})$$

Table 5: Error Calculation

<i>For each output layer unit_k</i>	<i>Error_k</i>
X_4	$Error_k = O_k * (1 - O_k) * (O_{desired} - O_k)$ $= 0.458(1 - 0.458)(1 - 0.458)$ $= 0.134$
<i>For each hidden layer unit_j</i>	<i>Error_j</i>
X_3	$Error_j = O_j * (1 - O_j) * (\sum_k Error_k * W_{jk})$ $= 0.450 * (1 - 0.450) * 0.134 * 0.3$ $= 0.0099$

2. Update Weights and biases

Table 6: Weight and Bias Calculation

<i>w_{ij}</i>	<i>$w_{ij} = w_{ij} + (\alpha * Error_j * O_i)$</i>	<i>Net Weight</i>
w_{13}	$w_{13} = w_{13} + (0.4 * Error_3 * O_1)$ $= 0.1 * (0.4 * 0.0099 * 0)$	0.1
w_{23}	$w_{23} = w_{23} + (0.4 * Error_3 * O_2)$ $= -0.4 * (0.4 * 0.0099 * 1)$	-0.396
w_{24}	$w_{24} = w_{24} + (0.4 * Error_4 * O_2)$ $= 0.3 * (0.4 * 0.134 * 0.450)$	0.324

θ_j	$\theta_j = \theta_j + (\alpha * Error_j)$	<i>Net Bias</i>
θ_3	$\theta_3 = \theta_3 + (0.4 * Error_3)$ $= 0.2 + (0.4 * 0.0099)$	0.203
θ_4	$\theta_4 = \theta_4 + (0.4 * Error_4)$ $= -0.3 + (0.4 * 0.134)$	-0.246

ITERATION 2:

Step 1: FORWARD PROPAGATION

1. Calculate net inputs and outputs in hidden and output layer

Table 7: Inputs and Outputs in Hidden and Output layer

<i>Unit_j</i>	<i>Net Input I_j</i>	<i>Net output O_j</i>
X_3	$I_3 = X_1W_{13} + X_2W_{23} + X_0\theta_3$ $= 0(0.1) + 1(-0.396) + 1(0.203)$ $= -0.193$	$O_3 = \frac{1}{1 + e^{-I_3}}$ $= \frac{1}{1 + e^{-(-0.193)}}$ $= 0.451$
<i>Unit_k</i>	<i>Net Input I_k</i>	<i>Net output O_k</i>
X_4	$I_4 = O_3W_{34} + X_0\theta_4$ $= (0.451 * 0.324) + 1(-0.246)$ $= -0.099$	$O_4 = \frac{1}{1 + e^{-I_4}}$ $= \frac{1}{1 + e^{-(-0.099)}}$ $= 0.475$

2. Calculate Error

$$Error = O_{desired} - O_{estimated}$$

$$= 1 - 0.475$$

$$Error = 0.525$$

<i>ITERATION</i>	<i>ERROR</i>
1	0.542
2	0.525

$$\downarrow = 0.542 - 0.525$$

$$= 0.017$$

In iteration 2 the error gets reduced to 0.525. This process will continue until desired output is achieved.

How a Multi-Layer Perceptron does solves the XOR problem. Design an MLP with back propagation to implement the XOR Boolean function.

Solution:

X ₁	X ₂	Y
0	0	1
0	1	0
1	0	0
1	1	1

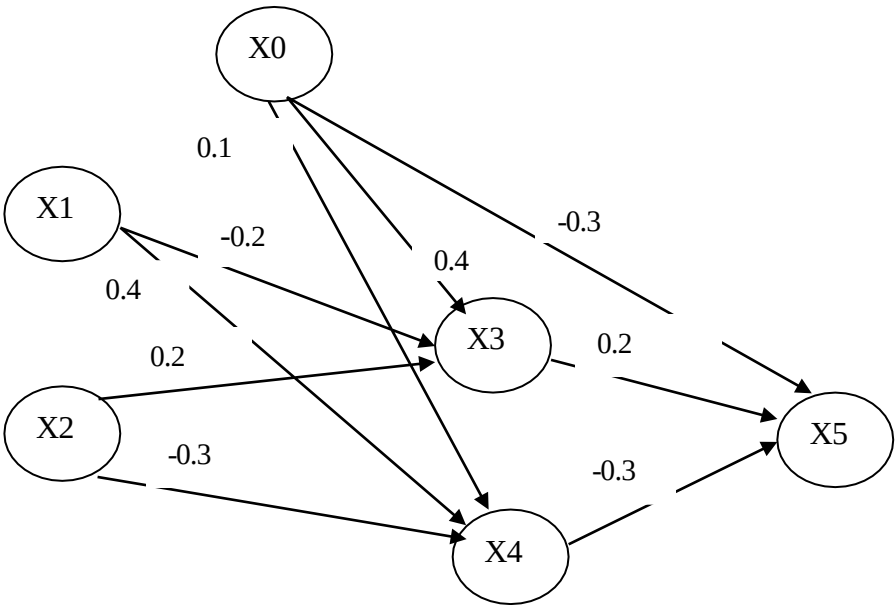


Figure 2: Multi Layer Perceptron for XOR

Learning rate: =0.8

Table 8: Weights and Biases

X ₁	X ₂	W ₁₃	W ₁₄	W ₂₃	W ₂₄	W ₃₅	W ₄₅	θ_3	θ_4	θ_5
1	0	-0.2	0.4	0.2	-0.3	0.2	-0.3	0.4	0.1	-0.3

Step 1: Forward Propagation

1. Calculate Input and Output in the Input Layer shown in Table 9.

Table 9: Net Input and Output Calculation

Input Layer	I _j	O _j
X ₁	1	1
X ₂	0	0

2. Calculate Net Input and Output in the Hidden Layer and Output Layer shown in Table 10.

Table 10: Unit j at Hidden Layer and Output Layer – Net Input and Output Calculation

Unit j	Net Input I _j	Output O _j
X ₃	$I_3 = X_1 * W_{13} + X_2 * W_{23} + X_0 * \theta_3$ $I_3 = 1 * -0.2 + 0 * 0.2 + 1 * 0.4 = 0.2$	$O_3 = \frac{1}{1+e^{-I_3}} = \frac{1}{1+e^{-0.2}} = 0.549$
X ₄	$I_4 = X_1 * W_{14} + X_2 * W_{24} + X_0 * \theta_4$ $I_4 = 1 * 0.4 + 0 * -0.3 + 1 * 0.1 = 0.5$	$O_4 = \frac{1}{1+e^{-I_4}} = \frac{1}{1+e^{-0.5}} = 0.622$
X ₅	$I_5 = O_3 * W_{35} + O_4 * W_{45} + X_0 * \theta_5$ $I_5 = 0.549 * 0.2 + 0.622 * -0.3 + 1 * -0.3 = -0.376$	$O_5 = \frac{1}{1+e^{-I_5}} = \frac{1}{1+e^{-0.376}} = 0.407$

3. Calculate Error = O_{desired} – O_{Estimated}

So error for this network is,

$$\text{Error} = O_{\text{desired}} - O_7 = 1 - 0.407 = 0.593$$

Step 2: Backward Propagation

1. Calculate Error at each node as shown in Table 11.

For each unit k in the output layer, calculate

$$\text{Error}_k = O_k (1 - O_k) (Y_N - O_k)$$

For each unit j in the hidden layer, calculate

$$\text{Error}_j = O_j (1 - O_j) \sum_k \text{Error}_k W_{jk}$$

Table 11: Error Calculation for each unit in the Output layer and Hidden layer

For Output Layer Unit k	Error _k
X ₅	$\text{Error}_5 = O_5 (1 - O_5) (1 - O_5)$ $= 0.407 * (1 - 0.407) * (1 - 0.407)$ $= 0.143$
For Hidden layer Unit j	Error _j
X ₄	$\text{Error}_4 = O_4 (1 - O_4) \sum_k \text{Error}_k W_{jk} = O_4 (1 - O_4) \text{Error}_5 W_{45}$ $= 0.622 (1 - 0.622) * -0.3 * 0.143$ $= -0.010$
X ₃	$\text{Error}_3 = O_3 (1 - O_3) \sum_k \text{Error}_k W_{jk} = O_3 (1 - O_3) \text{Error}_5 W_{35}$ $= 0.549 (1 - 0.549) * 0.143 * 0.2$

	= -0.007
--	----------

2. Update weight using the below formula,

Learning rate $\alpha = 0.8$

$$\Delta W_{ij} = \alpha * \text{Error}_j * O_i$$

$$W_{ij} = W_{ij} + \Delta W_{ij}$$

The updated weight and bias is shown in Table 12 and Table 13.

Table 12: Weight Updation

W_{ij}	$W_{ij} = W_{ij} + \alpha * \text{Error}_j * O_i$	New Weight
W_{13}	$W_{13} = W_{13} + 0.8 * \text{Error}_3 * O_1$ $= -0.2 + 0.8 * 0.007 * 1$	-0.194
W_{14}	$W_{14} = W_{14} + 0.8 * \text{Error}_4 * O_1$ $= 0.4 + 0.8 * -0.01 * 1$	0.392
W_{23}	$W_{23} = W_{23} + 0.8 * \text{Error}_3 * O_2$ $= 0.2 + 0.8 * 0.007 * 0$	0.2
W_{24}	$W_{24} = W_{24} + 0.8 * \text{Error}_4 * O_2$ $= -0.3 + 0.8 * -0.001 * 0$	-0.3
W_{35}	$W_{35} = W_{35} + 0.8 * \text{Error}_5 * O_3$ $= 0.2 + 0.8 * 0.143 * 0.4$	0.154
W_{45}	$W_{45} = W_{45} + 0.8 * \text{Error}_5 * O_4$ $= 0.3 + 0.8 * 0.143 * 0.1$	-0.288

Update bias using the below formula,

$$\Delta \theta_j = \alpha * \text{Error}_j$$

$$\theta_j = \theta_j + \Delta \theta_j$$

Table 13: Bias Updation

θ_j	$\theta_j = \theta_j + \alpha * \text{Error}_j$	New Bias
θ_3	$\theta_3 = \theta_3 + \alpha * \text{Error}_3$ $= 0.4 + 0.8 * 0.007$	0.405
θ_4	$\theta_4 = \theta_4 + \alpha * \text{Error}_4$ $= 0.1 + 0.8 * -0.01$	0.092
θ_5	$\theta_5 = \theta_5 + \alpha * \text{Error}_5$ $= -0.3 + 0.8 * 0.143$	-0.185

Iteration 2

Now with the updated weights and biases,

1. Calculate Input and Output in the Input Layer shown in Table 14.

Table 14: Net Input and Output Calculation

Input Layer	I _j	O _j
X ₁	1	1
X ₂	0	0

2. Calculate Net Input and Output in the Hidden Layer and Output Layer shown in Table 15.

Table 15: Net Input and Output Calculation in the Hidden Layer and Output Layer

Unit j	Net Input I _j	Output O _j
X ₃	$I_3 = X_1 * W_{13} + X_2 * W_{23} + X_0 * \theta_3$ $I_3 = 1 * -0.194 + 0 * 0.2 + 1 * 0.405 = 0.211$	$O_3 = \frac{1}{1+e^{-I_3}} = \frac{1}{1+e^{-0.211}} = 0.552$
X ₄	$I_4 = X_1 * W_{14} + X_2 * W_{24} + X_0 * \theta_4$ $I_4 = 1 * 0.392 + 0 * -0.3 + 1 * 0.092 = 0.484$	$O_4 = \frac{1}{1+e^{-I_4}} = \frac{1}{1+e^{-0.484}} = 0.618$
X ₅	$I_5 = O_3 * W_{35} + O_4 * W_{45} + X_0 * \theta_5$ $I_5 = 0.552 * 0.154 + 0.618 * -0.288 + 1 * -0.185 = -0.282$	$O_5 = \frac{1}{1+e^{-I_5}} = \frac{1}{1+e^{-0.282}} = 0.429$

The output we receive in the network at node 5 is 0.407.

$$\text{Error} = 1 - 0.429 = 0.571$$

Now when we compare the error, we get in the previous iteration and in the current iteration, the network has learnt which reduces the error by 0.022.

Error is reduced by 0.055: 0.593 – 0.571.

Consider the Network architecture with 4 input units and 2 output units. Consider four training samples each vector of length 4.

Training samples

i1: (1, 1, 1, 0)

i2: (0, 0, 1, 1)

i3: (1, 0, 0, 1)

i4: (0, 0, 1, 0)

Output Units: Unit 1, Unit 2

Learning rate $\eta(t) = 0.6$

Initial Weight matrix

$$\begin{bmatrix} \text{Unit 1} \\ \text{Unit 2} \end{bmatrix} : \begin{bmatrix} 0.2 & 0.8 & 0.5 & 0.1 \\ 0.3 & 0.5 & 0.4 & 0.6 \end{bmatrix}$$

Identify an algorithm to learn without supervision? How do you cluster them as we expected?

Solution:

Use Self Organizing Feature Map (SOFM)

Iteration 1:

Training Sample X_1 : (1, 1, 1, 0)

Weight matrix

$$\begin{bmatrix} \text{Unit 1} \\ \text{Unit 2} \end{bmatrix} : \begin{bmatrix} 0.2 & 0.8 & 0.5 & 0.1 \\ 0.3 & 0.5 & 0.4 & 0.6 \end{bmatrix}$$

Compute Euclidean distance between X_1 : (1, 1, 1, 0) and Unit 1 weights.

$$\begin{aligned} d^2 &= (0.2 - 1)^2 + (0.8 - 1)^2 + (0.5 - 1)^2 + (0.1 - 0)^2 \\ &= 0.94 \end{aligned}$$

Compute Euclidean distance between X_1 : (1, 1, 1, 0) and Unit 2 weights.

$$\begin{aligned} d^2 &= (0.3 - 1)^2 + (0.5 - 1)^2 + (0.4 - 1)^2 + (0.6 - 0)^2 \\ &= 1.46 \end{aligned}$$

Unit 1 wins

Update the weights of the winning unit

$$\begin{aligned} \text{New Unit 1 weights} &= [0.2 \ 0.8 \ 0.5 \ 0.2] + 0.6 ([1 \ 1 \ 1 \ 0] - [0.2 \ 0.8 \ 0.5 \ 0.2]) \\ &= [0.2 \ 0.8 \ 0.5 \ 0.2] + 0.6 [0.8 \ 0.2 \ 0.5 \ -0.2] \\ &= [0.2 \ 0.8 \ 0.5 \ 0.2] + [0.48 \ 0.12 \ 0.30 \ -0.12] \\ &= [0.68 \ 0.92 \ 0.80 \ 0.08] \end{aligned}$$

$$\begin{bmatrix} \text{Unit 1} \\ \text{Unit 2} \end{bmatrix} : \begin{bmatrix} 0.68 & 0.92 & 0.80 & 0.08 \\ 0.3 & 0.5 & 0.4 & 0.6 \end{bmatrix}$$

Iteration 2:

Training Sample X_2 : (0, 0, 1, 1)

Weight matrix

$$\begin{bmatrix} \text{Unit 1} \\ \text{Unit 2} \end{bmatrix} : \begin{bmatrix} 0.68 & 0.92 & 0.80 & 0.08 \\ 0.3 & 0.5 & 0.4 & 0.6 \end{bmatrix}$$

Compute Euclidean distance between X_2 : (0, 0, 1, 1) and Unit 1 weights.

$$\begin{aligned}d^2 &= (0.68 - 0)^2 + (0.92 - 0)^2 + (0.80 - 1)^2 + (0.08 - 1)^2 \\&= 2.1952\end{aligned}$$

Compute Euclidean distance between X_2 : (0, 0, 1, 1) and Unit 2 weights.

$$\begin{aligned}d^2 &= (0.3 - 0)^2 + (0.5 - 0)^2 + (0.4 - 1)^2 + (0.6 - 1)^2 \\&= 0.86\end{aligned}$$

Unit 2 wins

Update the weights of the winning unit

$$\begin{aligned}\text{New Unit 2 weights} &= [0.3 \ 0.5 \ 0.4 \ 0.6] + 0.6 ([0 \ 0 \ 1 \ 1] - [0.3 \ 0.5 \ 0.4 \ 0.6]) \\&= [0.3 \ 0.5 \ 0.4 \ 0.6] + 0.6 [-0.3 \ -0.5 \ 0.6 \ 0.4] \\&= [0.3 \ 0.5 \ 0.4 \ 0.6] + [-0.18 \ -0.30 \ 0.36 \ 0.24] \\&= [0.12 \ 0.2 \ 0.76 \ 0.84]\end{aligned}$$

$$\begin{array}{l} \text{Unit 1:} [0.68 \ 0.92 \ 0.80 \ 0.08] \\ \text{Unit 2:} [0.12 \ 0.2 \ 0.76 \ 0.84] \end{array}$$

Iteration 3:

Training Sample X_3 : (1, 0, 0, 1)

Weight matrix

$$\begin{array}{l} \text{Unit 1:} [0.68 \ 0.92 \ 0.80 \ 0.08] \\ \text{Unit 2:} [0.12 \ 0.2 \ 0.76 \ 0.84] \end{array}$$

Compute Euclidean distance between X_3 : (1, 0, 0, 1) and Unit 1 weights.

$$\begin{aligned}d^2 &= (0.68 - 1)^2 + (0.92 - 0)^2 + (0.80 - 0)^2 + (0.08 - 1)^2 \\&= 2.44\end{aligned}$$

Compute Euclidean distance between X_3 : (1, 0, 0, 1) and Unit 2 weights.

$$\begin{aligned}d^2 &= (0.12 - 1)^2 + (0.2 - 0)^2 + (0.76 - 0)^2 + (0.84 - 1)^2 \\&= 1.42\end{aligned}$$

Unit 2 wins

Update the weights of the winning unit

$$\begin{aligned}\text{New Unit 2 weights} &= [0.12 \ 0.2 \ 0.76 \ 0.84] + 0.6 ([1 \ 0 \ 0 \ 1] - [0.12 \ 0.2 \ 0.76 \ 0.84]) \\&= [0.12 \ 0.2 \ 0.76 \ 0.84] + 0.6 [0.88 \ -0.2 \ -0.76 \ 0.16]\end{aligned}$$

$$= [0.12 \ 0.2 \ 0.76 \ 0.84] + [0.53 \ -0.12 \ -0.46 \ 0.096]$$

$$= [0.65 \ 0.08 \ 0.3 \ 0.94]$$

$$\begin{bmatrix} \text{Unit 1} \\ \text{Unit 2} \end{bmatrix} = \begin{bmatrix} 0.68 & 0.92 & 0.80 & 0.08 \\ 0.65 & 0.08 & 0.3 & 0.94 \end{bmatrix}$$

Iteration 4:

Training Sample X₄: (0, 0, 1, 0)

Weight matrix

$$\begin{bmatrix} \text{Unit 1} \\ \text{Unit 2} \end{bmatrix} = \begin{bmatrix} 0.68 & 0.92 & 0.80 & 0.08 \\ 0.65 & 0.08 & 0.3 & 0.94 \end{bmatrix}$$

Compute Euclidean distance between X₄: (0, 0, 1, 0) and Unit 1 weights.

$$d^2 = (0.68 - 0)^2 + (0.92 - 0)^2 + (0.80 - 1)^2 + (0.08 - 0)^2$$

$$= 1.36$$

Compute Euclidean distance between X₁: (0, 0, 1, 0) and Unit 2 weights.

$$d^2 = (0.65 - 0)^2 + (0.08 - 0)^2 + (0.3 - 1)^2 + (0.94 - 0)^2$$

$$= 1.8025$$

Unit 1 wins

Update the weights of the winning unit

$$\begin{aligned} \text{New Unit 1 weights} &= [0.68 \ 0.92 \ 0.80 \ 0.08] + 0.6 ([0 \ 0 \ 1 \ 0] - [0.68 \ 0.92 \ 0.80 \ 0.08]) \\ &= [0.68 \ 0.92 \ 0.80 \ 0.08] + 0.6 [-0.68 \ -0.92 \ 0.2 \ -0.08] \\ &= [0.68 \ 0.92 \ 0.80 \ 0.08] + [-0.408 \ -0.552 \ 0.12 \ -0.258] \\ &= [0.27 \ 0.37 \ 0.92 \ -0.178] \end{aligned}$$

$$\begin{bmatrix} \text{Unit 1} \\ \text{Unit 2} \end{bmatrix} = \begin{bmatrix} 0.27 & 0.37 & 0.92 & -0.178 \\ 0.65 & 0.08 & 0.3 & 0.94 \end{bmatrix}$$

Best mapping unit for each of the sample taken are,

X₁: (1, 1, 1, 0) → Unit 1

X₂: (0, 0, 1, 1) → Unit 2

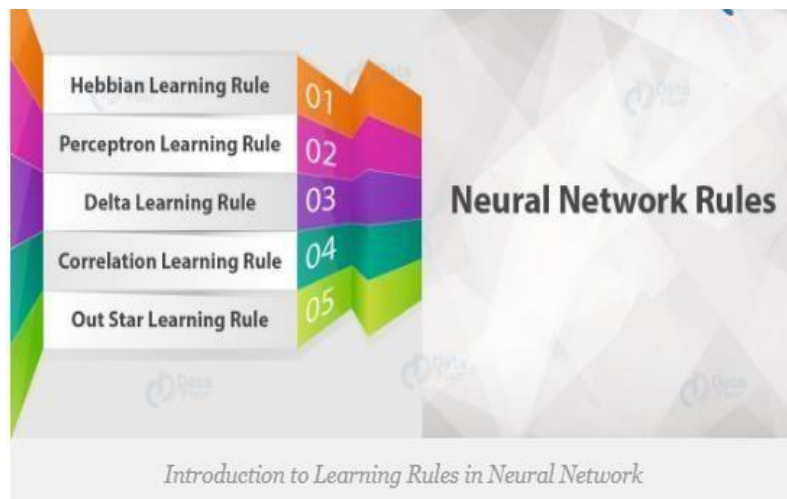
X₃: (1, 0, 0, 1) → Unit 2

X₄: (0, 0, 1, 0) → Unit 1

This process is continued for many epochs until the feature map doesn't change.

Learning Rules

Learning in NN is performed by adjusting the network weights in order to minimize the difference between the desired and estimated output.



Delta Learning Rule and Gradient Descent

- ▮ Developed by **Widrow and Hoff**, the delta rule, is one of the most common learning rules.
- ▮ It is **supervised** learning.
- ▮ Delta rule is derived from gradient descent method(Back-propagation).
- ▮ It is **Non-linearly separable**. Also called as continuous perceptron Learning rule.
- ▮ It updates the connection weights with the difference between the target and the output value. It is the least mean square learning algorithm.
- ▮ The Delta difference is measured as an **error function** or also called as **cost** function.

$$\text{Training Error} = \frac{1}{2} \sum_{d \in T} (O_{\text{Desired}} - O_{\text{Estimated}})^2$$

where, T is the training dataset, O_{Desired} and $O_{\text{Estimated}}$ are the desired target output and estimated actual output, respectively, for a training instance d .

The principle of gradient descent is an optimization approach which is used to minimize the cost function by converging to a local minimal point moving in the negative direction of the gradient and each step size during movement is determined by the learning rate and the slope of the gradient.

TYPES OF ANN

1. Feed Forward Neural Network
2. Fully connected Neural Network
3. Multilayer Perceptron
4. Feedback Neural Network

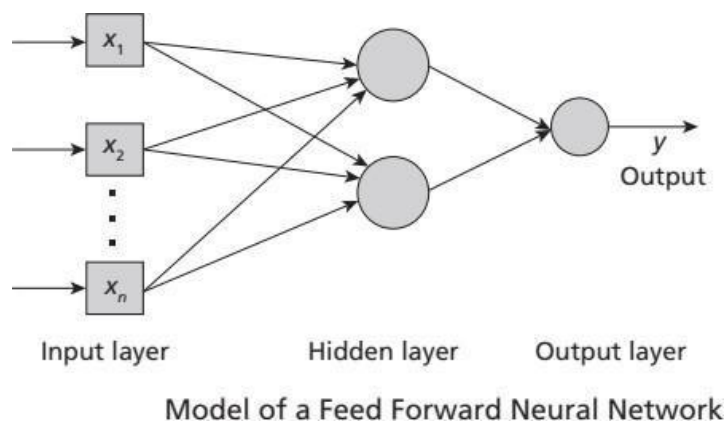
Feed Forward Neural Network:

Feed-Forward Neural Network is a single layer perceptron. A sequence of inputs enters the layer and are multiplied by the weights in this model. The weighted input values are then summed together to form a total. If the sum of the values is more than a predetermined threshold, which is normally set at zero, the output value is usually 1, and if the sum is less than the threshold, the output value is usually -1.

The single-layer perceptron is a popular feed-forward neural network model that is frequently used for classification.

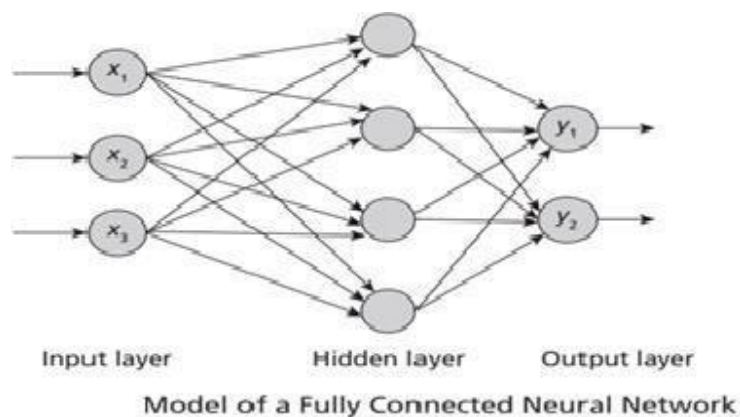
The model may or may not contain hidden layer and there is no backpropagation.

Based on the number of hidden layers they are further classified into single-layered and multilayered feed forward network.



Fully connected Neural Network:

- A fully connected neural network consists of a series of fully connected layers that connect every neuron in one layer to every neuron in the other layer.
- The major advantage of fully connected networks is that they are “structure agnostic” i.e. there are no special assumptions needed to be made about the input.



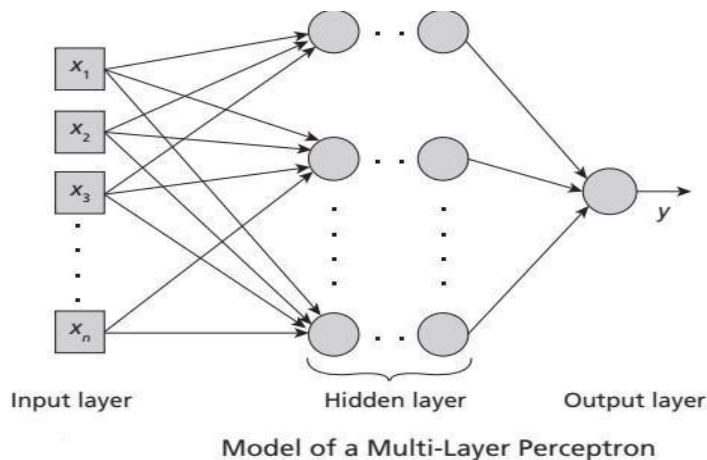
Multilayer Perceptron:

A multi-layer perceptron has one input layer and for each input, there is one neuron (or node), it has one output layer with a single node for each output and it can have any number of hidden layers and each hidden layer can have any number of nodes.

The information flows in both directions.

The weight adjustment training is done via **backpropagation**.

Every node in the multi-layer perceptron uses a sigmoid activation function. The sigmoid activation function takes real values as input and converts them to numbers between 0 and 1 using the sigmoid formula.



Feedback Neural Network:

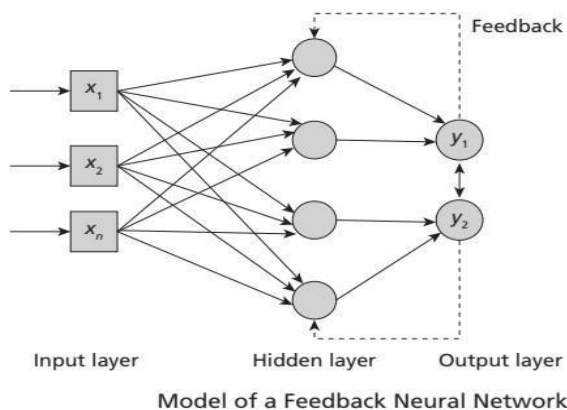
Feedback networks also known as **recurrent neural network** or interactive neural network are the deep learning models in which information flows in **backward** direction.

It allows feedback loops in the network. Feedback networks are dynamic in nature, powerful and can get much complicated at some stage of execution

Neuronal connections can be made in any way.

RNNs may process input sequences of different lengths by using their internal state, which can represent a form of memory.

They can therefore be used for applications like speech recognition or handwriting recognition.



Advantages and Disadvantages of ANN

1. ANN can solve complex problems involving non-linear processes.
2. ANNs can learn and recognize complex patterns and solve problems as humans solve a problem.
3. ANNs have a parallel processing capability and can predict in less time.
4. They have an ability to work with inadequate knowledge. It can even handle incomplete and noisy data.
5. They can scale well to larger data sets and outperforms other learning mechanisms.

Limitations of ANN

1. An ANN requires processors with parallel processing capability to train the network running for many epochs. The function of each node requires a CPU capability which is difficult for very large networks with a large amount of data.
2. They work like a 'black box' and it is exceedingly difficult to understand their working in inner layers. Moreover, it is hard to understand the relationship between the representations learned at each layer.

3. The modelling with ANN is also extremely complicated and the development takes a much longer time.
4. Generally, neural networks require more data than traditional machine learning algorithms, and they do not perform well on small datasets.
5. They are also more computationally expensive than traditional learning techniques.

Challenges of Artificial Neural Networks

The major challenges while modelling a real-time application with ANNs are:

1. Training a neural network is the most challenging part of using this technique. Overfitting or underfitting issues may arise if datasets used for training are not correct. It is also hard to generalize to the real-world data when trained with some simulated data. Moreover, neural network models normally need a lot of training data to be robust and are usable for a real-time application.
2. Finding the weight and bias parameters for neural networks is also hard and it is difficult to calculate an optimal model.

Module-5

Chapter – 01 - Clustering Algorithms

Introduction to Clustering Approaches

- Cluster analysis is the fundamental task of unsupervised learning. Unsupervised learning involves exploring the given dataset.
- Cluster analysis is a technique of partitioning a collection of unlabelled objects that have many attributes into meaningful disjoint groups or clusters.
- This is done using a trial and error approach as there are no supervisors available as in classification.
- The characteristic of clustering is that the objects in the clusters or groups are similar to each other within the clusters while differ from the objects in other clusters significantly.
- The input for cluster analysis is examples or samples. These are known as objects, data points or data instances.
- All these terms are same and used interchangeably in this chapter. All the samples or objects with no labels associated with them are called unlabelled.
- The output is the set of clusters (or groups) of similar data if it exists in the input.
- For example, the following Figure 13.1(a) shows data points or samples with two features shown in different shaded samples and Figure 13.1(b) shows the manually drawn ellipse to indicate the clusters formed.

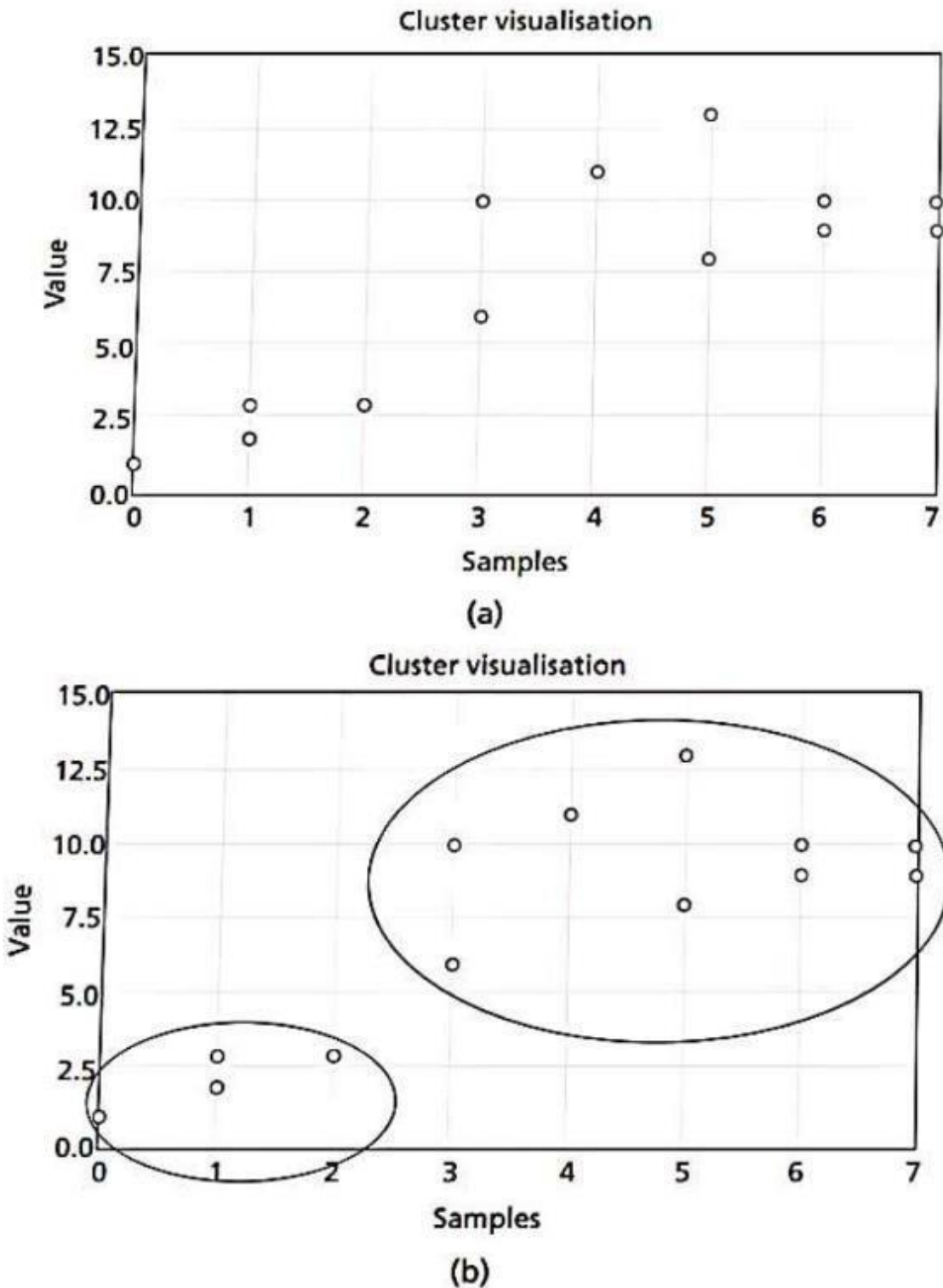


Figure 13.1: (a) Data Samples (b) Clusters' Description

Visual identification of clusters in this case is easy as the examples have only two features.

But, when examples have more features, say 100, then clustering cannot be done manually and automatic clustering algorithms are required.

Also, automating the clustering process is desirable as these tasks are considered difficult by humans and almost impossible. All clusters are represented by centroids.

Example: For example, if the input examples or data is (3, 3), (2, 6) and (7, 9), then the centroid is given as.

$$\left(\frac{3 + 2 + 7, 3 + 6 + 9}{3} \right) = (4, 6)$$

The clusters should not overlap and every cluster should represent only one class. Therefore, clustering algorithms use trial and error method to form clusters that can be converted to labels.

Difference between Clustering & Classification

S.No.	Clustering	Classification
1.	Unsupervised learning and cluster formation are done by trial and error as there is no supervisor	Supervised learning with the presence of a supervisor to provide training and testing data
2.	Unlabelled data	Labelled data
3.	No prior knowledge in clustering	Knowledge of the domain is must to label the samples of the dataset
4.	Cluster results are dynamic	Once a label is assigned, it does not change

Applications of Clustering

1. Grouping based on customer buying patterns
2. Profiling of customers based on lifestyle
3. In information retrieval applications (like retrieval of a document from a collection of documents)
4. Identifying the groups of genes that influence a disease
5. Identification of organs that are similar in physiology functions
6. Taxonomy of animals, plants in Biology
7. Clustering based on purchasing behaviour and demography
8. Document indexing
9. Data compression by grouping similar objects and finding duplicate objects

Challenges of Clustering Algorithms

High-Dimensional Data

- As the number of features increases, clustering becomes difficult.

Scalability Issue

- Some algorithms perform well for **small datasets** but fail for **large-scale data**.

Unit Inconsistency

- Different measurement units (e.g., kg vs. pounds) can create problems.

Proximity Measure Design

- Choosing an **appropriate distance metric** is crucial for accurate clustering.

Advantages and Disadvantages of Clustering Algorithms

S.No.	Advantages	Disadvantages
1.	Cluster analysis algorithms can handle missing data and outliers.	Cluster analysis algorithms are sensitive to initialization and order of the input data.
2.	Can help classifiers in labelling the unlabelled data. Semi-supervised algorithms use cluster analysis algorithms to label the unlabelled data and then use classifiers to classify them.	Often, the number of clusters present in the data have to be specified by the user.
3.	It is easy to explain the cluster analysis algorithms and to implement them.	Scaling is a problem.
4.	Clustering is the oldest technique in statistics and it is easy to explain. It is also relatively easy to implement.	Designing a proximity measure for the given data is an issue.

Proximity Measures

Proximity measures determine **similarity or dissimilarity** among objects. **Distance**

measures (dissimilarity) indicate **how different** objects are.

Similarity measures indicate **how alike** objects are. **property:**

More distance → Less similarity, and vice versa. **Properties of**

Distance Measures (Metric Conditions)

The properties of the distance measures are:

1. D_{ij} is always positive or zero.
2. $D_{ij} = 0$, i.e., the distance between the object to itself is 0.
3. $D_{ij} = D_{ji}$. This property is called symmetry.
4. $D_{ij} \leq D_{ik} + D_{kj}$. This property is called triangular inequality.

Types of Distance Measures Based on Data Types Quantitative

Variables

1. Euclidean Distance (L_2 norm)

- Formula:

$$D(x, y) = \sqrt{\sum (x_i - y_i)^2}$$

- **Advantage:** Simple and widely used.
- **Disadvantage:** Affected by scale changes (e.g., unit conversion).

2. Manhattan Distance (L_1 norm / City Block Distance)

- Formula:

$$D(x, y) = \sum |x_i - y_i|$$

- **Advantage:** Less sensitive to outliers than Euclidean.

3. Chebyshev Distance (L_∞ norm)

- Formula:

$$D(x, y) = \max |x_i - y_i|$$

- **Used when** max difference in any feature matters.

4. Minkowski Distance

- General form:

$$D(x, y) = \left(\sum |x_i - y_i|^r \right)^{1/r}$$

- When $r = 1$, it is **Manhattan Distance**.
- When $r = 2$, it is **Euclidean Distance**.
- When $r \rightarrow \infty$, it is **Chebyshev Distance**.

Binary Attributes

1. Simple Matching Coefficient (SMC)

- Measures similarity between two binary objects.
- Formula:

$$SMC = \frac{a + d}{a + b + c + d}$$

where:

- **a** = count of (0,0) matches
- **d** = count of (1,1) matches
- **b, c** = mismatches

2. Jaccard Coefficient

- Used when **only "1" values** are important.
- Formula:

$$J = \frac{d}{b + c + d}$$

3. Hamming Distance

- Counts positions where two binary objects differ.
- Example: Distance between **101** and **110** is **2**.

Categorical Variables

Distance is 1 if different, 0 if same.

Example: Gender (Male, Female) → Distance = 1

Ordinal Variables

- Have an **inherent order** (e.g., low < medium < high).
- Formula:

$$D(X, Y) = \frac{|Position(X) - Position(Y)|}{(n - 1)}$$

- Example: Distance between **Clerk (1)** and **Manager (3)**:

$$\frac{|1 - 3|}{3} = \frac{2}{3}$$

Vector-Based Distance Measures (For Text & Documents)

Cosine Similarity

- Measures angle between two vectors.
- Formula:

$$\text{sim}(X, Y) = \frac{\sum x_i y_i}{\sqrt{\sum x_i^2} \times \sqrt{\sum y_i^2}}$$

- Used in **text analysis and document clustering**.

Distance Measures

Measure	Formula	Best for
Euclidean	$\sqrt{\sum (x_i - y_i)^2}$	General numerical data
Manhattan	$\sum$	$x_i - y_i$
Chebyshev	$\max$	$x_i - y_i$
Minkowski	$\left(\sum$	$x_i - y_i$
Simple Matching	$\frac{a+d}{a+b+c+d}$	Binary attribute similarity
Jaccard	$\frac{d}{b+c+d}$	Sparse binary attributes
Hamming	Count of differing bits	Binary sequences
Cosine Similarity	$\frac{\sum x_i y_i}{\sqrt{\sum x_i^2} \times \sqrt{\sum y_i^2}}$	Text & document similarity

Hierarchical Clustering Algorithms

Overview

- Produces a nested partition of objects with hierarchical relationships.
- Represented using a **dendrogram**.
- Two main categories: **Agglomerative** and **Divisive** methods.

Types of Hierarchical Clustering

1. Agglomerative Methods (Bottom-Up)

- Each sample starts as an individual cluster.
- Clusters are merged iteratively until one cluster remains.
- Once a cluster is formed, it cannot be undone (**irreversible**).

2. Divisive Methods (Top-Down)

- Starts with a single cluster containing all data points.
- Splits iteratively into smaller clusters.
- Continues until each sample becomes its own cluster.

Agglomerative Clustering Techniques

Single Linkage (MIN Algorithm)

- Merges clusters based on the **smallest distance** between two points from different clusters.
- Related to the **Minimum Spanning Tree (MST)**.

Complete Linkage (MAX or Clique Algorithm)

In complete linkage algorithm, the distance (x, y) , (where x is from one cluster and y is from another cluster), is the largest distance between all possible pairs of the two groups or clusters (or simply the largest distance of two points where points are in different clusters) as given below. It is used for merging the clusters.

$$D_{CL}(C_i, C_j) = \text{maximum}_{a \in C_i, b \in C_j} d(a, b) \quad (13.11)$$

Dendrogram for the above clustering is shown in Figure 13.3.

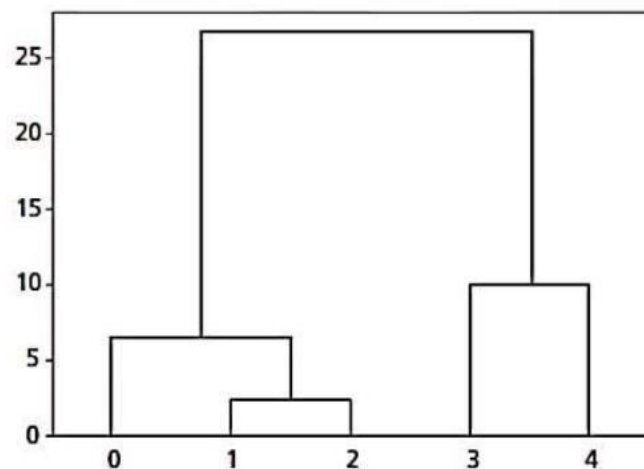


Figure 13.3: Dendrogram for Table 13.4

Average Linkage Algorithm

In case of an average linkage algorithm, the average distance of all pairs of points across the clusters is used to form clusters. The average value computed between clusters c_i, c_j is given as follows:

$$D_{AL}(C_i, C_j) = \frac{1}{m_i m_j} \sum_{a \in C_i, b \in C_j} d(a, b) \quad (13.12)$$

Here, m_i and m_j are sizes of the clusters.

The dendrogram for Table 13.4 is given in Figure 13.4.

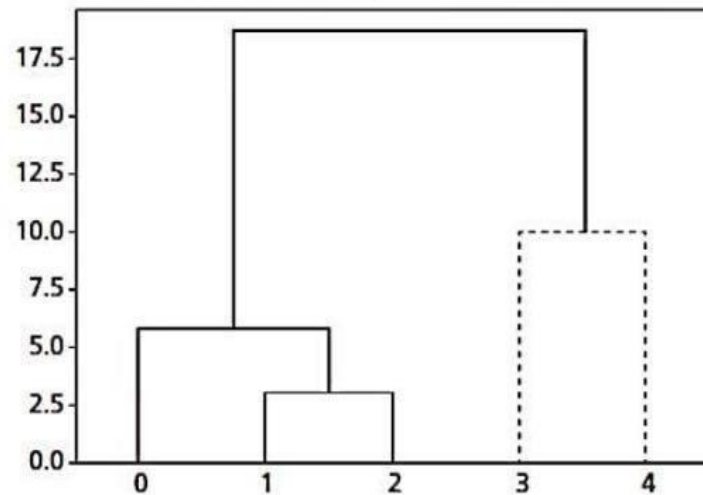


Figure 13.4: Dendrogram for Table 13.4

Mean-Shift Clustering Algorithm

- **Non-parametric and hierarchical** clustering technique.
- Also known as **mode-seeking** or **sliding window algorithm**.
- No prior knowledge of cluster count or shape required.
- Moves towards high-density regions in data using a **kernel function** (e.g., **Gaussian window**).

Advantages of Mean-Shift Clustering

No model assumptions

Suitable for all non-convex shapes

Only one parameter of the window, that is, bandwidth is required Robust to noise No

issues of local minima or premature termination

Disadvantages of Mean-Shift Clustering

Selecting the bandwidth is a challenging task. If it is larger, then many clusters are missed. If it is small, then many points are missed and convergence occurs as the problem.

The number of clusters cannot be specified and user has no control over this parameter.

Partitional Clustering Algorithm

- **k-means** is a widely used **partitional clustering** algorithm.
- The user specifies **k**, the number of clusters.
- Assumes **non-overlapping** clusters.
- Works well for **circular or spherical clusters**.

Process of k-means Algorithm

1. Initialization

- Select **k** initial cluster centers (randomly or using prior knowledge).
- Normalize data for better performance.

2. Assignment of Data Points

- Assign each data point to the **nearest centroid** based on **Euclidean distance**.

3. Update Centroids

- Compute the **mean vector** of assigned points to update cluster centroids.
- Repeat this process until no changes occur in cluster assignments.

4. Termination

- The process stops when cluster assignments remain unchanged.

Algorithm 13.3: *k*-means Algorithm

Step 1: Determine the number of clusters before the algorithm is started. This is called k .

Step 2: Choose k instances randomly. These are initial cluster centers.

Step 3: Compute the mean of the initial clusters and assign the remaining sample to the closest cluster based on Euclidean distance or any other distance measure between the instances and the centroid of the clusters.

Step 4: Compute new centroid again considering the newly added samples.

Step 5: Perform the steps 3–4 till the algorithm becomes stable with no more changes in assignment of instances and clusters.

Mathematical Optimization

- **Objective:** Minimize Sum of Squared Errors (SSE)

$$SSE = \sum_{i=1}^k \sum_{x \in C_i} \text{dist}(c_i, x)^2$$

Where:

- c_i = centroid of cluster i
- x = data point
- **dist** = Euclidean distance

Advantages

1. **Simple** and easy to implement.
2. **Efficient** for small to medium datasets.

Disadvantages

1. **Sensitive to initialization** – different initial points may lead to different results.
2. **Time-consuming for large datasets** – requires multiple iterations.

Choosing the Value of k

- No fixed rule for selecting **k**.
- Use **Elbow Method**:
 - Run k-means with different values of **k**.
 - Plot **Within Cluster Sum of Squares (WCSS)** vs. **k**.
 - The optimal **k** is at the "**elbow**" where the curve flattens.

Computational Complexity

$O(nkId)$, where:

- **n** = number of samples
- **k** = number of clusters
- **I** = number of iterations
- **d** = number of attributes

Density-based Methods

- **DBSCAN (Density-Based Spatial Clustering of Applications with Noise)** is a **density-based** clustering algorithm.
- Clusters are **dense regions** of data points separated by areas of **low density (noise)**.
- Works well for **arbitrary-shaped clusters** and datasets with **noise**.

Algorithm 13.4: DBSCAN

- Step 1:** Randomly select a point p . Compute distance between p and all other points.
- Step 2:** Find all points from p with respect to its neighbourhood and check whether it has minimum number of points m . If so, it is marked as a core point.
- Step 3:** If it is a core point, then a new cluster is formed, or existing cluster is enlarged.
- Step 4:** If it is a border point, then the algorithm moves to the next point and marks it as visited.
- Step 5:** If it is a noise point, they are removed.
- Step 6:** Merge the clusters if it is mergeable, $dist(c_i, c_j) < \epsilon$.
- Step 7:** Repeat the process 3–6 till all points are processed.

Uses two parameters:

1. ϵ (**epsilon**) – Neighborhood radius.
2. **m (minPts)** – Minimum number of points within ϵ to form a cluster.

Types of Points in DBSCAN

1. **Core Point**
 - A point with at least **m** points in its **ϵ -neighborhood**.
2. **Border Point**
 - Has **fewer than m** points in its **ϵ -neighborhood** but is adjacent to a **core point**.
3. **Noise Point (Outlier)**
 - Neither a **core point** nor a **border point**.

Density Connectivity Measures

1. **Direct Density Reachability**
 - Point **X** is directly reachable from **Y** if:
 - **X** is in the **ϵ -neighborhood** of **Y**.
 - **Y** is a **core point**.
2. **Densely Reachable**

- **X** is densely reachable from **Y** if there exists a **chain of core points** linking them.

3. Density Connected

- **X and Y** are density connected if they are both **densely reachable** from a common **core point Z**.

Advantages of DBSCAN

1. Can detect arbitrary-shaped clusters.
2. Robust to noise and outliers.
3. Does not require specifying the number of clusters (k-means does).

Disadvantages of DBSCAN

1. Sensitive to ϵ and m parameters – Poor parameter choice can affect results.
2. Fails in datasets with varying density – A single ϵ may not work for all clusters.
3. Computationally expensive for high-dimensional data.

Grid-based Approach

- **Grid-based clustering** partitions space into a **grid structure** and fits data into **cells** for clustering.
- Suitable for **high-dimensional data**.
- Uses **subspace clustering**, **dense cells**, and **monotonicity property**.

Concepts

Subspace Clustering

- Clusters are formed using a **subset of features (dimensions)** rather than all attributes.
- Useful for **high-dimensional data** like **gene expression** analysis.

- **CLIQUE (Clustering in Quest)** is a widely used **grid-based subspace clustering algorithm**.

Concept of Dense Cells

- CLIQUE partitions dimensions into **intervals (cells)**.
- A cell is **dense** if its data point density **exceeds a threshold**.
- Dense cells are **merged** to form clusters.

Algorithm 13.5: Dense Cells

Step 1: Define a set of grid points and assign the given data points on the grid.

Step 2: Determine the dense and sparse cells. If the number of points in a cell exceeds the threshold value τ , the cell is categorized as dense cell. Sparse cells are removed from the list.

Step 3: Merge the dense cells if they are adjacent.

Step 4: Form a list of grid cells for every subspace as output.

Monotonicity Property

Algorithm 13.6: CLIQUE

Stage 1:

Step 1: Identify the dense cells.

Step 2: Merge dense cells c_1 and c_2 if they share the same interval.

Step 3: Generate Apriori rule to generate $(k + 1)^{\text{th}}$ cell for higher dimension. Then, check whether the number of points cross the threshold. This is repeated till there are no dense cells or new generation of dense cells.

Stage 2:

Step 1: Merging of dense cells into a cluster is carried out in each subspace using maximal regions to cover dense cells. The maximal region is an hyperrectangle where all cells fall into.

Step 2: Maximal region tries to cover all dense cells to form clusters.

In stage two, CLIQUE starts from dimension 2 and starts merging. This process is continued till the n -dimension.

- Uses **anti-monotonicity (Apriori property)**:
 - If a **k-dimensional cell** is dense, then **all (k-1) dimensional projections** must also be dense.
 - If a lower-dimensional cell is **not dense**, then higher-dimensional cells containing it are **also not dense**.
- Similar to **association rule mining** in frequent pattern mining.

Advantages of CLIQUE

1. **Insensitive to input order of objects.**
2. **No assumptions about data distribution.**
3. **Finds high-density clusters in subspaces** of high-dimensional data.

Disadvantage of CLIQUE

- **Tuning grid parameters (grid size, density threshold) is difficult.**
- **Finding the optimal threshold** to classify a cell as dense is challenging.

Chapter :- 2

Reinforcement Learning

Overview of Reinforcement Learning

What is Reinforcement Learning?

- **Reinforcement Learning (RL)** is a **machine learning** paradigm that mimics how **humans** and **animals** learn through **experience**.
- Humans **interact with the environment**, receive **feedback (rewards or penalties)**, and adjust their behavior accordingly.
- **Example:** A child touching fire learns to avoid it after experiencing pain (negative reinforcement).

How RL Works in Machines

- RL **simulates** real-world scenarios for a computer program (agent) to **learn** by trial and error.
- The agent **executes actions**, receives **positive or negative rewards**, and **optimizes its future actions** based on these experiences.

Types of Reinforcement Learning

1. Positive Reinforcement Learning

- **Rewards encourage good behavior** (reinforce correct actions).
- **Example:** A robot gets **+10 points** for reaching a goal successfully.
- **Effect:** Increases the likelihood of repeating the rewarded action.

2. Negative Reinforcement Learning

- **Negative rewards discourage unwanted actions.**
- **Example:** A game agent loses **-10 points** for stepping into a danger zone.
- **Effect:** Helps the agent learn to **avoid negative outcomes**.

Characteristics of RL

- **Sequential Decision-Making:** The agent **makes a series of decisions** to maximize total rewards.
- **Trial and Error Learning:** The agent learns by **exploring different actions** and their consequences.
- **No Supervised Labels:** Unlike **supervised learning**, RL does **not require labeled data**; it learns from experience.

Applications of Reinforcement Learning

- **Robotics:** Teaching robots to **walk, grasp objects, or perform complex tasks**.
- **Gaming:** AI agents in **chess, Go, and video games** (e.g., AlphaGo, OpenAI Five).
- **Autonomous Vehicles:** Self-driving cars learn **optimal driving strategies**.
- **Finance:** AI-based **trading strategies** for stock markets.
- **Healthcare:** Personalized treatment plans based on patient responses.

Scope of Reinforcement Learning

Reinforcement Learning (RL) is well-suited for **decision-making problems** in **dynamic and uncertain environments**. It excels in cases where an agent must **learn through trial and error** and optimize its actions based on delayed rewards.

Situations Where RL Can Be Used

Pathfinding and Navigation

- Consider a **grid-based game** where a **robot** must navigate from a **starting node (E)** to a **goal node (G)** by choosing the optimal path.
- RL can learn the **best route** by exploring different paths and receiving feedback on their efficiency.

- In **obstacle-based games**, RL can identify **safe paths** while avoiding dangerous zones.

Dynamic Decision-Making with Uncertainty

- RL is useful in environments where **not all information is known upfront**.
- It is **not suitable** for tasks like **object detection**, where a classifier with complete labeled data performs better.

Characteristics of Reinforcement Learning

1. Sequential Decision-Making

- In RL, decisions are made in **steps**, and each step influences future choices.
- Example: In a **maze game**, a wrong turn can lead to failure.

2. Delayed Feedback

- Rewards are **not always immediate**; the agent may need to **take multiple steps** before receiving feedback.

3. Interdependence of Actions

- Each action affects the **next set of choices**, meaning an incorrect move can have long-term consequences.

4. Time-Related Decisions

- Actions are taken in a **specific sequence over time**, affecting the final outcome.

Challenges in Reinforcement Learning

Reward Design

- Setting the **right reward values** is crucial. **Incorrectly designed rewards** may lead the agent to learn undesired behavior.

Absence of a Fixed Model

- Some environments, like **chess**, have fixed rules, but many real-world problems **lack predefined models**.
- Example: Training a **self-driving car** requires **simulations** to generate experiences.

Partial Observability

- Some environments, like **weather prediction**, involve **uncertainty** because **complete state information** is unavailable.

High Computational Complexity

- Games like **Go** involve a **huge state space**, making RL training time-consuming.
- **More possible actions** → **More training time needed**.

Applications of Reinforcement Learning

1. Industrial Automation

- Optimizing **robot movements** for efficiency.

2. Resource Management

- Allocating resources in **data centers and cloud computing**.

3. Traffic Light Control

- Reducing **congestion** by adjusting signal timings dynamically.

4. Personalized Recommendation Systems

- Used in **news feeds, e-commerce, and streaming services** (e.g., **Netflix recommendations**).

5. Online Advertisement Bidding

- Optimizing **ad placements** for maximum engagement.

6. Autonomous Vehicles

- RL helps in training **self-driving cars** to navigate safely.

7. Game AI (Chess, Go, Dota 2, etc.)

- AI models like **AlphaGo** use RL to master complex games.

8. DeepMind Applications

- AI systems that **generate programs, images, and optimize machine learning models.**

Reinforcement Learning as Machine Learning

Reinforcement Learning (RL) is a distinct branch of **machine learning** that differs significantly from **supervised learning**.

While **supervised learning** depends on **labeled data**, **reinforcement learning** learns through **interaction with the environment**, making decisions based on **trial and error**.

Why RL Is Necessary?

Some tasks **cannot** be solved using **supervised learning** due to the **absence of a labeled training dataset**. For example:

- **Chess & Go:** There is no dataset with all possible game moves and their outcomes. RL allows the agent to **explore** and improve over time.
- **Autonomous Driving:** The car must **learn** from real-world experiences rather than relying on a fixed dataset.

Challenges in Reinforcement Learning Compared to Supervised Learning

- **More complex decision-making** since every action affects future outcomes.
- **Longer training times** due to trial-and-error learning.
- **Delayed rewards**, making it difficult to attribute success or failure to a specific action.

Differences between Supervised Learning and Reinforcement Learning

Reinforcement Learning	Supervised Learning
No supervisor and labelled dataset initially	Presence of supervisor and labelled data
Decisions are dependent and are made sequentially	Decisions are independent of each other and based on input given in the training phase
Feedback is not instantaneous and delayed by time	Usually, the feedback is instantaneous once the model is created
Agent action affects the next input data	Dependent of initial input or the input given at start
No target values, only goal oriented	Target class is predefined by the problem
Example: Chess, GO, Atari Games	Example: Classifiers

Components of Reinforcement Learning

Reinforcement Learning (RL) is based on an **agent** interacting with an **environment** to learn an optimal strategy through **trial and error**.

Basic Components of RL

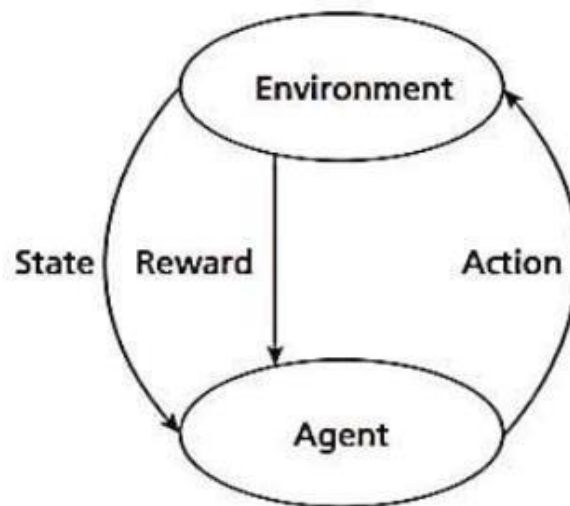


Figure 14.4: Basic Components of RL

1. **Agent** – The decision-maker (e.g., a robot, self-driving car, AI player in a game).
2. **Environment** – The external world where the agent interacts (e.g., a game board, real-world traffic).
3. **State (S)** – A representation of the environment at a specific time.
4. **Actions (A)** – The possible choices available to the agent.
5. **Rewards (R)** – The feedback signal received by the agent for taking an action.
6. **Policy (π)** – The agent's strategy for selecting actions based on states.
7. **Episodes** – The sequence of states, actions, and rewards from the start state to the goal state.

Types of RL Problems

Learning Problems

- **Unknown environment** – The agent learns by **trial and error**.
- **Goal** – Improve the policy through interaction.
- **Example** – A robot navigating through an unknown maze.

Planning Problems

- **Known environment** – The agent can compute and improve the policy using a model.
- **Example** – Chess AI that plans its moves based on game rules.

Environment and Agent

- The **environment** contains all elements the agent interacts with, including obstacles, rewards, and state transitions.
- The **agent** makes decisions and performs actions to maximize rewards.

Example

In **self-driving cars**,

- The **environment** includes roads, traffic, and signals.
- The **agent** is the AI system making driving decisions.

States and Actions

- **State (S)** – Represents the current situation.
- **Action (A)** – Causes a transition from one state to another.

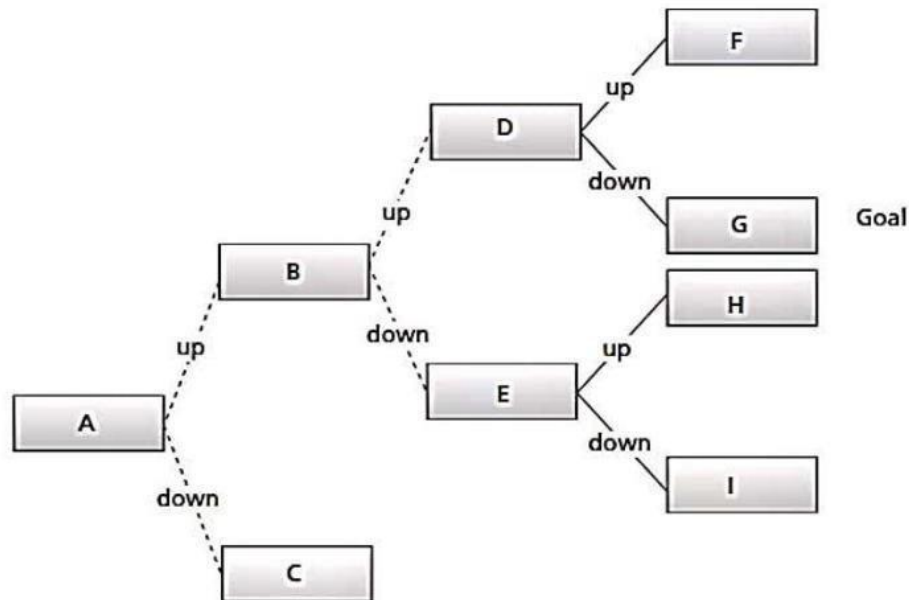


Figure 14.5: A Game: UP Action Results in Darker Line and Down Operation Results in Dotted Lines

Example (Navigation)

In a **grid-based game**, states represent **positions** (A, B, C, etc.), and actions are movements (**UP, DOWN, LEFT, RIGHT**).

Types of States

1. **Start State** – Where the agent begins.
2. **Goal State** – The target state with the highest reward.
3. **Non-terminal States** – Intermediate steps between start and goal.

Types of Episodes

- **Episodic** – Has a definite **start** and **goal state** (e.g., solving a maze).
- **Continuous** – No fixed goal state; the task continues indefinitely (e.g., stock trading).

Policies in RL

A **policy** (π) is the strategy used by the agent to choose actions. **Types of**

Policies

1. **Deterministic Policy** – Always maps a state to the same action:

$$a = \pi(s)$$

2. **Stochastic Policy** – Gives a probability of taking an action in a given state:

$$\pi(a|s) = P(A_t = a | S_t = s)$$

(Probability of choosing action a in state s).

Choosing the Best Policy

- The **optimal policy** is the one that **maximizes cumulative expected rewards**.

Rewards in RL

- **Immediate Reward (r)** – The instant feedback for an action.
- **Total Reward (G)** – The sum of all rewards collected during an episode.
- **Long-term Reward** – The cumulative future reward.

Discount Factor (γ)

To prioritize immediate rewards over distant ones, we use a discount factor (γ):

$$G = r_1 + \gamma r_2 + \gamma^2 r_3 + \dots$$

- $\gamma = 1 \rightarrow$ The agent considers all future rewards equally.
- $\gamma < 1 \rightarrow$ The agent prioritizes immediate rewards.
- Typical values: 0.8 or 0.9.

Example: Calculating Discounted Reward

If at step 5, reward = +1, and $\gamma = 0.7$, then:

$$(0.7)^5 \times 1 = 0.16807$$

RL Algorithm Categories

- **Model-Based RL** – Uses a predefined model (e.g., Chess AI).
- **Model-Free RL** – Learns by trial and error (e.g., a robot navigating an unknown environment).

Markov Decision Process

A **Markov Chain** is a stochastic process that satisfies the Markov property.

It consists of a sequence of random variables where the probability of transitioning to the next state depends only on the current state and not on the past states.

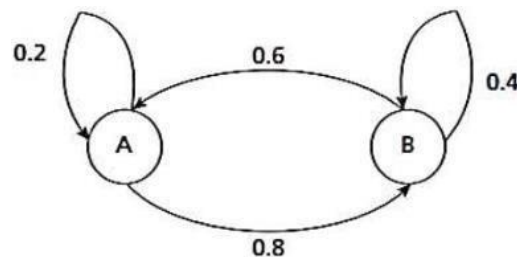


Figure 14.6: Markov Chain

Example: University Transition

Consider two universities:

- **80%** of students from University A move to University B for a master's degree, while **20%** remain in University A.
- **60%** of students from University B move to University A, while **40%** remain in University B.

This can be represented as a Markov Chain, where:

- States represent the universities.
- Edges denote the probability of transitioning between states.

A **transition matrix** at time is defined as:

Each row represents a probability distribution, meaning the sum of elements in each row equals **1**.

Probability Prediction

Let the initial distribution be:

$$u_0 = (0.6, 0.4)$$

To find the state distribution after one time step

$$u_1 = u_0 \times P = (0.6, 0.4) \times \begin{bmatrix} 0.2 & 0.8 \\ 0.4 & 0.6 \end{bmatrix} = (0.28, 0.72)$$

:

After two time steps:

$$\nu_2 = \nu_1 \times P = (0.28, 0.72) \times \begin{bmatrix} 0.2 & 0.8 \\ 0.4 & 0.6 \end{bmatrix} = (0.344, 0.656)$$

The system stabilizes over time, reflecting the equilibrium distribution.

Markov Decision Process (MDP)

An **MDP** extends a Markov Chain by incorporating rewards. It consists of:

1. **Set of states**
2. **Set of actions**
3. **Transition probability function**
4. **Reward function**
5. **Policy**
6. **Value function**

Markov Assumption

The Markov property states that the probability of reaching a state and receiving a reward depends only on the previous state and action :

$$P(s'|s, a) = P(s'|s, a, s_{t-1}, s_{t-2}, \dots)$$

MDP Process

1. **Observe** the current state .
2. **Choose an action** .
3. **Receive a reward** .
4. **Move to the next state** .
5. **Repeat to maximize cumulative rewards.**

State Transition Probability

The probability of moving from state to after taking action is given by:

$$P(s'|s, a)$$

This forms a **state transition matrix**, where each row represents transition probabilities from one state to another.

Expected Reward

The expected reward for an action in state is given by:

$$R(s, a) = E[r|s, a]$$

Training and Testing of RL Systems

Once an MDP is modeled, the system undergoes:

1. **Training:** The agent repeatedly interacts with the environment, adjusting parameters based on rewards.
2. **Inference:** A trained model is deployed to make decisions in real-time.
3. **Retraining:** When the environment changes, the model is retrained to adapt and improve performance.

Goal of MDP

The agent's objective is to maximize **total accumulated rewards** over time by following an optimal policy.

Multi-Arm Bandit Problem and Reinforcement Problem Types Reinforcement

Learning Overview

Reinforcement learning (RL) uses trial and error to learn a series of actions that maximize the total reward. RL consists of two fundamental sub-problems:

Prediction (Value Estimation):

- The goal is to predict the total reward (return), also known as **policy evaluation** or **value estimation**.
- This requires the formulation of a function called the **state-value function**.
- The estimation of the state-value function can be performed using **Temporal Difference (TD) Learning**.

Policy Improvement:

- The objective is to determine actions that maximize returns.
- This process is known as **policy improvement**.
- Both prediction and policy improvement can be combined into **policy iteration**, where these steps are used alternately to find an optimal policy.

Multi-Arm Bandit Problem

A commonly encountered problem in reinforcement learning is the **multi-arm bandit problem** (or **N-arm bandit problem**).

Consider a hypothetical casino with a robotic arm that activates a **5-armed slot machine**. When a lever is pulled, the machine returns a reward within a specified range (e.g., \$1 to \$10).

The challenge is that each arm provides rewards randomly within this range.

Objective:

Given a limited number of attempts, the goal is to maximize the total reward by selecting the best lever.

A logical approach is to determine which lever has the highest **average reward** and use it repeatedly.

Formalization:

Given **k** attempts on an **N-arm slot machine**, with rewards , the expected reward (action- value function) is:

$$Q(a) = \frac{1}{k} \sum_{i=1}^k r_i$$

The best action is defined as:

$$a^* = \arg \max Q(a)$$

This indicates the **action that returns the highest average reward** and is used as an indicator of action quality.

Example:

If a slot machine is chosen five times and returns rewards , the quality of this action is:

$$Q(a) = \frac{1 + 2 + 0 + 7 + 9}{5} = 3.8$$

Exploration vs Exploitation and Selection Policies

In reinforcement learning, an agent must decide how to select actions:

Exploration:

- Tries all actions, even if they lead to sub-optimal decisions.
- Useful in games where exploring different actions provides better long-term rewards.
- Risky but informative.

Exploitation:

- Uses the **current best-known action** repeatedly.
- Focuses on **short-term gains**.
- Simple but often sub-optimal.

A balance between exploration and exploitation is crucial for optimal decision-making.

Selection Policies

Greedy Method

- Picks the best-known action at any given time.
- Based solely on exploitation.
- Risk: It may miss out on exploring better options. ϵ -

Greedy Method

- Balances exploration and exploitation.
- With probability ϵ , the agent explores a random action.
- With probability $1 - \epsilon$, it selects the best-known action.
- ϵ ranges from 0 to 1 (e.g., $\epsilon = 0.1$ means a 10% chance of exploration).

ϵ – Greedy Method

This method combines both exploration and exploitation. In ϵ – greedy, ϵ value ranges from 0 to 1 (refer to Algorithm 14.1).

Algorithm 14.1: ϵ - Greedy

Step 1: Generate a random number p in the range 0 – 1.

Step 2: If $p \geq \epsilon$, the algorithm chooses the action with highest reward value.

Step 3: If $p < \epsilon$, actions are chosen randomly.

Step 4: Update the value of action and repeat for all actions.

With probability ϵ , the action a is chosen randomly, but rest of the time with probability $1 - \epsilon$, the best actions based on greedy approach are selected. Thus, there is a balance of exploration as well as exploitation. For example, if ϵ is 0.2, then 80% best actions are chosen and 20% random actions are chosen.

Reinforcement Learning Agent Types

An RL agent can be classified into different approaches based on how it learns:

1. Value-Based Approaches

- Optimize the value function, which represents the maximum expected future reward from a given state.
- Uses **discount factors** to prioritize future rewards.

2. Policy-Based Approaches

- Focus on finding the **optimal policy**, a function that maps states to actions.
- Rather than estimating values, it directly learns which action to take.

3. Actor-Critic Methods (Hybrid Approaches)

- Combine value-based and policy-based methods.
- The **actor** updates the policy, while the **critic** evaluates it.

4. Model-Based Approaches

- Create a model of the environment (e.g., using **Markov Decision Processes (MDPs)**).
- Use simulations to plan the best actions.

5. Model-Free Approaches

- No predefined model of the environment.
- Use methods like **Temporal Differencing (TD) Learning** and **Monte Carlo methods** to estimate values from experience.

Reinforcement Algorithm Selection

The choice of a reinforcement learning algorithm depends on factors such as:

- **Availability of models**
- **Nature of updates** (incremental vs. batch learning)
- **Exploration vs. exploitation trade-offs**
- **Computational efficiency**

Reinforcement Algorithm Selection

Reinforcement agent algorithms are decided based on the criteria of availability of models, environment and nature of updates. The agent algorithm selections are shown in Table 14.3.

Table 14.3: Algorithm Selection

Model	Environment	On/off	Algorithms Used
Present	Static	Off	Dynamic Programming
Absent	Dynamic	On	SARSA
Absent	Dynamic	-	Temporal Learning, Monte Carlo Methods
Absent	Dynamic	Off	Q-Learning

Model-based Learning

Passive Learning refers to a **model-based environment**, where the environment is **known**. This means that for any given state, the next state and action probability distribution are known.

Markov Decision Process (MDP) and **Dynamic Programming** are powerful tools for solving reinforcement learning problems in this context.

The mathematical foundation for passive learning is provided by **MDP**. These model- based reinforcement learning problems can be solved using **dynamic programming** after constructing the model with MDP.

The primary objective in reinforcement learning is to take an action **a** that transitions the system from the current state to the end state while maximizing rewards. These rewards can be positive or negative.

The goal is to maximize expected rewards by choosing the **optimal policy**: for all possible values of **s** at time **t**.

Policy and Value Functions

An **agent** in reinforcement learning has multiple courses of action for a given state. The way the agent behaves is determined by its **policy**.

A **policy** is a distribution over all possible actions with probabilities assigned to each action.

Different actions yield different rewards. To quantify and compare these rewards, we use **value functions**.

Value Function Notation

A **value function** summarizes possible future scenarios by averaging expected returns under a given policy π .

It is a prediction of future rewards and computes the expected sum of future rewards for a given state **s** under policy π :

$$v(s) = E[G_t | S_t = s]$$

where $\mathbf{v(s)}$ represents the **quality of the state** based on a long-term strategy. **Example**

If we have two states with values **0.2** and **0.9**, the state with **0.9** is a better state to be in. Value

functions can be of two types:

- **State-Value Function** (for a state)
- **State-Action Function** (for a state-action pair)

State-Value Function

Denoted as $\mathbf{v(s)}$, the **state-value function** of an MDP is the expected return from state $\mathbf{s}$ under a policy π :

This function accumulates all expected rewards, potentially discounted over time, and helps determine the **goodness of a state**.

The **optimal state-value function** is given by:

$$v_{\pi}(s) = E_{\pi}[G_t | S_t = s]$$

Action-Value Function (Q-Function)

Apart from $\mathbf{v(s)}$, another function called the **Q-function** is used. This function returns a real value indicating the total expected reward when an agent:

1. Starts in state $\mathbf{s}$
2. Takes action $\mathbf{a}$
3. Follows a policy π afterward

It is denoted as:

$$Q_{\pi}(s, a) = E[G_t | S_t = s, A_t = a]$$

The optimal Q-value function is given by:

$$v^*(s) = \max_a Q_{\pi}(s, a)$$

Bellman Equation

Dynamic programming methods require a **recursive formulation** of the problem. The recursive formulation of the **state-value function** is given by the **Bellman equation**:

$$v(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R_{s,a} + \gamma v(s')]$$

where:

- $P(s' | s, a)$: Probability of transitioning to s' from s after taking action a
- $R(s, a)$: Reward received after taking action a in state s
- γ : Discount factor ($0 \leq \gamma \leq 1$) controlling the weight of future rewards

Solving Reinforcement Problems

There are two main algorithms for solving reinforcement learning problems using conventional methods:

1. **Value Iteration**
2. **Policy Iteration**

Value Iteration

Value iteration estimates $v(s)$ iteratively:

$$v_{t+1}(s) = \max_a \sum_{s'} P(s'|s, a) [R_{s,a} + \gamma v_t(s')]$$

Algorithm

1. Initialize $\mathbf{v}(\mathbf{s})$ arbitrarily (e.g., all zeros).
2. Iterate until convergence:
 - For each state $\mathbf{s}$, update $\mathbf{v}(\mathbf{s})$ using the Bellman equation.
 - Repeat until changes are negligible.

Policy Iteration

Policy iteration consists of **two main steps**:

1. **Policy Evaluation**
2. **Policy Improvement**

Policy Evaluation

Initially, for a given policy π , the algorithm starts with $\mathbf{v}(\mathbf{s}) = \mathbf{0}$ (no reward). The Bellman equation is used to obtain $\mathbf{v}(\mathbf{s})$, and the process continues iteratively until the optimal $\mathbf{v}(\mathbf{s})$ is found.

Policy Improvement

The policy improvement process is performed as follows:

1. Evaluate the current policy using policy evaluation.
2. Solve the Bellman equation for the current policy to obtain $\mathbf{v}(\mathbf{s})$.
3. Improve the policy by applying the **greedy approach** to maximize expected rewards.
4. Repeat the process until the policy converges to the optimal policy.

Algorithm

1. Start with an arbitrary policy π .
2. Perform **policy evaluation** using Bellman's equation.

3. Improve the policy greedily.
4. Repeat until convergence.

Model Free Methods

Model-free methods do not require complete knowledge of the environment. Instead, they learn through experience and interaction with the environment.

The reward determination in model-free methods can be categorized into three formulations:

1. **Episodic Formulation:** Rewards are assigned based on the outcome of an entire episode. For example, if a game is won, all actions in the episode receive a positive reward (+1). If lost, all actions receive a negative reward (-1). However, this approach may unfairly penalize or reward intermediate actions.
2. **Continuous Formulation:** Rewards are determined immediately after an action. An example is the multi-armed bandit problem, where an immediate reward between \$1 - \$10 can be given after each action.
3. **Discounted Returns:** Long-term rewards are considered using a discount factor. This method is often used in reinforcement learning algorithms.

Model-free methods primarily utilize the following techniques:

- **Monte Carlo (MC) Methods**
- **Temporal Difference (TD) Learning**

Monte-Carlo Methods

Monte Carlo (MC) methods do not assume any predefined model, making them purely experience-driven. This approach is analogous to how humans and animals learn from interactions with their environment.

Algorithm 14.5: MC

Step 1: For a policy π , select state s .

Step 2: Generate an episode.

Step 3: Compute average returns for every first occurrence of state s in an episode.

Step 4: Update the returns and start a new episode. Repeat for all episodes.

Characteristics of Monte Carlo Methods:

- Experience is divided into **episodes**, where each episode is a sequence of states from a starting state to a goal state.
- **Episodes must terminate**; regardless of the starting point, an episode must reach an endpoint.
- **Value-action functions** are computed only after the completion of an episode, making MC an **incremental** method.
- MC methods compute rewards at the end of an episode to estimate maximum expected future rewards.
- **Empirical mean** is used instead of expected return; the total return over multiple episodes is averaged.
- Due to the non-stationary nature of environments, value functions are computed for a fixed policy and revised using **dynamic programming**.

Monte Carlo Mean Value Computation:

The mean value of a state is calculated as:

$$V(s_i) = \frac{1}{N} \sum G_i$$

Incremental Monte Carlo Update:

The value function is updated incrementally using the following formula:

$$V(s_i) = V(s_i) + \alpha(G_i - V(s_i))$$

where α is the learning rate.

Temporal Difference (TD) Learning

Temporal Difference (TD) Learning is an alternative to Monte Carlo methods. It is also a model-free technique that learns from experience and interaction with the environment.

Characteristics of TD Learning:

- **Bootstrapping Method:** Updates are based on the current estimate and future reward.
- **Incremental Updates:** Unlike MC, which waits until the end of an episode, TD updates values at each step.
- **More Efficient:** TD can learn before an episode ends, making it more sample-efficient than MC methods.
- **Used for Non-Stationary Problems:** Suitable for environments where conditions change over time.

Differences between Monte Carlo and TD Learning

Feature	Monte Carlo (MC)	Temporal Difference (TD)
Update Timing	End of episode	After each step
Learning Type	Experience-based	Bootstrapping
Computational Efficiency	Less efficient	More efficient
Suitability	Static environments	Dynamic environments

Monte-Carlo Method	Temporal Difference Method
It is easy to understand and does not exploit Markov principle	Exploits Markov principle
This requires termination and episode	Does not require termination

Eligibility Traces and TD(λ)

TD Learning can be accelerated using **eligibility traces**, which allow updates to be spread over multiple states. This leads to a family of algorithms called **TD(λ)**, where λ is the decay parameter ($0 \leq \lambda \leq 1$):

- $\lambda = 0$: Only the previous prediction is updated.
- $\lambda = 1$: All previous predictions are updated.

By incorporating eligibility traces, TD(λ) provides an alternative short-term memory mechanism to enhance learning efficiency.

Algorithm 14.6: TD-Learning

1. Initialize policy π .
2. Set $v(s) = 0$ for all states $s \in S$.
3. Repeat for each episode.
4. Initialize s .
5. Repeat for each step of episode.
6. Choose action ' a ' and execute it. Observe the new state and new reward.
7. Update value using Bellman Eq. (14.22).
8. Update as s' .
9. Until s is terminal state.

Q-Learning

Q-Learning Algorithm

1. **Initialize Q-table:**

- Create a table $Q(s,a)$ with states s and actions a .
- Initialize Q-values with random or zero values.

2. **Set parameters:**

- Learning rate α (typically between 0 and 1).
- Discount factor γ (typically close to 1).
- Exploration-exploitation trade off strategy (e.g., ϵ -greedy policy).

3. **Repeat for each episode:**

- Start from an initial state s .
- Repeat until reaching a terminal state:

1. Choose an action a using an exploration strategy (e.g., ϵ -greedy).

2. Perform action a , observe new state s' and reward r .

3. Update Q-value using the **Bellman equation**:

$$Q(s, a) = Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

4. Set $s = s'$.

4. **End the training** once convergence is reached (Q-values become stable).

This iterative process helps the agent learn optimal Q-values, which guide it to take actions that maximize rewards.

Algorithm 14.7: Q-Learning

1. Set $Q(s, a) = 0$ where s_t is the terminal node in an episode.
2. For all states s and action a , set Q Value = 0.
3. Repeat.
4. Select s_t randomly.
5. Choose action a_t from Q using ϵ -greedy.
6. Perform action a_t such that $r(s_t, a_t) > 0$ and reach the next state.
7. Update the action-value function using TD using Eq. (14.24) and Bellman equation:

$$Q_t(s_t, a_t) = Q_{t-1}(s_t, a_t) + \alpha \times TD_t(s_t, a_t) \quad (14.25)$$

Until the terminal state s is reached.

Here, α is the learning rate. It is between 0 and 1. If α is zero, then nothing is updated and there is no learning at all. When setting it to a higher value such as 0.9, the learning happens fast. γ is the discount factor. It is in the range 0 – 1. It should be less so that the algorithm can converge.

The inference is simple, the best action is the maximum of Q function as follows:

$$a_t = \arg \max_a (Q(s_t, a))$$

SARSA Learning

SARSA Algorithm (State-Action-Reward-State-Action)

Algorithm 14.8: SARSA

1. For all states and actions, set $Q(s, a) = 0$.
2. Initialize α , ϵ and γ .
Initialize the state s_t .
Choose action a_t based on selection policy ϵ -greedy strategy and execute it.
Get a new state s_{t+1} , observe the reward r_{t+1} .
Choose action a_{t+1} from Q based on ϵ -Greedy procedure and execute it.
Update Action-value function of Eq. (14.28).
3. Update the values $s_t = s_{t+1}$, $a_t = a_{t+1}$.
4. If s' is terminal node, then start a new episode else repeat above steps.

Initialize Q-table:

- Create a table $Q(s,a)$ for all state-action pairs.

- Initialize Q-values with random or zero values.

Set parameters:

- Learning rate α (typically between 0 and 1).
- Discount factor γ (typically close to 1).
- Exploration-exploitation strategy (e.g., ϵ -greedy policy).

Repeat for each episode:

- Start from an initial state s .
- Choose an action a using the ϵ -greedy policy.

Repeat until the terminal state is reached:

1. Take action a , observe reward r and next state s' .
2. Choose next action a' from s' using ϵ -greedy policy.
3. Update Q-value using the **SARSA update rule**:

$$Q(s, a) = Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)]$$

4. Set $s = s'$ and $a = a'$.

End the training when Q-values converge. **Differences**

between SARSA and Q-Learning

Feature	Q-Learning (Off-Policy)	SARSA (On-Policy)
Policy	Learns from the best possible actions (greedy)	Learns from actions taken (ϵ -greedy)
Update Rule	Uses maximum future Q-value: $r + \gamma \max Q(s', a')$	Uses estimated future Q-value: $r + \gamma Q(s', a')$
Exploration	More aggressive; optimizes for long-term rewards	More conservative; balances risk and reward